UAV-NOMA Disaster Network Planning Simulator

Project Book

A simulation-based planning tool for drone base stations using NOMA in disaster areas

Generated 21 October 2026 from docs/book/ (the Markdown chapters are the source; this PDF is rebuilt after every change)

Contents

The UAV-NOMA Project Book 5

Chapters . . . 5

The project in five sentences . . . 5

Chapter 1. The project 6

1.1 The problem . . . 6

1.2 Where the idea came from . . . 6

1.3 The research question and the hypotheses . . . 6

1.4 The gates . . . 6

1.5 The team (from the task documents) . . . 7

1.6 What "finished" means . . . 7

1.7 The rule about the AI . . . 7

Chapter 2. Journal 8

2026-10-14 . . . 8

2026-10-15 . . . 11

2026-10-16 . . . 13

2026-10-19 . . . 13

2026-10-21 . . . 20

Chapter 3. Decision log 23

Chapter 4. The simulator 31

4.1 Scenarios and seeds . . . 31

4.2 People . . . 31

4.3 Movement . . . 32

4.4 Collapsed buildings (obstacles) . . . 32

4.5 The radio channel (air to ground) . . . 32

4.6 NOMA (and the OMA reference) . . . 32

4.7 Energy and the battery . . . 33

4.8 The drone position sweep (dataset) . . . 34

4.9 What the simulator does not model yet . . . 34

Chapter 5. The dataset 35

5.1 How to regenerate it . . . 35

5.2 Versions . . . 35

5.3 Files and splits . . . 35

5.4 Columns . . . 35

5.5 What the workbook contains (read back on 2026-10-15) . . . 36

5.6 Key charts . . . 36

5.7 Is it enough data? . . . 37

Chapter 6. The game and the AI 38

6.1 The game (Gymnasium environment) . . . 38

6.2 How the AI controls the drone (wrappers) . . . 39

6.3 The AI's brain . . . 39
6.4 Training (rl_train.py) . . . 39
6.5 Evaluation and pictures . . . 40
6.6 Curriculum (from the plan) . . . 40
Chapter 7. Baselines and their results **41**
7.1 The simple pilots . . . 41
7.2 How results are measured . . . 41
7.3 What the results say (summary) . . . 41
7.4 Default scenarios, 10 Wh (100 scenarios, seeds 0-99) . . . 43
7.5 Default scenarios, 16 Wh (100 scenarios) . . . 44
7.6 Profile lowpower (50 scenarios, 10 Wh) . . . 45
7.7 Profile bigarea (50 scenarios, 10 Wh) . . . 45
7.8 Profile hard, all 7 pilots (100 scenarios, 16 Wh) . . . 46
7.9 Still crowd rl_static, validation seeds 100-119 (reference for the AI) . . . 47
7.10 The circling pilot B7: does circling save battery? (plan step 5, 2026-10-21) . . . 47
7.11 How much could an AI that anticipates gain? The clairvoyant pilot (plan step 3, 2026-10-21) . . . 48
Chapter 8. AI experiments **50**
8.1 The task and the bar (Gate G4) . . . 50
8.2 All runs . . . 50
8.3 What each step taught us . . . 50
8.4 Status and what happens next . . . 51
8.7 Why the AI stalls: the information ladder and the train/validation check (plan step 1) . . . 51
8.5 Validation table: goto_hold10 . . . 53
8.6 Validation table: E3 e3_cnn_full (best so far) . . . 53
8.8 Track 2: a better learner, and the G4 verdict (2026-10-21) . . . 54
Chapter 9. Engineering **57**
9.1 Repository . . . 57
9.2 How work flows (D-12) . . . 57
9.3 Continuous integration (.github/workflows/ci.yml) . . . 57
9.4 Tests (315 on 2026-10-21) . . . 58
9.5 Pull requests . . . 58
9.6 Software versions (owner's laptop) . . . 59
9.7 Commands . . . 59
9.8 Troubleshooting (things that actually happened) . . . 59
Chapter 10. What is verified and what is not **60**
10.1 Status labels . . . 60
10.2 Summary . . . 60
10.3 Other checks done . . . 60
10.4 Not done yet . . . 61
10.5 What the results may and may not claim . . . 61
Chapter 11. Outlook **62**

11.1 Roadmap (agreed 2026-10-19, full version in docs/roadmap.md) . . . 62
11.2 Is it novel? (first literature check, 2026-10-19) . . . 62
11.3 The real-world gap . . . 62
11.4 Hardware path (outside this project, in stages) . . . 63
11.5 Business track . . . 63
11.6 Open decisions for the owner . . . 63
Chapter 12. Glossary **65**
Chapter 13. NOMA in this project **67**
13.1 What NOMA is . . . 67
13.2 Where NOMA sits in the code . . . 67
13.3 NOMA in the dataset (all drone positions, from the sweep) . . . 67
13.4 NOMA measured inside the game (hypothesis H3) . . . 68
13.5 What the measurements say . . . 69
13.6 Why NOMA loses coverage at full power: the 232 kbps ceiling . . . 70
13.7 Pictures . . . 70
13.8 Next steps for NOMA (roadmap stage B) . . . 70
13.9 Adaptive NOMA vs equally optimised OMA (Track 1, 2026-10-21) . . . 71
13.10 How to explain it to an examiner . . . 73
Chapter 14. How a senior communications and AI engineer would solve this **74**
14.1 Reading the evidence first . . . 74
14.2 The principle: separate the problem by time scale . . . 74
14.3 The plan, in priority order . . . 74
14.4 What this changes in the project story . . . 76
14.5 The independent review . . . 76
14.6 The agreed plan (draft and review combined), in order . . . 77
14.7 Decisions needed from the owner . . . 77
14.8 Progress . . . 77
Appendix. Figure gallery **79**

The UAV-NOMA Project Book

This book records everything the project has done, from the first run of the dataset generator to the latest experiment: what was built, why each decision was taken, what came out (with the real numbers), what went wrong, and what is still open. It is written so that a new team member, the guide, or an examiner can follow the whole story without the chat history.

PDF version. `UAV-NOMA-Project-Book.pdf`, rebuilt with `python docs/book/build_pdf.py`.

How it is kept up to date. After every step of work (a new feature, an experiment, a decision, a merge) the relevant chapters are updated in the same pull request: the journal gets a dated entry, the decision log gets a new numbered decision, and results go into the results chapters with their exact numbers. Old entries are never rewritten to look better: if a result turned out wrong, a later entry says so.

Reading guide. Each technical idea is explained twice: first in plain words, then in the scientific terms you will need in the report and the viva. Chapter 12 is a glossary of every term.

Chapters

#	Chapter	What is in it
1	The project	The problem, the idea, the hypotheses, the gates, the team, what "done" means
2	Journal	Day-by-day record of everything we did, with outputs
3	Decision log	Every decision: the question, the options, what was chosen, by whom, and why
4	The simulator	The physics: crowds, movement, buildings, the radio channel, NOMA, energy
5	The dataset	The generated tables, the Excel file, what is in every column, versions
6	The game and the AI	The Gymnasium environment, the controls, the reward, the training pipeline
7	Baselines and their results	The seven simple pilots and every comparison table
8	AI experiments	Every training run, what it showed, and the diagnosis of each failure
9	Engineering	Repository, branches, pull requests, tests, CI, every command, troubleshooting
10	What is verified and what is not	Parameters checked against papers, assumptions, known gaps
11	Outlook	Roadmap, novelty and related work, the real-world gap, hardware path, business track
12	Glossary	Every term in plain words
13	**NOMA in this project**	What NOMA is, where it sits, NOMA vs OMA measured in the game, its limits and next steps
14	A senior engineer's plan, reviewed	How an expert would solve the problem, an independent review, the agreed next steps

Start here for NOMA: Chapter 13.

Figures are in `figures/` and animations in `figures/gifs/`. Each caption says which dataset version or run produced it.

The project in five sentences

After a disaster, phone towers are down; a drone carrying a radio can act as a temporary tower. We built a simulator of survivors in crowds, collapsed buildings, a battery-limited drone and a NOMA radio link, so that strategies for flying the drone can be compared in software before any real flight. We built seven simple, non-AI pilots and compared them fairly on hundreds of scenarios. We are training an AI pilot that learns by itself, and we report honestly where it wins and where it loses. The project is a simulation-based planning tool, not a digital twin and not a real flight system (yet).

Chapter 1. The project

1.1 The problem

In plain words. When an earthquake or flood hits, the mobile phone towers in the area often stop working: no power, broken cables, collapsed buildings. Survivors cannot call for help and rescue teams cannot coordinate. A drone carrying a small mobile base station can fly over the area and act as a temporary tower. But a drone has a small battery, the signal is blocked by buildings, people move around, and one drone has to serve many people at once. Where should it fly, how high, and how strongly should it transmit?

Technically. A UAV-mounted aerial base station (UAV-BS) restores downlink coverage in a disaster area. Users are spatially clustered and mobile, the air-to-ground channel has a line-of-sight probability that depends on the elevation angle, the UAV is energy-constrained (rotary-wing propulsion dominates consumption), and the downlink uses non-orthogonal multiple access (NOMA) so that pairs of users share one frequency resource. The control problem is the joint choice of 3D position and transmit power over time.

1.2 Where the idea came from

- **The guide's idea:** a moving base station using NOMA.
- **The team's extension** (Srivathsa's pitch): add moving survivor crowds, collapsed buildings, a battery limit, and a reinforcement-learning (RL) controller, then compare it fairly with strong simple strategies. The full plan is in `ARCHITECTURE.md` and `PROJECT_PLAN.md` (shared as PDFs at the start, summarised in `CLAUDE.md`).
- **Positioning.** A *simulation-based planning tool*: rescue planners can test drone strategies in software before flying real hardware, saving battery, hardware and flight time. It may only be called a "digital twin" (and only "digital-twin-style") if it is fed real or replayed state data (plan section 11). Until then the wording is "simulation-based planning tool".

1.3 The research question and the hypotheses

Research question: can a crowd-aware RL controller adapt the drone's position, altitude and transmit power to shifting survivor demand better than static and heuristic strategies, under a battery limit?

Every experiment must serve one of these hypotheses:

ID	Hypothesis	In plain words
H1	Adaptive UAV control beats a fixed UAV once crowds move	A drone that follows people does better than one parked in the middle
H2	RL beats strong heuristics (k-means B2, grid search B3, PSO B4) only when dynamics and battery limits make heuristics short-sighted; report where it wins and where it doesn't	The AI should only win in hard, changing situations, and we must say where it loses
H3	NOMA pairing improves throughput and fairness over OMA at equal bandwidth and power	Sharing a frequency between two people (NOMA) beats giving everyone a separate slice (OMA)
H4 (stretch)	Coordinated multi-UAV control beats independent or centroid placement	Several drones working together beat drones acting alone

1.4 The gates

The plan forbids starting a layer before the previous one passes its gate:

Gate	Criterion	Status (2026-10-19)
G1 Simulator core	Python matches MATLAB reference vectors; unit tests green; one scenario renders	Tests green and scenarios render; **MATLAB parity not done** (needs the MATLAB reference numbers)

Gate	Criterion	Status (2026-10-19)
G2 Physics	Altitude optimum reproduced; energy model sanity tests; NOMA feasibility logic tested	**Met** (Chapter 4, 10)
G3 Baselines	All baselines run on 100+ seeded scenarios; reproducible comparison table	**Met** (Chapter 7)
G4 Single-agent RL	PPO learns beyond random on the static scenario and matches or beats B2 there	**Half met**: beats random, not B2 (Chapter 8)
G5 Dynamic RL	Evidence on at least one dynamic scenario, positive or negative, with 5 seeds	Not started
G6 Multi-UAV	G5 stable; 2-UAV environment passes tests	Not started
G7 Freeze	Feature freeze at month 10	Not reached

1.5 The team (from the task documents)

Person	Role	Owns	Backup
Srivathsa	Wireless physics, project owner, repo owner and reviewer	physics, energy, MATLAB parity, assumptions, API	Sagar
Sagar	Simulation	world, environment, scenarios, obstacles, mobility	Srivathsa
Arani	Baselines and experiments	baselines, experiment runner, statistics, plots, CI	Harshita
Harshita	RL and the real-data link	RL training, trace replay, state sync	Arani

Rules from the plan: every module has a primary and a backup who can explain it; nobody merges code they cannot explain; the viva rule is "no 'that is Sagar's part'". Much of the simulator, baselines and RL pipeline described in this book was built in an AI-assisted session with the owner before the team started; the team's tasks are being re-scoped to *own, review and extend* this code (see the decision log, D-31).

1.6 What "finished" means

The plan's deliverables: literature matrix (verified citations), assumptions table with sources, validated simulator with test suite and MATLAB parity report, baseline suite, Gymnasium environments (single and multi-UAV), trained models with logs, experiment scripts reproducing every figure, dashboard and API, report or paper draft, demo video, viva slides and a 30-second pitch. The roadmap from today to there is in Chapter 11 and `docs/roadmap.md`.

1.7 The rule about the AI

Decided by the owner on 2026-10-19 (decision D-24): **the AI learns by itself.** It receives only raw measurements that a real drone could have and the coverage score. No hints, no menu of candidate spots, no imitation of the baselines, no privileged information. Allowed: the network design, training length, hyperparameters, seeds and the curriculum order.

Chapter 2. Journal

A dated record of everything done, in order, with the real outputs. Decisions are numbered **D-xx** and explained in Chapter 3. Pull requests are **#n** (Chapter 9). Numbers are copied from the run outputs; "val8" means the score on the first 8 validation scenarios printed during training, "val20" the official score on all 20 validation scenarios.

2026-10-14

1. First run of the dataset generator

The folder `uav_dataset_generator/uav_dataset/` contained `make_dataset.py`, `config.yaml`, a README and an earlier `data/` folder.

- `python make_dataset.py all` failed: `ModuleNotFoundError: No module named 'matplotlib'`.
- Installed the README's packages (`pip install numpy pandas matplotlib pyyaml`): matplotlib 3.11.2, pandas 3.0.6, pyyaml 6.0.3.
- Second run succeeded:
  - users tables: train 20,000 rows, val 4,000, test 4,000 (200 users x 100/20/20 scenarios);
  - sweep tables: train 387,200 rows, val 77,440, test 77,440;
  - pictures `users_preview.png` and `altitude_check.png`.

2. The owner shared the plan

`ARCHITECTURE.md` (system design: crowd generator, mobility, obstacles, UAV and energy model, A2G channel, NOMA engine, metrics, Gymnasium env, baselines, RL, FastAPI backend, React dashboard) and `PROJECT_PLAN.md` (pitch, objectives, hypotheses H1-H4, scope tiers, gates G1-G7, roles, month-by-month phases, experiments E1-E14, viva questions, deliverables). Both are summarised in Chapter 1 and `CLAUDE.md`.

3. Explanations given to the owner

What the project is, what RL is (agent, state, action, reward, policy, PPO vs SAC), whether a business is possible, whether NOMA is present (yes: superposition, SIC, feasibility check, OMA reference in `noma_rates`), why there are six CSV files (two tables x three splits) and what every column means.

4. Review of the owner's parameter specification

The owner pasted a full parameter list (scenario, UAV, energy, channel, NOMA, sweep). Review findings:

1. A 50-100 Wh battery never runs out in 300-600 s episodes (hover is about 170 W), so the battery would not matter.
2. A 10-20 m/s drone is 10-20 times faster than walking crowds, so following a crowd is trivial.
3. A 25 m grid sweep with 30 altitudes, 5 powers and 50 seeds is about 12-19 million rows: too big for Excel and slow.
4. With 1,000 users on 10 MHz each user gets about 10 kHz: almost everyone is in outage.
5. Moving crowds need a time column; obstacles need their own table.
6. RL does not train on this dataset; the environment generates its own experience. Decision: go phase by phase (D-01).

5. Phase 1: make the existing slice trustworthy (plan approved)

1. **Pictures checked.** `users_preview.png` shows clustered crowds plus scattered users with rescue users circled. `altitude_check.png` shows a coverage peak at 0 dBm near 250 m: **the altitude optimum (Gate G2) is reproduced** on dataset v1.
2. **NOMA vs OMA** on `sweep_train.csv` (v1, 100 scenarios):

Power	NOMA / OMA sum rate	Rows where NOMA wins	Coverage NOMA / OMA	Fairness NOMA
0 dBm	0.894	5.9%	0.081 / 0.104	0.525
10 dBm	1.097	64.9%	0.456 / 0.447	0.658
20 dBm	1.170	97.3%	0.838 / 0.793	0.772
30 dBm	1.152	100%	0.994 / 0.994	0.766

NOMA gives 10-17% more total rate from 10 dBm up and loses at 0 dBm (interference-limited weak user at low SNR; plausible physics, not proven). Figure: noma_vs_oma_v1.png.

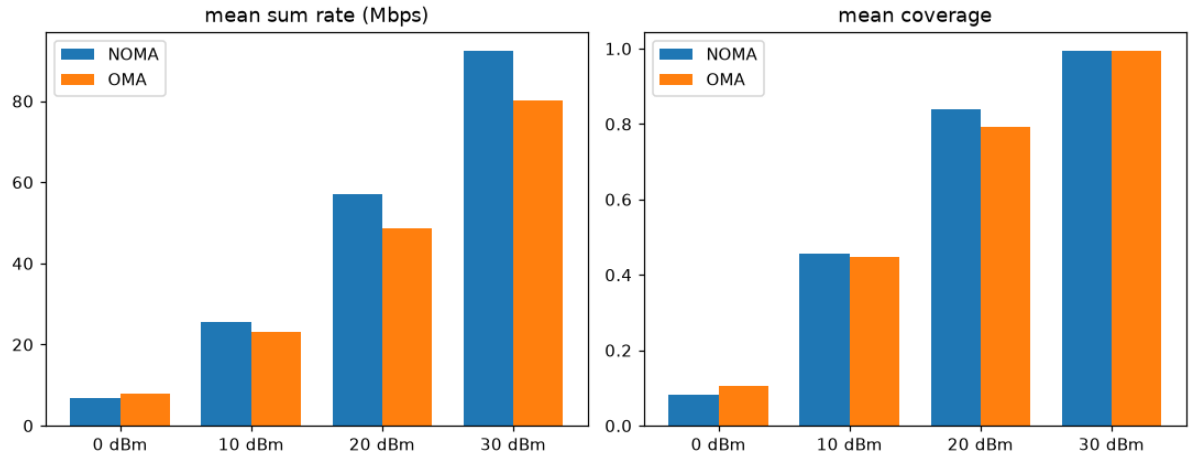


Figure: noma_vs_oma_v1.png

3. **NOMA unit tests** (`test_noma.py`, 5 tests, all pass; pytest installed).
4. **One Excel file** (`export_excel.py`, xlsxwriter installed): `uav_noma_dataset.xlsx` with sheets `users` (28,000 rows) and `sweep` (542,080 rows), a `split` column, 46 MB. Row counts confirmed from the sheet dimensions in the file (D-02).
5. `CLAUDE.md` written for the project root.

Observations: `energy_j_per_step` was the same constant (168.5 J) in every row; at 0 dBm OMA had a higher sum rate than NOMA.

6. "Is this enough data?"

Answer: count scenarios, not rows (140). Enough for the altitude and NOMA-vs-OMA charts and for Gate G3 (100+ seeded scenarios); not relevant for RL (the environment generates experience); not enough for generalisation (every scenario had 200 users and 3 clusters); no moving crowds, battery or obstacles yet. The fixes were mapped onto phases 2-5.

7. Phase 2: configurable scenarios (dataset v2)

Owner decisions: **bandwidth scales with users** (D-03) and **the spec's class mix** (D-04).

- `data/` copied to `data_v1/` before regenerating.
- Each seed now draws its own number of users (100, 300, 600 or 1,000), clusters (2-5), cluster radius (50-200 m) and scatter share (5-10%). Five user classes with their own minimum rates. Bandwidth = users x 50 kHz. Pairing `extreme` or `random`. The SIC check became its own function `sic_feasible`. 9 tests pass.

- **Correction of an earlier claim:** random pairing does not make SIC fail. When the "weak" and "strong" roles are assigned by channel gain, SIC is always feasible; a failure needs imperfect SIC or need-based rules. The test now checks the function with a deliberately bad pair.
- **Results (v2, train):** NOMA/OMA sum-rate ratio 0.800 / 1.008 / 1.144 / 1.166 at 0 / 10 / 20 / 30 dBm; coverage NOMA vs OMA 0.146/0.167, 0.409/0.410, 0.682/0.654, 0.847/0.816. By user count the ratio stays 1.13-1.15 (1.145 at 100 users, 1.127 at 300, 600 and 1,000), so NOMA's gain does not shrink with more users.
- **The altitude peak disappeared** from the mixed chart: coverage kept rising to 500 m (best altitude 400 m in 54 scenarios, 500 m in 44, 300 m in 2). Owner approved: extend altitudes to 1,000 m and check per class (D-05).
- **Per-class check** (`altitude_by_class.py`, 20 scenarios): an optimum exists for every class. Best altitude:

Class (min rate)	0 dBm	10 dBm
iot (10 kbps)	500 m	600 m
voice (50 kbps)	200 m	600 m
normal (100 kbps)	100 m	400 m
rescue (200 kbps)	50 m	200 m
video (500 kbps)	50 m	50 m

Higher rate target means a lower best altitude. The earlier mixed curve was dominated by low-rate users. Video coverage is near zero at 0 dBm, which comes from the assumed minimum rates (not signed off). Figure: altitude_by_class_v2.png. Sweep size: 813,120 rows.

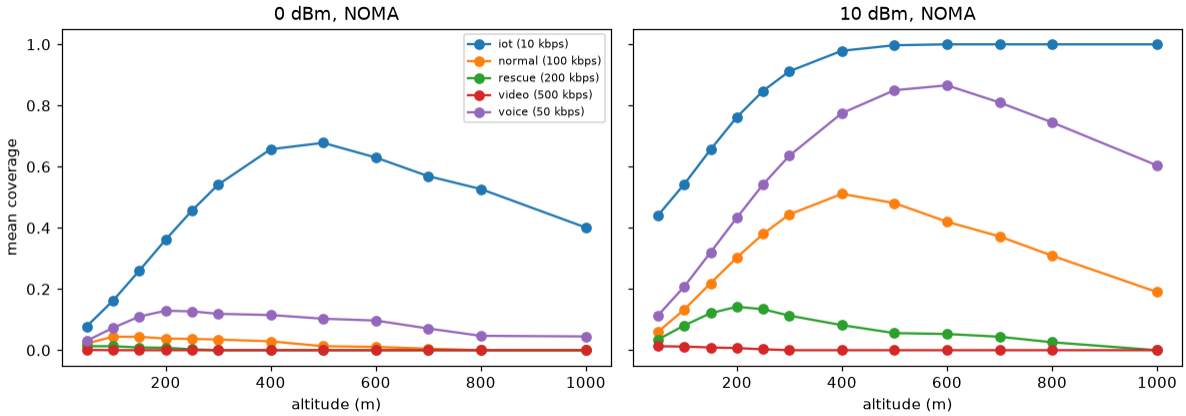


Figure: altitude_by_class_v2.png

8. Phase 3: time and moving crowds

- `simulate()` moves users for 300 s in 1 s steps and stores a snapshot every 30 s; models: random waypoint, group drift, attraction to shelters; speeds 0.5-1.5 m/s (D-06). New `trajectory` table (823,900 rows) and Excel sheet; the Excel file became 147 MB. 20 tests pass.
- **H1 check** (`mobility_check.py`, 15 scenarios, fixed drone at the centre, 200 m, 20 dBm): a fixed drone already covered 79-89% and the best possible position added only 1-6 points (attraction 0.016 at t = 0 to 0.054 at t = 300; group drift 0.055/0.060; random waypoint 0.024/0.011). **Crowd movement barely mattered: the dynamics were too gentle.**
- Not done in this phase: evacuation model, a `demand` field, time in the sweep.

9. Other discussions that day

Real maps (OpenStreetMap) were first raised; the owner chose "dataset first". The owner asked what phases 4 and 5 are, what H1 is, and for the Excel file (shown in Explorer).

2026-10-15

10. Phases 4 and 5: questions answered by the owner

Battery **10 Wh with 300 s episodes** (D-07), UAV limits **15 m/s, 50-300 m** (D-08), **add harder dynamics** (surge, hotspot jump, D-09), **install Gymnasium and build the environment with a simple viewer** (D-10).

11. Phase 4 built: obstacles and fading

- 0-5 collapsed-building rectangles per scenario (40-150 m sides). A drone-user link whose 2D segment crosses a rectangle gets 15 dB extra loss (engineering assumption). Geometry: vectorised slab (Liang-Barsky) test.
- Optional fading: Rician (K = 10 dB) for LoS links and Rayleigh for NLoS, unit mean power, off in the dataset.
- Two harder mobility models: `surge` (everyone suddenly runs to one point) and `hotspot_jump` (one crowd suddenly relocates), at 3 m/s.

12. Phase 5 built: energy, battery and the game

- Rotary-wing energy model (Zeng-Xu-Zhang) with a climbing term (weight x climb rate, no recovery when descending), communication power (P_tx / 0.4) and 5 W of circuit power. Hover = 168.49 W. Battery 10 Wh.
- `uav_env.py`: Gymnasium environment with 211 observation numbers (11 scalars and two 10 x 10 heatmaps), 4 actions [vx, vy, vz, power], reward v1 (coverage) or v2 (coverage minus 0.05 x energy relative to hovering), episode ends when the battery is empty (with a penalty) or after 300 s. It passes Gymnasium's own checker.
- `view_run.py`: a top-down map viewer that saves a GIF. First demo (seed 3, surge, 1,000 users, follow-the-crowd): 247 steps, mean coverage 0.904, battery empty. Figure: viewer_first_run.png.

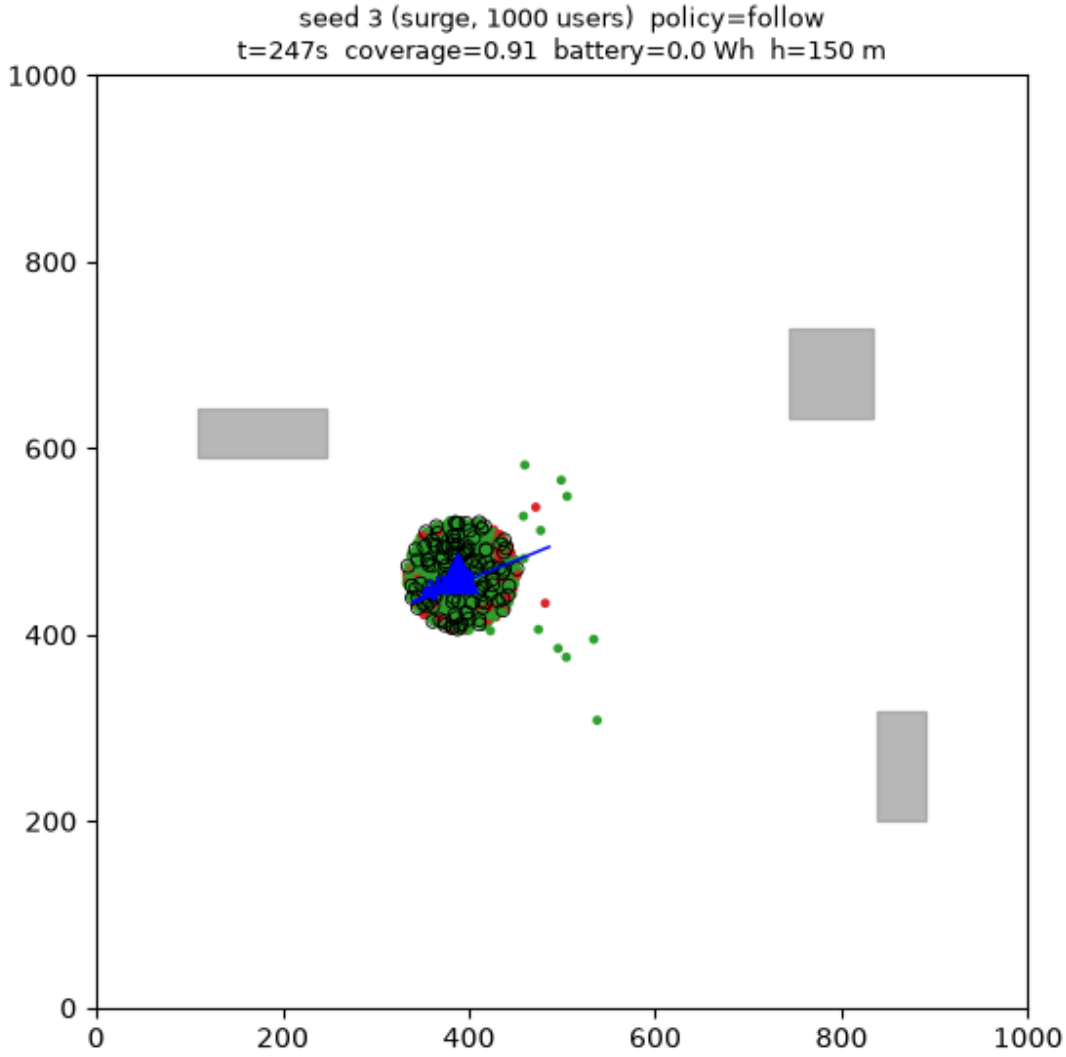


Figure: viewer_first_run.png

- **A test caught a real bug:** `UAVEnv(fading=True)` did not turn fading on (the channel only read the config switch). Fixed.
- 52 tests pass.

13. Dataset v3 and the H1 re-check

- The Excel export first failed with `PermissionError` because the file was open in Excel; after the owner closed it the file was rebuilt: `scenarios` 140, `obstacles` 341, `users` 74,900, `trajectory` 823,900, `sweep` 813,120 rows, 145 MB.
- **H1 re-check** at 10 dBm with obstacles, 30 scenarios (fixed drone vs the best position at each snapshot):

Mobility	Fixed	Best	Gap	Largest gap at one moment
surge	0.527	0.754	0.227	0.753
attraction	0.602	0.738	0.137	0.393
random waypoint	0.614	0.677	0.063	0.233
group drift	0.644	0.700	0.056	0.203
hotspot jump	0.720	0.761	0.041	0.180

Sudden events make a fixed drone fall behind; gentle drift does not. Only 3-9 scenarios per model, so this was a first sign.

14. The Excel file verified cell by cell

The workbook was read back with openpyxl. No empty cells; every sheet has the `split` column; ranges as expected (users 100-1,000 per scenario, bandwidth 5-50 MHz, 0-5 obstacles, mobility counts group drift 33, surge 31, hotspot jump 28, random waypoint 24, attraction 24; class counts voice 22,639, normal 15,023, iot 14,950, video 14,899, rescue 7,389). Notable: only `extreme` pairing appears; NOMA coverage peaks at 0.990 while OMA reaches 1.000 (not investigated); `outage_rescue` averages 0.650. Full list in Chapter 5.

15. The team's task documents

The owner shared task documents for Sagar (world and environment), Arani (baselines, statistics, CI) and Harshita (RL). Finding: most of Sagar's weeks 1-3 (crowd generator, random-waypoint movement, Gymnasium environment) already existed. A new task message was drafted for Sagar: own, review and extend the world code, and build the evacuation model (D-31). Small errors found in the task documents (wrong example branch names, CI missing `gymnasium` and `scikit-learn`).

16. GitHub repository

- Repository `vathsa1703/Major_project` (public). First commit `4cd4cc4`: 15 files, about 90 KB; the generated data (about 430 MB) is excluded and regenerated with one command (D-11). The commit author was set for this repository only (the machine's global git identity belongs to another project).
- **Strict workflow requested by the owner:** everyone works on branches, only reviewed code reaches `main` (D-12). Built: CI (ruff lint, pytest on Python 3.11 and 3.12, an 80% coverage gate, a smoke run of the generator), `CODEOWNERS`, a PR template, `CONTRIBUTING.md`, `pyproject.toml`. Ruff found 91 style findings; the real ones were fixed (a lambda capturing loop variables, an unused variable, `zip` without `strict`) and three style rules were relaxed. 9 pipeline tests added: 61 tests, 87% coverage. **#1 merged.** Branch protection on `main`.
- Arani (`Arani-Acharjee`) invited with write access.

2026-10-16

17. Parameters checked against the papers (#2)

- **Zeng, Xu, Zhang** (arXiv:1804.02238) read: the propulsion formula (Eq. 6), the hover formulas P0 and Pi (Eq. 61), and the constants delta = 0.012, k = 0.1, rho = 1.225, s = 0.05 are verified. Our P0 = 79.86 W, Pi = 88.63 W, Utip = 120 m/s and v0 = 4.03 m/s **follow from those formulas** for a small 20 N drone (R = 0.4 m, Omega = 300 rad/s): computed 79.80 W and 88.66 W (within 0.1%). **The paper's own drone is much bigger** (W = 100 N, hover about 1,370 W). d0 = 0.6 is unverified (the paper uses 0.3).
- **Al-Hourani:** the original paper (IEEE WCL 2014, DOI 10.1109/LWC.2014.2342736) could not be read (paywalled; the RMIT repository returned 403). The formulas match two secondary papers (arXiv:1802.01117, 1705.03304); the values (9.61, 0.16, 1, 20) appear in their simulation tables without an environment label, so "urban" is unconfirmed.
- `docs/assumptions.md` (status of every parameter) and `test_parameters.py` (7 tests): 68 tests.

2026-10-19

18. Baselines B0-B5 and Gate G3 (#3)

- Built `baselines.py` (B0 fixed, B1 follow the biggest crowd, B2 weighted k-means, B3 grid search, B4 particle swarm, B5 energy-aware), `UAVEnv.evaluate()` (scores candidate spots without changing the game), `run_baselines.py` (parallel runner, table with mean +/- 95% CI and paired differences), `evalstats.py`.
- **Finding from a 3-scenario trial:** with 10 Wh **no policy survives 300 s.** Hovering at 30 dBm uses 176 J/s, so 10 Wh lasts 205 s and 300 s needs 14.7 Wh. **Flying at 10 m/s uses only 134 J/s** (Zeng's power curve), so moving lasts longer than hovering. B5's idea ("hover to save energy") does not hold, and its energy cost became negative; it was clamped at 0 (D-14).
- **Full runs (100 scenarios)** at 10 Wh and 16 Wh, tables in Chapter 7. At 16 Wh: B4 0.914, B3 0.909, B5 0.908, B2 0.887, B1 0.884, B0 0.882. **Headroom is small: a parked drone already covers 88%.**
- **CI caught a dependency bug:** `pandas.to_markdown()` needs `tabulate`, which was installed locally but not in `requirements.txt`. Replaced with a small hand-written table function. 89 tests.

19. Harder scenarios (#4)

Owner chose "both": harder scenarios and a battery decision with numbers. Profiles `lowpower`, `bigarea`, `hard` added (D-15). In `hard` (2 x 2 km, 20 dBm, 3-8 obstacles, only sudden or drifting crowds, 16 Wh), B0 falls to 0.428 and B4 reaches 0.740. **H1 holds in this regime.** A bug in my own runner (the saved summary's header replaced the table) was found by reading the output and fixed; summaries were rebuilt from the saved episode data. 99 tests.

20. Merges and a conflict

Owner: "merge". #2, #3, #4 merged in order (admin bypass because the owner cannot approve his own PRs). #3 conflicted with #2 in `.gitignore` (both added entries); both were kept.

21. B6 shows where B4's lead came from (#5)

B6 = grid search that also tries five altitudes. On `hard`: **B6 0.741 vs B4 0.740, a tie**; B6 beats B3 by 0.066. B4's lead over B3 was **altitude choice, not smarter placement**. The bar for RL on `hard` is 0.74 (D-16).

22. RL libraries approved and installed

stable-baselines3 2.9.0, PyTorch 2.14.1 (CPU), TensorBoard 2.21.0, with Gymnasium 1.4.0 and Python 3.14 (D-17).

23. RL pipeline (#6)

A `static` crowd model and the `rl_static` profile (as hard as `hard` but the crowd stands still), training only on seeds 0-99 via a scenario pool (D-18), a control wrapper for the curriculum, `rl_train.py` (config-driven PPO, reward parts logged separately), `rl_eval.py` (paired comparison on a split). Reference on val20: Random 0.413, B0 0.441, B1 0.600, B2 0.610, B3 = B5 0.659.

24. First PPO run: below random

300k steps, velocity controls: **val20 0.394, below random (0.413).** Diagnosis: training coverage rose slowly (0.40 to 0.49); the action noise was still 0.93; the deterministic policy drifted to the map edge (x = 1990); train and validation scores were alike, so not over-fitting. The value function fitted well (explained variance 0.99).

25. Two retries and a slow laptop

Variant A (reward normalisation, starting noise 0.37) and variant B (reward normalisation only) at 1M steps in parallel ran at only 172 steps/s each. Stopping B did not help. **Cause: the laptop was on battery** (Intel Core 3 100U throttles). Plugged in: 1,074 steps/s. A session restart killed A at about 487k steps; it was re-run (`A_full`) and **reproduced the same first scores** (0.431, 0.468), which confirms training is deterministic by seed. `A_full` val8 over 1M steps: 0.431, 0.468,

0.414, 0.474, 0.481, 0.420, 0.426, 0.453, 0.427, 0.408: **a plateau.**

26. Viewer for any scenario, any pilot (#6) and the comparison picture (#7)

`view_run.py` now plays any seed, profile or custom scenario (`--users 600 --clusters 4 --mobility surge --obstacles 6`) with B0-B5, random or a trained model. `compare_run.py` puts several pilots on one scenario in one picture. Gallery:

Figure	Scenario	Result
1_hard_moving_crowds.png	hard, seed 5, 600 users, 5 crowds, group drift	B0 0.49, B2 0.51, B3 0.55, B4 0.71, B6 0.71
2_still_crowd_AI_vs_pilots.png	rl_static, seed 105	random 0.51, B0 0.53, B2 0.68, B3 0.78, PPO 0.40
3_surge_custom.png	custom: 800 users, 5 crowds, surge, 5 obstacles	B0 0.590, B1 0.600, B3 0.635, B4 0.605 (all die near 200 s)

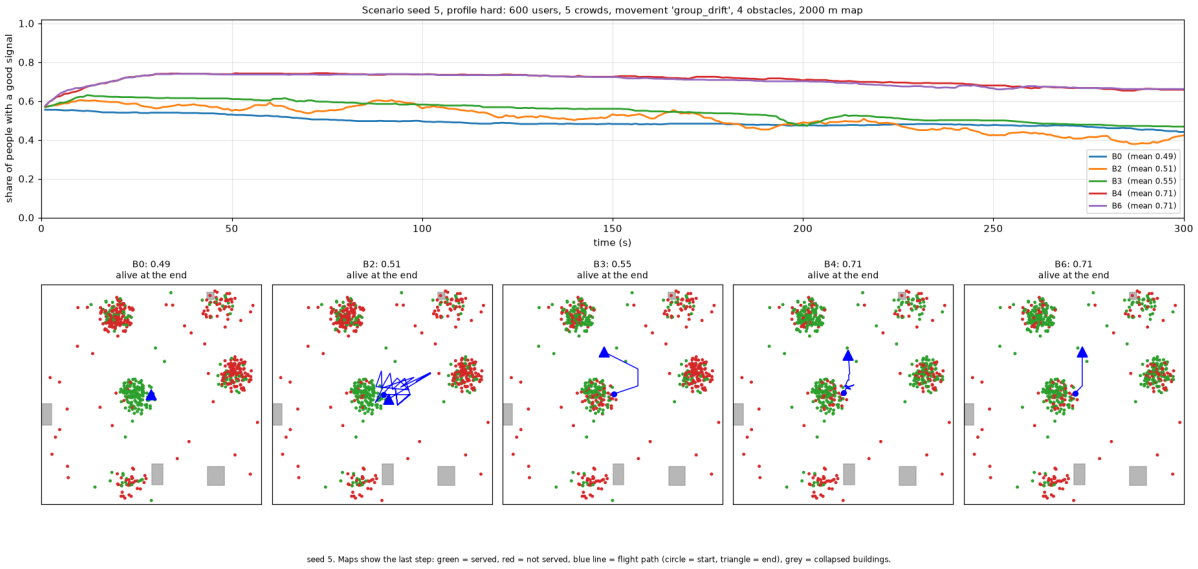


Figure: 1_hard_moving_crowds.png

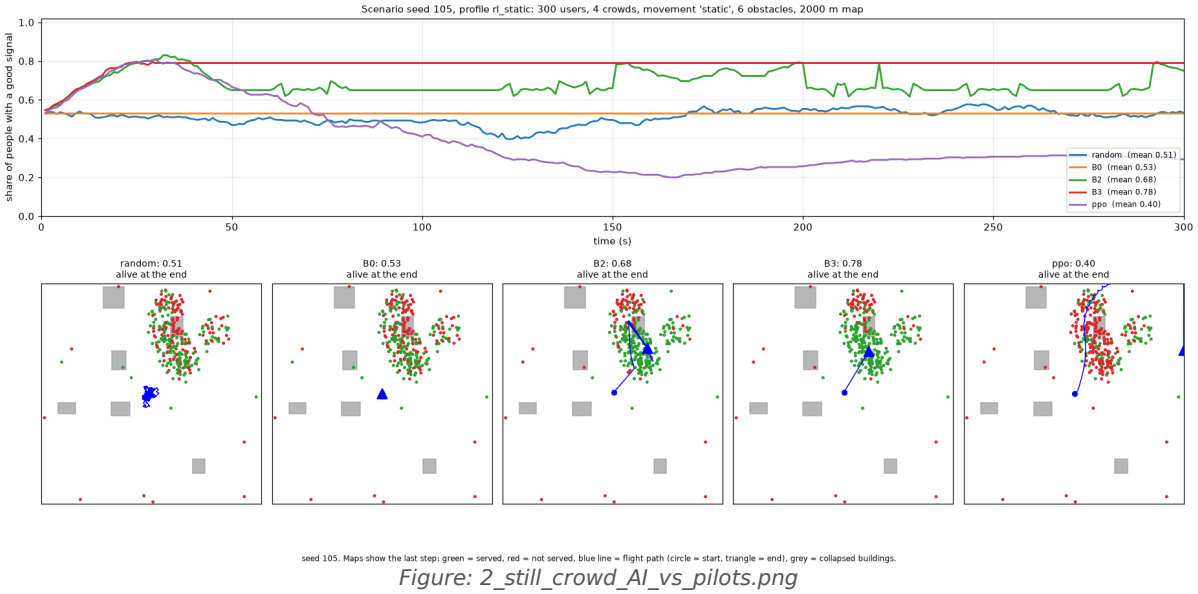


Figure: 2_still_crowd_AI_vs_pilots.png

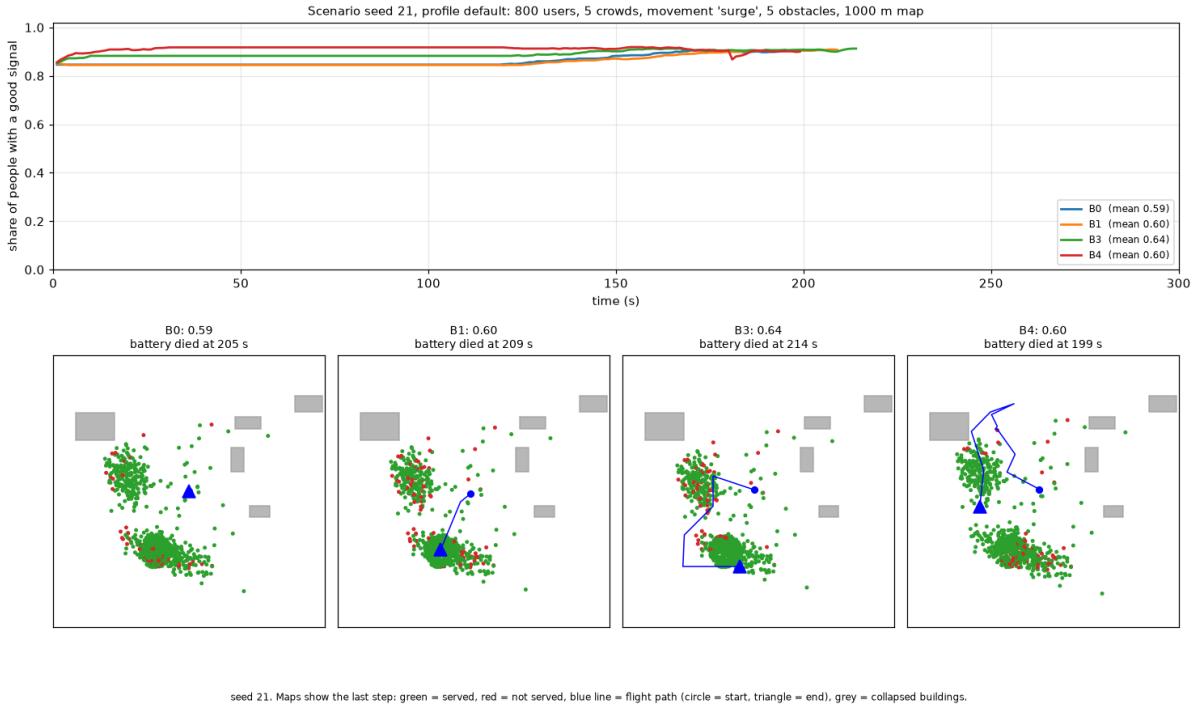


Figure: 3_surge_custom.png

Picture 2 explained the failure: the PPO drone flew to the crowd (0.8 coverage at 30 s, like B3) and then **kept flying**, off the map edge, down to 0.2. It learned to find the crowd, not to stop.

27. Real scenarios parked

The owner asked about dynamic scenarios and real places (a Nepal flood as an example). Explained what public data can provide (buildings, shelters, population density, flood extents, terrain, tower locations) and what it cannot (survivor positions). Owner: finish the plan first; the ideas were written into `CLAUDE.md` (D-25).

28. "Fly to this spot" controls and a 10-second rhythm (#8)

- Waypoint controls (`goto`): the action is where to fly; the drone stays when it arrives (D-19). `goto_v1` (decides every second, 500k steps): val8 0.447, 0.454, 0.444, 0.472, 0.465. Picture 4_goto_checkpoint.png: no more drifting off, but the drone circled around the crowd (seed 105: 0.638).

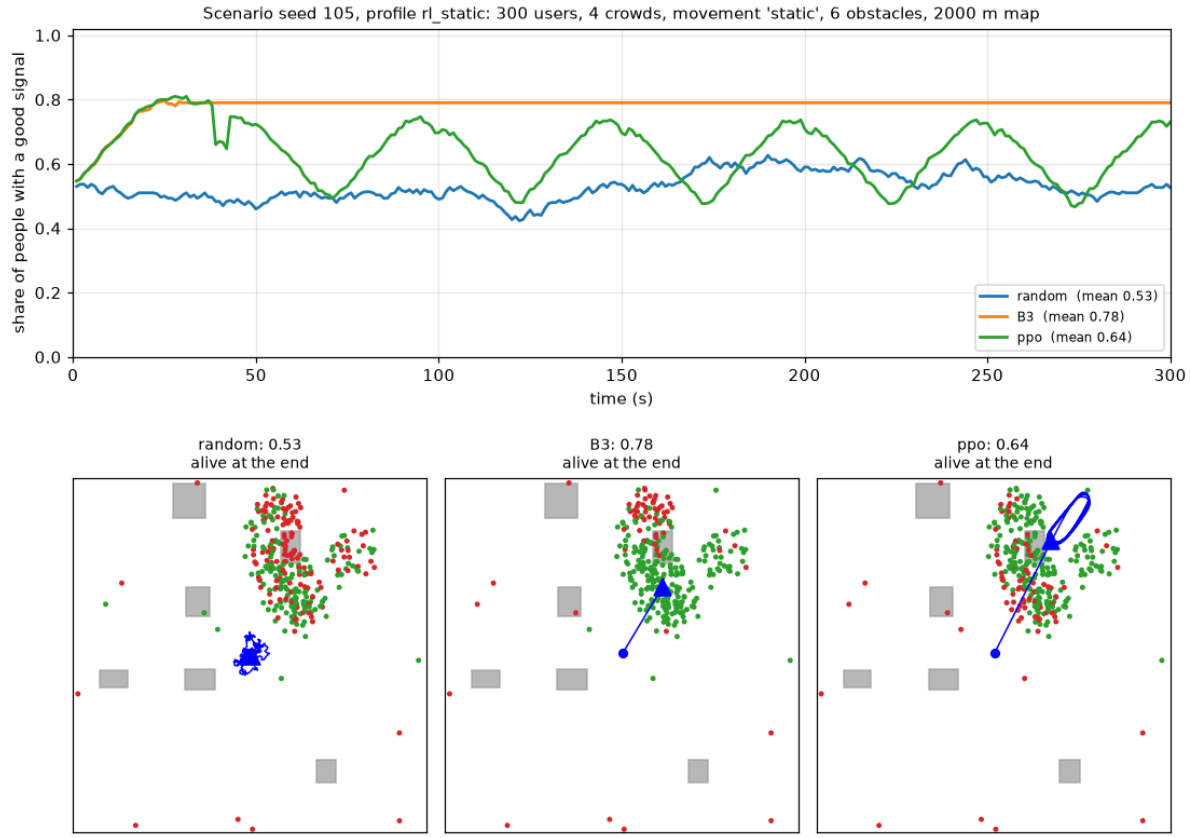


Figure: 4_goto_checkpoint.png

- Decide once every 10 s (`HoldWrapper`, D-20): `goto_hold10` (60k decisions): val8 0.477, 0.486, 0.481. **val20 0.525: paired +0.095 against random (significant), -0.085 against B2 (significant), -0.134 against B3.** Picture 5_goto_hold10.png: the drone holds still but settles at a worse spot after passing a good one. Gate G4 half met.

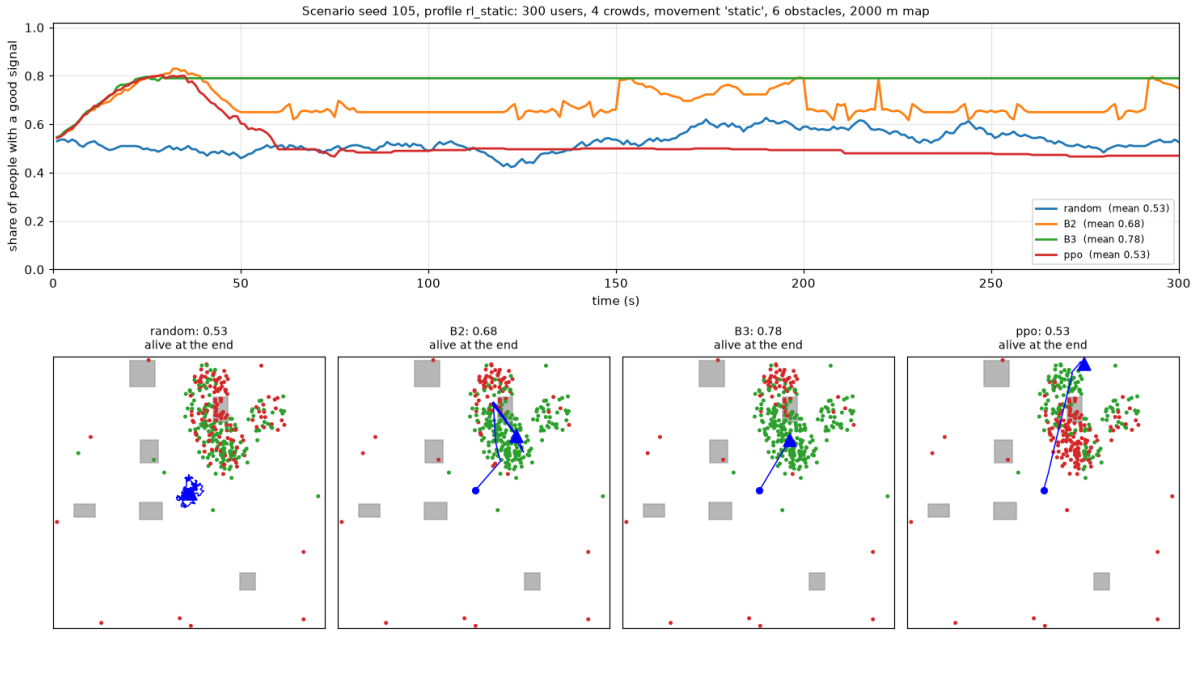


Figure: 5_goto_hold10.png

29. The owner's rule and the stopping rule

The owner rejected the idea of letting the AI pick from 100 map cells: the AI must learn by itself (D-24). A plan was approved with five experiments, a stopping rule and fallbacks (D-26).

30. Experiment 1: why does it not stay at the best spot?

Probe of the `goto_hold10` model on seed 105: it chooses about the same target from the very first decision ((1261, 1934)) and never changes it; it only passes through the good area on the way. Changing one input group moves its chosen target by: own velocity 103 m, previous coverage/throughput/fairness 89 m, battery and time 5 m, altitude 6 m, own x,y 8 m, the all-users map 266 m, the rescue map 445 m. Distance from its target to the best spot: seed 105 546 m (coverage 0.48 vs best 0.83), seed 100 348 m (0.48 vs 0.51), seed 110 109 m (0.51 vs 0.53). **It reads the map but aims imprecisely.**

31. Experiments 2 and 3: leaner observation and a CNN

`obs_groups` (the observation can drop groups; default unchanged), `rl_net.py` (a CNN that reads the two maps as a picture with coordinate channels), settings saved next to each model (D-22, D-23). 154 tests. Results:

Run	Network	Observation	val8 at 20k / 40k / 60k	val20	Paired vs B2
E2 `e2_mlp_lean`	MLP	position + maps	0.481 / 0.466 / 0.492	0.543	-0.067 (sig.)
E3 `e3_cnn_full`	CNN	all	0.496 / 0.502 / 0.516	**0.549**	-0.061 (sig.)
E3b `e3b_cnn_lean`	CNN	position + maps	0.467 / 0.477 / 0.464	0.528	-0.082 (sig.)

E3 against random: +0.119 (significant); against B3: -0.110. Architecture and observation barely moved the score.

32. The ceiling of the still-crowd test

Best single spot on a 41 x 41 grid at 150 m, full power, reached instantly: mean 0.686 over the 20 validation scenarios; map centre 0.441. B3 (0.659) is already close to the ceiling: the still-crowd test is the "hello world" step of the curriculum.

33. Experiments 4 and 5

- E4 `e4_cnn_grid20` (20 x 20 map, profile `rl_static_g20`): val8 0.487, 0.520, 0.509 (1,080 s). A finer map did not help.
- E5a `e5a_cnn_150k` (CNN, 150k decisions): val8 0.496, 0.502, 0.516, 0.512, 0.532, 0.514, 0.516 (1,670 s; exploration noise 0.25 at the end). Slow creep, peak 0.532.

34. Diagnosis: the exploration noise

In every run the exploration noise ended near 0.32-0.34 (start 0.37): about +-330 m of random wobble on each chosen waypoint during training, wider than a crowd. The learning signal is dominated by that wobble, which explains why all variants plateau at 0.52-0.55. Fix approved by the owner (D-27): shrink the noise linearly from 0.37 to 0.05 over training.

35. Discussions with the owner

"Isn't this basic?" (yes, the still-crowd test is curriculum step 1); where NOMA sits (in `noma_rates`, behind every coverage number; nobody decides it yet); what the finished project contains; novelty (two literature searches: the broad idea is published, our contribution would be the benchmark, the fair evaluation and the findings; Chapter 11); the real-world gap (sim-to-real); why research is not deployed (AT&T's tethered "Flying COW" drone exists; NOMA was standardised in LTE as MUST according to one survey); the hardware path; a business track. The owner asked for the no-hardware 3D flying simulation as the last stage of the project (D-28) and for this book (D-29).

36. Merges

#7 and #8 merged (owner's instruction). #8 was first updated with the latest `main` in a separate checkout and re-tested; CI green on Python 3.11, 3.12 and the RL job.

37. Experiment 6: noise annealing

`StdAnnealCallback` in `rl_train.py` (`--std_end 0.05`), with a test. E6 (CNN) and E6b (MLP), 60k decisions each; the noise fell as planned (0.27 at 20k, 0.16 at 40k, 0.06 at the end). val8: E6 0.488, 0.497, 0.475; E6b 0.476, 0.498, 0.490. **Official val20: E6 0.509 (-0.102 vs B2), E6b 0.536 (-0.074).** The fix did not help: the noise was not the main bottleneck, and my diagnosis in entry 34 was wrong.

38. Official scores of the remaining runs, and the stopping rule

val20 (paired vs B2): E4 finer map 0.545 (-0.065*), **E5a longer training 0.555 (-0.055*, best so far)**. Every variant lies between 0.51 and 0.56. The five planned experiments and the extra one are done; the best AI is still significantly below B2. **The agreed stopping rule (D-26) has been reached.** The owner decides whether to relax G4 and move to moving crowds.

39. The project book

The owner asked for everything to be documented in detail, "like a book", and kept up to date after every step (D-29). This book was written: twelve chapters plus a NOMA chapter, figures and GIFs, rebuilt from the session history, the commits and the saved results.

40. NOMA in the game: the OMA switch and hypothesis H3

The owner asked for NOMA to be front and centre ("my sir goes berserk without it"). Built (D-32):

- `UAVEnv(access="noma" | "oma")` (config `env.access`): the same game with NOMA or with OMA (FDMA, same total bandwidth and power). Both schemes are always reported in `info`; the search pilots evaluate candidate spots with the scheme being tested.
- `noma_vs_oma.py`: every pilot plays the same scenarios under both schemes; paired NOMA minus OMA per scenario.
- `noma_picture.py`: a picture of the NOMA pairs on the map and every user's speed under NOMA and OMA.
- 8 tests (`test_noma_access.py`).

Results (100 scenarios each, 16 Wh so every pilot survives; Chapter 13 has the tables):

- **30 dBm (default):** NOMA gives +46 to +66 Mbps more total speed (about +20-25%, significant for every pilot). Coverage is mixed: B0 +0.012 for NOMA, but B1, B2, B3 0.017-0.020 lower with NOMA. Fairness is about 0.19-0.20 lower with NOMA.
- **20 dBm (`lowpower`):** NOMA wins coverage for every pilot, +0.052 to +0.066 (5-7 more people served per 100), and total speed +30 to +47 Mbps. Fairness 0.10-0.13 lower.
- **Hard scenarios (`hard`, 20 dBm, 2 x 2 km):** NOMA covers 6-7 more people per 100 for the moving pilots (B2 +0.065, B3 +0.059, significant), total speed up to +48% (B3 155.8 vs 104.9 Mbps), rescue outage lower for B3 (-0.031); B0's coverage gain (+0.008) is not significant.
- **Why NOMA loses coverage at 30 dBm:** with the fixed 80/20 split, the weak user of each pair always hears the strong user's signal as interference, so its SINR can never exceed 0.8 / 0.2 = 4 and its speed is capped at 100 kHz x log2(5) = **232 kbps**, whatever the power (checked: still 232 kbps at 60 dBm, while OMA gives the same users 507 kbps at 30 dBm). Video users (500 kbps) in the weak half of a pair can never be served with NOMA. This is the classic limitation of fixed NOMA power allocation; the fix (choosing the split per pair) is roadmap stage B.
- Pictures: `noma_picture.py` on scenario 7 at 20 dBm (1,000 users): NOMA coverage 0.81 vs OMA 0.76, total speed 337 vs 263 Mbps, 58 users served only thanks to NOMA, none only by OMA

(7_noma_pairs_lowpower.png).

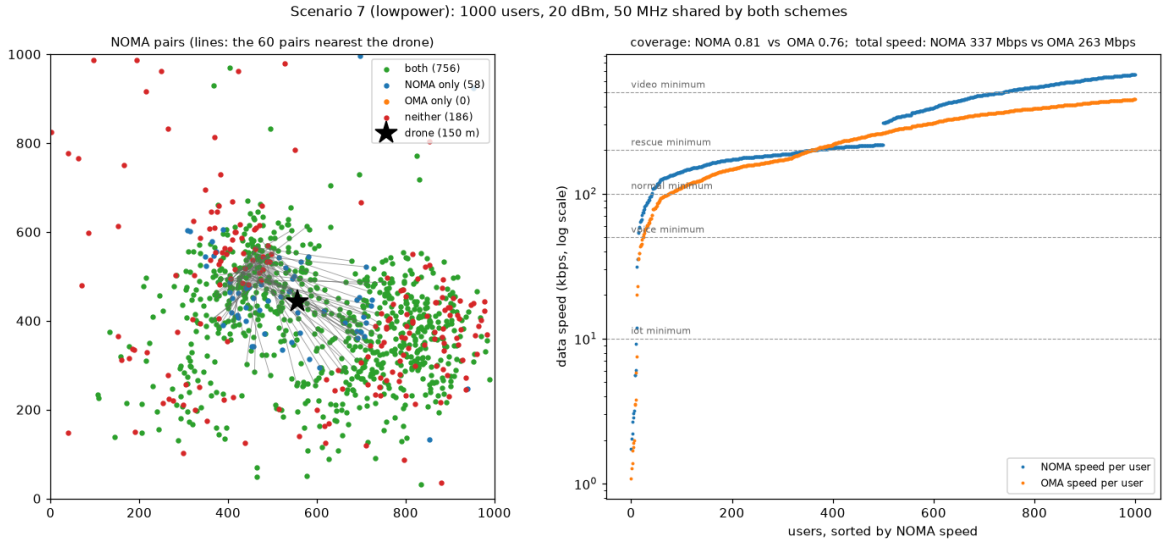


Figure: 7_noma_pairs_lowpower.png

41. GIFs saved into the book

`figures/gifs/`: the first viewer run, a custom 600-user surge flown by B2, the first PPO model, the best AI (E5a) on scenario 105 (coverage 0.609), B3 on the same scenario (0.781), and B4 on the hard scenario 5 (0.709).

42. A long pause

The NOMA vs OMA batch, started on 10-19 at 18:17, seemed hung on 10-21: no output for about 42 hours. Its workers had used only about 140 s of CPU each: the laptop had been asleep, pausing the run. It resumed when the laptop woke and finished normally (all three H3 runs complete on 10-21).

2026-10-21

43. The book as a PDF

`docs/book/build_pdf.py` turns the Markdown chapters into `docs/book/UAV-NOMA-Project-Book.pdf` (ReportLab, with the DejaVu fonts that come with matplotlib): a title page, contents with page numbers, every table, list and code block, every figure once, GIFs as their last frame, and an appendix gallery for figures the text does not link. A bug in my converter (a line starting with "#2," was neither a heading nor a paragraph, so the parser never moved on) made the first attempt run for 8 minutes; fixed. New rule: rebuild the PDF after every change to the book.

44. A senior-level plan, reviewed by a second agent (Chapter 14)

The owner asked me to approach the problem as a senior communications and AI engineer would, and to have the solution reviewed by another agent. My draft: split the problem by time scale (compute the NOMA allocation and the placement, learn only the mission strategy), fix NOMA with adaptive power allocation, and measure the headroom for RL with an oracle before training more. The reviewer confirmed the maths and found real gaps: an unfair OMA comparison, a guarantee that is true by construction, a ceiling that ignores flight time, and above all that **the AI never sees obstacles or demand while the search pilots do**. It also proposed better steps. I checked its claims in the code, accepted most of them, and declined one (a spatial action head that looks like the rejected 100-cell menu) until the owner decides. The agreed 7-step plan and four owner decisions are in Chapter 14.

45. Step 1 of the plan: why the AI stalls (two agents in parallel)

The owner merged PR #9 and asked for step 1 with several agents working in parallel. Agent 1 built an "information ladder": three copies of B3's search that know less (only the AI's capped maps; uncapped maps; plus buildings), with 19 tests. Agent 2 checked the AI on its own training scenarios and measured what the capped map hides. Results (Chapter 8.7): the search that sees only what the AI sees scores 0.605 (level with B2), the AI 0.555; so half the gap to B3 is information and half is learning. The AI is behind even on scenarios it trained on (underfitting). The map cap hides the size of 59% of the crowd, but a 20 x 20 map did not help, and B3's real trick is parking *between* crowds, which the AI never learned. I checked the code, reproduced the main number and showed it does not depend on the assumed crowd size (0.588-0.606).

46. Owner's decisions, a second review, and two tracks in parallel

The owner merged PR #10, answered the four questions of Chapter 14.7 (D-38) and asked me to discuss the next designs with another agent before building, in case it had a better idea. It did. I gave a reviewer my designs for adaptive NOMA and for a better learner; it ran the real code and found three flaws in the NOMA design (D-39) and one in the learner design (a privileged critic helps in SAC, hardly in PPO; D-40). I questioned it on speed and training cost; it measured both and we agreed the specs. Two agents now build the tracks in parallel, each in its own copy of the repository: Track 1 adaptive NOMA vs equally optimised OMA; Track 2 the better learner. The physics default stays as it is while G4 is open (D-41). The owner also asked how far the project is from completion: about 55-60% of the core; 5-7 weeks of focused work for the research question (tracks 1 and 2, G5 on moving crowds, final evaluation), 10-12 weeks with the extras and the flying sim.

47. Track 1 done: adaptive NOMA beats equally optimised OMA

The Track 1 agent built two new access schemes (`oma_opt`, `noma_opt`) exactly as agreed in D-39, with 45 tests, and ran them in the game on 100 scenarios per profile while the AI trained alongside (about 1.5 hours in total on the shared laptop). Result (Chapter 13.9): with both schemes optimised the same way, NOMA serves 3-5 more people per 100 for every pilot, with more total speed, better fairness and fewer unserved rescue users; B3 on lowpower reaches 98.3% coverage. Imperfect SIC (eps 0.05) halves the gain but does not remove it. The catch, for both optimised schemes: much less total speed than the old fixed schemes. I checked the work: the old schemes are byte-identical and reproduce the Chapter 13 numbers, 227 tests pass, and I re-derived the power formula independently (exact to 1e-15 on 2,000 random pairs).

48. The GIFs were never on GitHub (found and fixed)

Building the PDF from a clean copy of the repository showed 22 pictures instead of 26: the 6 GIFs of the book had never been committed, because `.gitignore` ignores every `*.gif` (a rule from the start, meant for viewer output). They existed only on the owner's laptop. Fixed with an exception for `docs/book/figures/gifs/`; the 6 GIFs (4.7 MB) are now in the repository.

49. The owner sets the G4 pass mark, approves the physics fixes, and Track 2 ends

While the AI trained, two AI runs scored the same 0.573 but one "passed" the old test and one "failed", because 20 scenarios give wide error bars. I explained the problem (first too technically; the owner asked again and an exam analogy worked) and the owner chose a fixed pass mark **before** the last result: 3-run average at least 0.58 (D-43). A reviewer then found that the game lets a drone save 25% battery by shaking back and forth on the spot (no acceleration limit); I reproduced it (0 m moved, 131 J/s instead of 174). The owner approved an acceleration limit, run-to-empty as the circling pilot's main measure and a turning-energy term (D-44 to D-46). Steps 3 and 5 were built in parallel while the AI trained. Track 2 result (Chapter 8.8): best AI 0.576 over 3 runs, 0.004 short of the pass mark and clearly behind B2 on a second unseen set; the privileged critic did not help. **G4 not met; stopping rule reached; on to moving crowds.** Merging Track 1 into Track 2 had one conflict (both added arguments to `UAVEnv.__init__`), resolved by keeping both; 251 tests pass.

50. Circling beats hovering on a tight battery; settings for G5; a new roadmap

Step 5 (Chapter 7.10, Chapter 4.7): with the acceleration limit and turning cost on (the shaking trick closed; I checked: shaking now costs 175 J/s, more than hovering), a pilot that circles at 9.8 m/s instead of hovering serves 25.6% more people-seconds per battery, and in a 5-minute mission with a 12-13 Wh battery it finishes while the hovering pilot dies (+0.08 to +0.12 coverage). The gain survives every parameter change tried (+21% to +31%). Merging step 5 with Tracks 1 and 2 had one conflict (both added pilots to the list), resolved by keeping all; 277 tests pass. The owner chose the settings for G5 (D-48), moved the still-crowd fixes to the backlog with the "move by" steering tested inside G5 (D-49), approved a new roadmap and asked for a dashboard to show the work (D-50). When the owner wondered whether the project was failing, I listed what is proven (H1, H3, B7, the simulator, the dataset) against the one open problem (the AI does not hold the best spot), and made two GIFs of scenario 105 showing that problem (AI 0.66, B3 0.78).

51. The prize for anticipation is almost zero (step 3)

The step 3 agent built a clairvoyant pilot that knows where every person will be for the whole mission and plans the best possible flight, replayed in the real game (about 3 hours of computing on one process). Against B6 it gains only +0.002 coverage on average and at most +0.003 for any crowd type (Chapter 7.11), far below the 0.02 pass mark fixed beforehand, so the rule fired (D-51): today's moving crowds leave no room for an AI that anticipates, because the drone (15 m/s, re-planning every 10 s) is much faster than the crowds. I checked the numbers from the raw data and re-ran one scenario myself. This is a useful negative result found *before* two weeks of training. The scenarios must be redesigned before G5; the owner chooses how. The owner also merged PRs #13, #14 and #15 (tracks 1 and 2, step 5, the dashboard).

Chapter 3. Decision log

Every significant decision, with the question, the options considered, what was chosen, who decided, and the consequences. New decisions are appended with the next number. A decision is never deleted; if it is reversed, a later decision says so. "Owner" means Srivathsa (repository owner and reviewer). "Proposed" means suggested by the assistant and accepted by the owner.

#	Date	Decision	Who
D-01	10-14	Work phase by phase, each with a check	Owner
D-02	10-14	One Excel workbook with a `split` column, CSVs kept	Owner (proposed)
D-03	10-14	Bandwidth scales with the number of users	Owner
D-04	10-14	Five user classes with the spec's mix; minimum rates as assumptions	Owner
D-05	10-14	Extend altitudes to 1,000 m (diagnostic) and check the optimum per class	Owner
D-06	10-14	Three walking mobility models, 300 s, snapshots every 30 s	Proposed
D-07	10-15	Battery 10 Wh with 300 s episodes	Owner
D-08	10-15	UAV limits 15 m/s, 50-300 m	Owner
D-09	10-15	Add sudden crowd events (surge, hotspot jump)	Owner
D-10	10-15	Gymnasium environment plus a simple viewer	Owner
D-11	10-15	Generated data is not committed; it is regenerated from config and seeds	Proposed
D-12	10-15	Branches, pull requests, owner review, CI, protected `main`	Owner
D-13	10-16	Our drone is a small 20 N UAV derived from Zeng's formulas	Proposed
D-14	10-19	Main metric `coverage_full` (a dead drone counts zero); B5's energy cost clamped at 0	Proposed
D-15	10-19	Harder scenarios as config profiles; `hard` uses 16 Wh	Owner (both options)
D-16	10-19	Add B6 (grid search plus altitude) before judging RL	Proposed
D-17	10-19	Install stable-baselines3 and PyTorch; keep them in `requirements-rl.txt`	Owner
D-18	10-19	Train on seeds 0-99, choose on 100-119, test 120-139 once	Proposed (plan)
D-19	10-19	Waypoint ("fly to this spot") controls	Proposed
D-20	10-19	The agent decides once every 10 s	Proposed
D-21	10-19	Merge PRs with the owner's admin bypass after CI is green	Owner
D-22	10-19	Observation groups and a CNN in a separate module	Proposed
D-23	10-19	Save each model's settings next to it	Proposed
D-24	10-19	The AI learns by itself: no hints, no menu, no privileged information	Owner
D-25	10-19	Park dynamic events and real-place scenarios until the gates are done	Owner
D-26	10-19	G4 tuning: five experiments, a stopping rule, agreed fallbacks	Owner (plan approved)
D-27	10-19	Anneal the exploration noise (outside the five experiments)	Owner
D-28	10-19	The no-hardware 3D flying simulation is the last stage	Owner
D-29	10-19	Keep a detailed project book, updated after every step	Owner
D-30	10-14	Never call it a digital twin; it is a simulation-based planning tool	Plan
D-31	10-15	Sagar's task becomes own, review and extend (draft, not yet sent)	Proposed
D-32	10-21	NOMA/OMA switch in the game; H3 measured for every pilot	Owner ("add NOMA")
D-33	10-21	G4 tuning stops: the stopping rule is reached at 0.555 vs B2 0.610	Rule D-26 (owner to decide next)
D-34	10-21	Keep figures and GIFs in the book; the book is updated in every pull request	Owner
D-35	10-21	The book is also published as a PDF, rebuilt after every change	Owner
D-36	10-21	Major plans are reviewed by an independent agent before work starts; the agreed plan is Chapter 14.6	Owner
D-37	10-21	Information-ladder pilots B3obs/B3obsU/B3obsUO as diagnostics; assumed crowd 500; not in default runs	Claude (plan step 1)
D-38	10-21	Owner's answers to 14.7: asymmetric critic yes; cell action head no; hierarchical design later; move to moving crowds after tracks 1 and 2	Owner
D-39	10-21	Adaptive NOMA design after a second review: admit-then-pair, minimum-power closed form, imperfect SIC from the start, same slot rule for both schemes	Claude + reviewer

#	Date	Decision	Who
D-40	10-21	Better-learner design after a second review: uncapped map, crowd size, own measurement history, fresh seeds, SAC with a training-only privileged critic	Claude + reviewer
D-41	10-21	The physics default stays `access: noma` until G4 is closed, so the bars the AI is judged against do not move	Claude + reviewer
D-42	10-21	Adaptive NOMA added as `noma_opt` (with `oma_opt` as its fair reference); default unchanged; G5 default is the owner's call	Claude
D-43	10-21	G4 pass mark: 3-run average on val20 >= B2 - 0.03 (0.580) and clearly above random; extra runs only from 0.570	Owner (set before the result)
D-44	10-21	Acceleration limit 4 m/s^2 (sensitivity 2-8), off until G4 closes; closes the battery-saving 'shaking' exploit	Owner
D-45	10-21	Circling pilot B7 is judged by total service until the battery is empty	Owner
D-46	10-21	Turning/acceleration energy term added, labelled as our extension of the Zeng model	Owner
D-47	10-21	G4 closed as NOT met (best 0.576 < 0.580); stop still-crowd tuning; E0 is the starting agent for G5	Rule D-43 + D-26
D-48	10-21	Settings for G5 onwards: access `noma_opt`, sic_eps 0.05, rescue_first on, acceleration limit 4 m/s^2 on	Owner
D-49	10-21	Still-crowd fixes (G4b: move-by steering, longer training, bigger net; fix C) go to the backlog; the move-by fix is tested inside G5	Owner
D-50	10-21	New roadmap (stages 0-5) replaces the 10-19 order; dashboard built now as a parallel showcase track (Streamlit approved)	Owner
D-51	10-21	Clairvoyant test: no room for anticipation in today's moving-crowd scenarios (+0.002 < 0.02); redesign scenarios before G5	Pre-registered rule

D-01 Work phase by phase

Question. The owner's parameter spec described the finished simulator; build it all at once? **Decision.** No: phases 1-5, each with a check (validate the slice, configurable scenarios, time and mobility, obstacles and fading, energy and the environment). **Why.** Several spec values conflicted (battery never runs out, rates collapse at 1,000 users, sweep too large). Building in layers lets each layer be tested before the next depends on it.

D-02 One Excel workbook

Question. The owner expected one dataset file; the generator produced six CSVs (two tables x three splits). **Options.** One workbook with sheets; two CSVs with a `split` column; keep six CSVs. **Decision.** One workbook (`export_excel.py`) with one sheet per table and a `split` column; the CSVs stay as the source. **Consequence.** Excel's limit is about 1,048,575 rows per sheet; the exporter fails loudly above it. The workbook later grew to 145 MB, slow to open; a small preview workbook was offered for the guide.

D-03 Bandwidth scales with users

Question. With 1,000 users on 10 MHz each user gets about 10 kHz and nobody reaches a minimum rate. **Options.** Scale bandwidth with users (50 kHz each); scale minimum rates down; keep both fixed. **Decision.** Bandwidth = users x 50 kHz (200 users = 10 MHz). **Consequence.** Per-user rates stay comparable across scenario sizes; unrealistic beyond about 100 MHz. Listed as an engineering assumption.

D-04 Five user classes

Decision. rescue 10%, video 20%, voice 30%, iot 20%, normal 20% (the spec's example). Minimum rates rescue 200 kbps, video 500 kbps, voice 50 kbps, iot 10 kbps, normal 100 kbps. **Status.** The minimum rates are **engineering assumptions, not signed off.** Video is almost never covered at low power. They must be decided before any number goes into the report.

D-05 Altitudes to 1,000 m and a per-class check

Why. After D-04 the mixed altitude curve lost its peak. Extending the sweep and checking per class showed the optimum exists for every class and moves lower as the rate target rises.

Altitudes above 500 m are diagnostic only (the plan's maximum is 300-500 m).

D-06 Walking mobility

Random waypoint, group drift and attraction to shelters; speeds 0.5-1.5 m/s; 300 s at 1 s steps; snapshots every 30 s so the trajectory table fits in Excel. The sweep still uses the t = 0 positions.

D-07 Battery 10 Wh, 300 s

Options. Small battery (10 Wh, about 3.5 minutes of hover) with 300 s; a realistic 50 Wh battery with 1,800 s episodes; the spec's 50-100 Wh with short episodes (never runs out). **Decision.** 10 Wh, 300 s. **Consequence found later (D-14).** Hover needs 14.7 Wh for 300 s and the cheapest cruise about 11 Wh, so no policy survives; rankings then mostly measure survival time. The `hard` profile uses 16 Wh. The battery size remains an open decision for the final experiments (a battery sweep is experiment E7).

D-08 UAV limits

Maximum horizontal speed 15 m/s, vertical 5 m/s, altitude 50-300 m, start at 150 m, maximum power 30 dBm.

D-09 Sudden crowd events

`surge` (everyone suddenly runs to one point) and `hotspot_jump` (one crowd relocates), 3 m/s, starting between 60 and 180 s. **Why.** The first H1 check showed gentle movement barely hurts a parked drone.

D-10 Gymnasium and a viewer

New dependency approved by the owner. The environment is the "game" all pilots and the AI play through the same `env.step()`.

D-11 Data is not committed

About 430 MB of generated data is regenerated in about a minute from `config.yaml` and the fixed seeds, so it stays out of git.

D-12 Strict review workflow

Everyone works on `name/what-it-does` branches; pull requests need green CI (lint, tests on Python 3.11 and 3.12, coverage gate, smoke run) and the code owner's approval; stale approvals are dismissed; `main` cannot be force-pushed or deleted. The owner's admin bypass stays on because GitHub does not let an author approve their own PR (D-21).

D-13 Our drone

Zeng's paper's own simulated drone weighs 100 N and hovers at about 1.37 kW. Our parameters correspond to a small 20 N drone (R = 0.4 m, Omega = 300 rad/s), hovering at 168.49 W. This is our choice, stated in `docs/assumptions.md`; d0 = 0.6 is unverified.

D-14 The main metric and B5

`coverage_full` = coverage averaged over the whole 300 s, counting 0 after the battery dies. It rewards staying alive. Because forward flight at about 10 m/s costs less than hovering in Zeng's model, B5's "extra energy for moving" became negative and was clamped at 0; B5 then behaves like B3.

D-15 Harder scenarios as profiles

Profiles (`lowpower`, `bigarea`, `hard`, later `rl_static`, `rl_static_g20`) override config values without changing the default dataset. `hard`: 2 x 2 km, 20 dBm, 3-8 obstacles of 60-200 m, only sudden or drifting crowds, 16 Wh, 14 x 14 search grid.

D-16 B6

B4's lead could have been altitude choice. B6 (grid search plus five altitudes) tied B4, which settled the question and set the RL bar on `hard` at 0.74.

D-17 RL libraries

stable-baselines3 and PyTorch (CPU) approved by the owner. They live in `requirements-rl.txt` so the simulator, baselines and most tests do not need them; a separate CI job runs the RL tests.

D-18 Data splits for RL

Training scenarios come only from seeds 0-99 (a scenario pool in the environment); validation seeds 100-119 are used to compare and choose models; test seeds 120-139 are read once at the end.

D-19 Waypoint controls

Problem. With velocity controls the trained drone found the crowd and then drifted away: hovering requires exactly zero velocity and nothing rewarded it. **Decision.** The action is a spot on the map (and later an altitude and a power); the drone flies there and stays. Velocity controls remain available (`xy`, `xyz`, `xyzp`).

D-20 Decide every 10 seconds

Problem. Deciding every second, the drone circled around the crowd. **Decision.** The choice is held for 10 s, the rhythm the baselines re-plan at; a game becomes 30 decisions instead of 300.

D-21 Merging

The owner instructs merges; they are done with the admin bypass after CI is green, never before.

D-22 Observation groups and the CNN

The observation is built from named groups (position, velocity, battery, time, previous scores, the two maps), so runs can drop groups. The CNN lives in `rl_net.py` so importing the rest never needs PyTorch.

D-23 Settings next to each model

`config.json` is saved next to every model, and the viewers and evaluator read controls, decision hold and observation groups from it. This prevents playing an old model with the wrong settings.

D-24 The AI learns by itself

Question. Should the AI choose from 100 map cells to make learning easier? **Owner's decision.** No. The AI gets raw measurements and the coverage score and must learn by itself. Not allowed: hints, a menu of candidate spots, imitating the baselines, privileged information. Allowed: network design, training length, hyperparameters, seeds, curriculum order. Recorded in `CLAUDE.md`.

D-25 Parked ideas

Dynamic events (rising flood, evacuation, aftershock, survivors appearing, drone failure) and real-place scenarios (for example a Nepal flood from public map data) wait until the planned gates are done. They are now stages C and D of the roadmap.

D-26 Stopping rule for G4

Five experiments (diagnose, leaner observation, CNN, finer map, more training and seeds), then stop. If PPO is still significantly below B2: report it, consider moving to moving crowds (the owner may relax G4), try SAC or a memory network, and only with the owner's approval a hybrid design reported separately.

D-27 Noise annealing

Diagnosis showed the exploration noise stays near 0.33 (about +-330 m), wider than a crowd. The owner approved one extra experiment outside the five: shrink the noise linearly from 0.37 to 0.05 over training. It gives the AI no information about the task.

D-28 The flying simulation comes last

A simulated quadcopter with an autopilot model, driven by our controller through "fly to this spot" commands, is the final stage (stage F of the roadmap). Real hardware is outside this project.

D-29 This book

Everything is documented in `docs/book/` and updated after every step (journal entry, decision, results).

D-30 Wording

From the plan: "simulation-based planning tool"; "digital-twin-style" only with real state data or trace replay.

D-31 Sagar's task

Most of Sagar's first three weeks of tasks (crowd generator, movement, environment) already existed. A message was drafted: own, review and extend the world code, write the explanation document, build the evacuation model, then scenario presets. Not yet sent by the owner.

D-32 NOMA measured inside the game

Question. NOMA was only visible inside the physics; the guide needs to see it working. **Decision.** `UAVEnv(access="noma"|"oma")` and `noma_vs_oma.py`: every pilot plays the same scenarios under NOMA and under OMA with the same total bandwidth and power, compared scenario by scenario; `noma_picture.py` shows the pairs. This is hypothesis H3 (experiment E3) done properly. **Consequence.** NOMA wins total speed everywhere and coverage when power is scarce; with the fixed 80/20 split OMA is slightly better on coverage at full power because the weak user is capped at 232 kbps. This motivates stage B: choosing the power split.

D-33 The G4 stopping rule is reached

Five experiments and the extra noise experiment are done; the best AI (E5a, 0.555) is still significantly below B2 (0.610). Per D-26, tuning stops and the result is reported. The next step is the owner's decision: relax G4 and move to moving crowds, or try other learners (SAC, memory) that give no hints.

D-34 Figures and GIFs in the book

Pictures and animations that support a result are copied into `docs/book/figures/` (and `figures/gifs/`) with captions saying which run produced them. Every pull request that changes results updates the book.

D-35 The book as a PDF

`python docs/book/build_pdf.py` rebuilds `docs/book/UAV-NOMA-Project-Book.pdf` from the chapters. It needs `reportlab` (`requirements-docs.txt`); Pillow comes with matplotlib. The Markdown stays the source of truth; the PDF is rebuilt in every pull request that changes the book.

D-36 Independent review of plans

The owner asked that major plans be reviewed by a second agent acting as a senior engineer, and that both look for a better solution together. Claims from the review are checked in the code before being accepted. First use: Chapter 14 (draft, review, agreed plan, decisions for the owner).

D-37 The information-ladder pilots

To find out whether the AI is limited by what it sees, three diagnostic pilots run B3's exact search with less information (Chapter 8.7). They rebuild a stand-in crowd from the maps, so they need an assumed crowd size: **500 people (engineering assumption, the mean of the crowd sizes drawn by the generator;** `baselines.obs_pseudo_users`**)**; checked at 200 and 1,000 (validation 0.588 / 0.605 / 0.606). Non-rescue people are given the population's class mix, since their needs are not on the maps. The pilots are not part of the standard baseline runs (`run_baselines.py` runs B0-B6 unless they are named), so the G3 tables are unchanged.

D-38 The owner's four decisions (Chapter 14.7)

1. **Asymmetric critic: yes.** During training only, the critic (the part that judges, thrown away afterwards) may see the true raw state: true crowd density, buildings, rescue users, needs, crowd size. The flying policy never sees it. Not allowed even for the critic: anything computed by a baseline's scorer (for example a map of coverage at every spot), which would amount to copying the baseline. Disclosed in every result that uses it.
2. **Cell action head: no.** It resembles the rejected 100-cell menu (D-24).
3. **Hierarchical design (Option A): later**, after the moving-crowd results.
4. **Move on to moving crowds (G5)** after tracks 1 and 2, whatever the still-crowd score.

D-39 Adaptive NOMA: the reviewed design (Track 1)

My first design (Chapter 14.3, P1) had four flaws the second review found and I verified:

- With perfect SIC, NOMA never needs more power than OMA for a pair, so a per-pair "NOMA or OMA" choice is decided before any experiment. The choice only matters with **imperfect SIC** (a residual epsilon of the removed signal), which is therefore part of the design from the start (epsilon = 0 and 0.05, engineering assumption).
- Deciding per pair after extreme pairing ties every cheap user to an expensive partner; the reviewer measured that this is even worse than optimised OMA. Instead: **admit users cheapest-first, then pair the admitted users** (strongest with weakest). The largest number of users that fits the power budget is found by binary search (measured: same result as an exhaustive scan on 4 of 5 scenarios, 40-100 ms per evaluation of 196 spots).
- The split alpha* assumed equal power per pair. Instead each pair gets its **minimum power**, in closed form: p_s = gamma_s N (eps gamma_w / g_w + 1 / g_s) / (1 - eps gamma_s gamma_w), p_w = gamma_w (p_s + N / g_w).
- Bandwidth accounting decides the result, so one rule for both schemes: the slot of a user who is not served is wasted. Both schemes are optimised the same way (`oma_opt`, `noma_opt`); leftover power is scaled up by one common factor. Optional "rescue users first" admission. The old schemes stay byte-identical.

D-40 A better learner: the reviewed design (Track 2)

- Inputs (all raw measurements): an **uncapped** crowd map (no saturation), the **number of connected users**, and the drone's own last 4 positions and measured coverages. Fresh procedural training scenarios (10,000+ seeds, never val/test). Planning horizon gamma 0.95 to match 30 decisions per game.
- **SAC with a privileged critic.** In PPO a privileged critic only estimates how good a situation is (it reduces noise but cannot show where to go). In SAC the critic scores *actions*, so a critic that knows the true state can learn where the good spots are and steer the policy towards them, while the policy itself still sees only raw measurements. A test checks that changing the privileged inputs leaves the policy's output unchanged.
- Budget: SAC is about 5-20 decisions per second on this laptop (PPO about 60), so one seed per variant first, more seeds only for a variant that beats B3obs (0.605). Stopping rule: if none

does, close G4 as a negative result and go to moving crowds.

D-41 Keep `access: noma` while G4 is open

Adaptive NOMA changes coverage by 10-25 points and moves the best spots, so it would change B2, B3 and B3obs, the bars the AI is judged against. Track 2 trains and evaluates with today's NOMA; the project default for G5 is decided after Track 1's results.

D-42 Adaptive NOMA is an option, not yet the default

`noma_opt` and `oma_opt` are new values of `env.access`; `sic_eps: 0.0` and `rescue_first: false` are new settings under `access:` (engineering assumptions). The default stays `noma` (D-41), so every earlier number is unchanged and reproducible. Before G5 the owner chooses the project default once. My recommendation: `noma_opt` with `sic_eps: 0.05` (realistic, still beats optimised OMA) and `rescue_first: true` (fits a disaster mission), reported next to `oma_opt` so H3 stays a fair test.

D-43 The G4 pass mark

The old test ("not significantly worse than B2") flipped on luck: two runs with the same 0.573 got opposite verdicts, because 20 scenarios give wide error bars. The owner chose, before E3's result was known: **G4 passes if the average over 3 training runs on the 20 validation scenarios is at least B2 - 0.03 = 0.580 and the AI is clearly better than random.** Only a version whose first run reaches 0.570 gets the 2 extra runs.

D-44 to D-46 Physics fixes approved before the circling experiment

A reviewer found, and I reproduced, that the drone can change velocity instantly, so alternating +10 and -10 m/s every second keeps it in place at the cheaper forward-flight power (131 J/s instead of 174 J/s hover). No earlier result used this (no pilot shakes, the AI's reward ignores energy), but any energy result or energy reward could be gamed. The owner approved: an acceleration limit (4 m/s^2, engineering assumption, sensitivity 2/4/8, **off by default until G4 closes** so earlier results stay bit-exact); total service until the battery is empty as the circling pilot's main measure; and a load-factor energy term for turning and braking, labelled as our extension of Zeng-Xu-Zhang (whose rotary-wing model covers steady straight flight).

D-47 G4 closed as not met

Best 3-run average 0.576 (E0) < 0.580; E3 0.565; E2 below the extra-run threshold. On a second unseen set the AI is clearly behind B2 (-0.050). By D-43 and the stopping rule (D-26) the still-crowd tuning stops; this is reported as a negative result with its causes (Chapter 8.7-8.8). Per the owner's decision 4 (D-38) the project moves to moving crowds (G5) with E0 as the starting agent.

D-48 Settings from G5 on

The owner chose, for every experiment from G5 on: adaptive NOMA (`env.access: noma_opt`) with imperfect SIC `sic_eps: 0.05` and rescue users admitted first, plus the acceleration limit (`uav.a_max_mps2: 4`) with the turning-energy term. Optimised OMA (`oma_opt`) is always reported next to it so H3 stays a fair test. The old settings stay selectable, so every earlier number remains reproducible. Before any G5 result, the baseline tables are re-run once under these settings.

D-49 Still-crowd fixes deferred

"We have been stuck here for a long time" (owner): the still-crowd fixes go to the backlog. The fault they target (the AI flies past the best spot) would also hurt on moving crowds, so the "move by" steering and longer training are built into the G5 plan instead of a separate round.

D-50 Roadmap and dashboard

The roadmap of 10-19 is replaced by stages 0-5 (`docs/roadmap.md`): 0 housekeeping and new defaults; 1 G5 (core); 2 disaster events; 3 realism and safety layer; 4 final evaluation and write-up

(core); 5 a simulated drone that really flies (last). The owner asked for a dashboard now, to show the work visually; it is a parallel track (Streamlit, approved) built on the same engine, so its numbers equal the experiments'.

D-51 No room for anticipation in today's scenarios

Pre-registered rule (step 3 plan): if a pilot that knows the crowd's whole future beats B6 by less than 0.02 coverage for every crowd type, the moving-crowd scenarios leave no room for an anticipating AI, and they are redesigned before G5. Result: +0.001 to +0.003 for every type (Chapter 7.11). The rule fired. G5 does not start on today's scenarios; the redesign direction is the owner's choice (options in Chapter 14.8 progress note).

Chapter 4. The simulator

Everything here is in `uav_dataset/make_dataset.py` (physics and scenarios) and is configured only in `uav_dataset/config.yaml`. The Gymnasium game (Chapter 6) and the dataset (Chapter 5) call the same functions, so they cannot disagree. Parameter sources and their verification status are in Chapter 10 and `docs/assumptions.md`.

4.1 Scenarios and seeds

Plain words. A scenario is one invented disaster area: how many people, how many crowds, how big, where the collapsed buildings are, how people move. A seed is the scenario's ID number; the same seed always produces exactly the same scenario.

Technically. `make_scenario(seed)` draws, with a NumPy generator seeded by `[seed, 0]` (domain randomisation):

- number of users from {100, 300, 600, 1000} (must be even, because NOMA works in pairs);
- number of clusters 2-5, cluster radius 50-200 m, share of scattered users 5-10%;
- the mobility model from the profile's list, the number of obstacles from its range;
- bandwidth = number of users x 50 kHz (D-03); pairing method from the config. New draws are always appended at the end, so adding a field never changes the earlier draws.

Seeds are split and never mixed: **train 0-99, validation 100-119, test 120-139.** Profiles (Chapter 3, D-15) override config values; `apply_profile(name)` merges them in place and always starts from an untouched copy of the base config.

Profile	Changes
default	1 x 1 km, 30 dBm, 0-5 obstacles, five mobility models, 10 Wh
`lowpower`	20 dBm
`bigarea`	2 x 2 km
`hard`	2 x 2 km, 20 dBm, 3-8 obstacles of 60-200 m, mobility only hotspot jump / surge / group drift / attraction, 16 Wh, baseline grid 14 x 14
`rl_static`	as `hard` but the crowd never moves (RL curriculum step 1)
`rl_static_g20`	as `rl_static` with a 20 x 20 heatmap for the agent

4.2 People

Plain words. Crowds are blobs of people around random centres, plus a few people scattered alone over the map. Every person belongs to a class (rescue, video, voice, iot, normal) that decides how much data speed they need to count as "served".

Technically. `make_users(seed, sc)` with generator `[seed, 1]`:

- cluster centres uniform in [radius, L - radius]; users split between clusters with Dirichlet(3) weights and a multinomial draw;
- positions: 2D Gaussian around the centre with standard deviation radius / 2, truncated at the radius (redrawn if outside);
- scattered users uniform over the map; everything clipped to the map;
- class drawn per user with the configured shares; `min_rate_bps` from the class.

Class	Share	Minimum rate	Status
rescue	10%	200 kbps	engineering assumption, not signed off
video	20%	500 kbps	same
voice	30%	50 kbps	same
iot	20%	10 kbps	same

Class	Share	Minimum rate	Status
normal	20%	100 kbps	same

4.3 Movement

`simulate_positions(seed, sc, users)` (generator `[seed, 3]`) moves everyone for 300 steps of 1 s, vectorised over users (no loop over people). It returns every position and velocity; `simulate()` keeps a snapshot every 30 s for the dataset.

Model	Behaviour
`static`	nobody moves (used for RL step 1)
`random_waypoint`	each person walks (0.5-1.5 m/s) to a random point, then picks a new one
`group_drift`	each crowd drifts together with its own heading and speed, bouncing off the edges; members wobble (0.3 m/s); scattered people walk alone
`attraction`	everyone walks to the nearest of 1-2 shelters and stops within 20 m of it
`hotspot_jump`	as group drift, but at a random moment between 60 and 180 s one crowd heads to a new place at 3 m/s
`surge`	everyone stands still until a random moment (60-180 s), then all run at 3 m/s to one point and pile up within 60 m

Speeds are engineering assumptions. An evacuation model (social-force-lite) is planned, not built.

4.4 Collapsed buildings (obstacles)

Plain words. Grey rectangles on the map. If the straight line from the drone to a person crosses one, that person's signal is weaker.

Technically. `make_obstacles(seed, sc)` (generator `[seed, 4]`) draws `n_obstacles` axis-aligned rectangles with sides 40-150 m (60-200 m in `hard`). `link_blocked()` tests every drone-user segment against every rectangle with the slab (Liang-Barsky) method, vectorised over drones and users. A blocked link gets **15 dB** extra loss (engineering assumption, sensitivity range 10-25 dB). The test is 2D (no building heights): a user inside a rectangle counts as blocked.

4.5 The radio channel (air to ground)

Plain words. The higher the drone flies, the more likely it has a clear line of sight to a person, but the farther away it is. That trade-off creates a best height. Line-of-sight links lose a little extra signal; blocked links lose a lot.

Technically. `channel_gain()` for drone positions (X, Y, H) and users (x, y), following Al-Hourani et al.:

```
d       = sqrt((x - X)^2 + (y - Y)^2 + H^2)                    3D distance (m)
theta   = (180 / pi) * arcsin(H / d)                            elevation angle in degrees
P_LoS   = 1 / (1 + a * exp(-b * (theta - a)))                   a = 9.61, b = 0.16
FSPL    = 20 * log10(4 * pi * fc * d / c)                       fc = 2 GHz
PL_dB   = FSPL + P_LoS * eta_LoS + (1 - P_LoS) * eta_NLoS       eta_LoS = 1 dB, eta_NLoS = 20 dB
        + 15 dB if the link crosses an obstacle
gain    = 10^(-PL_dB / 10)
```

The architecture document's version of this formula omits the NLoS term; we use the full mean path loss.

Fading (optional). With fading on, each link is drawn LoS with probability P_LoS and then gets Rician fading (K = 10 dB) if LoS or Rayleigh fading if NLoS, both with unit mean power. Off in the dataset and for the baselines (deterministic average).

Noise. Noise power = -174 dBm/Hz + 7 dB noise figure, times the bandwidth of each user's resource.

4.6 NOMA (and the OMA reference)

Plain words. Instead of giving every person their own slice of frequency, NOMA puts two people on the same slice: a person with a strong signal and a person with a weak one. The weak person gets most of the power. The strong person first removes the weak person's message from what they receive (successive interference cancellation, SIC) and then reads their own. NOMA is inside every coverage number in this project, but nobody chooses its settings yet: pairing and the 80/20 split are fixed.

Technically. `noma_rates(g, P, N0_density, B, pairing)` for one drone position:

1. **Pairing** (`pair_users`): `extreme` sorts users by gain and pairs the weakest with the strongest, the second weakest with the second strongest, and so on; `random` pairs at random and makes the lower-gain user of each pair the "weak" one.
2. **Resources:** N users, N/2 pairs; each pair gets bandwidth b = B / (N/2) and power p = P / (N/2) (total power and bandwidth are shared by everyone). Noise per pair N0 = noise density x b.
3. **Power split:** the weak user gets a_w = 0.8, the strong user a_s = 0.2.
4. **Rates:**

```
SINR_weak         = a_w p g_w / (a_s p g_w + N0)
SINR_strong->weak = a_w p g_s / (a_s p g_s + N0)        the strong user decoding the weak message
SINR_strong       = a_s p g_s / N0                       after SIC
R = b * log2(1 + SINR)
```

5. **SIC check** (`sic_feasible`): SIC works only if SINR_strong->weak >= SINR_weak; otherwise the strong user's rate is 0. Because roles are assigned by gain, this always holds in our pairings; a real failure needs imperfect SIC or need-based pairing (planned).
6. **OMA reference (FDMA):** every user gets bandwidth B / N and power P / N; R = (B / N) log2(1 + (P / N) g / (N0_density B / N)). Same total bandwidth and power as NOMA, which makes the comparison fair (hypothesis H3).

A user is **covered** if their NOMA rate is at least their class's minimum rate. **Coverage** is the share of users covered. Fairness is Jain's index (sum R)^2 / (N sum R^2).

4.7 Energy and the battery

Plain words. Keeping a drone in the air costs far more energy than its radio. Our drone is a small one (about 2 kg) that needs about 168 W just to hover. A surprising and important fact from the model: flying forward at about 10 m/s costs *less* than hovering (134 J/s against 176 J/s including radio and electronics), as with real helicopters.

Technically. Rotary-wing propulsion power (Zeng, Xu, Zhang, verified, Chapter 10):

```
P(V) = P0 (1 + 3 V^2 / Utip^2) + Pi * sqrt( sqrt(1 + V^4 / (4 v0^4)) - V^2 / (2 v0^2) ) + 0.5 d0 rho s A V^3
hover = P(0) = P0 + Pi = 79.86 + 88.63 = 168.49 W
```

with Utip = 120 m/s, v0 = 4.03 m/s, d0 = 0.6, rho = 1.225 kg/m^3, s = 0.05, A = 0.503 m^2 (our 20 N drone, D-13). Energy per 1 s step = P(V_horizontal) + 20 N x max(climb rate, 0) + P_tx / 0.4 + 5 W. Climbing term, amplifier efficiency 0.4 and circuit power 5 W are engineering assumptions. Battery 10 Wh by default (D-07), 16 Wh in `hard` and `rl_static`.

Situation	Energy per second
Hover, 30 dBm	176.0 J
5 m/s, 30 dBm	151.1 J
10 m/s, 30 dBm	133.5 J
15 m/s, 30 dBm	146.0 J

10 Wh lasts about 205 s of hovering; 300 s needs 14.7 Wh.

Acceleration limit and turning energy (added 2026-10-21, D-44 to D-46). A reviewer found that the drone could change its velocity instantly. Shaking back and forth (+10, -10 m/s every second) kept it in place at the cheaper forward-flight power: 0 m moved, 131 J/s instead of

174 J/s hovering, a battery saving that is impossible in reality. No earlier result used it. The fix, switched on with `uav.a_max_mps2: 4` (off by default until the switch for G5, so all earlier numbers stay bit-exact):

- velocity changes by at most a_max x 1 s per step (horizontal 4 m/s^2, vertical 3 m/s^2), and `go_to` brakes so it does not overshoot;
- accelerating, braking and turning cost extra: the rotors must carry a load factor kappa = sqrt(1 + (a/g)^2), so induced power grows as kappa^1.5. This is **our extension** of the Zeng model (which covers steady straight flight), labelled as such. Checked: with the limit on, shaking costs 175 J/s, slightly more than hovering, as it should. The limit barely changes earlier results (B6 0.741 to 0.739, B3 0.675 to 0.674 on `hard`), because pilots decide only every 10 s.

4.8 The drone position sweep (dataset)

`sweep_one()` evaluates every grid position (11 x 11 at 100 m steps), every altitude (12 levels, 50-1,000 m) and every power level (0, 10, 20, 30 dBm) for the users at t = 0, with obstacles and the average channel, and records coverage, sum rate, spectral efficiency, fairness, rescue outage, OMA coverage and sum rate, and the hover energy per step. Chapter 5 has the table layout.

4.9 What the simulator does not model yet

Fading in the dataset, building heights, terrain, wind, GPS error, control delays, real battery behaviour, the drone's own backhaul link, interference from other cells, more than one drone, imperfect SIC, bit-error rate, NOMA settings as decisions, and not knowing where people are. All are in the roadmap (Chapter 11).

Chapter 5. The dataset

Plain words. The dataset is a set of tables describing 140 invented disaster scenarios: who is where, how they move, where the collapsed buildings are, and how good the signal would be for every drone position, height and power. It is *synthetic*: generated from `config.yaml` and the seeds, so anyone can regenerate exactly the same data. The AI does not learn from these tables (it learns by playing the game in Chapter 6); the tables are for checking the physics, drawing charts and comparing NOMA with OMA.

5.1 How to regenerate it

```
cd uav_dataset
python make_dataset.py all       # about 1 minute; writes data/*.csv and two pictures
python export_excel.py           # one Excel workbook (close it in Excel first, or the write fails)
```

The data folder is not in git (D-11). `make_dataset.py quick` writes a small 3-scenario sweep for smoke tests.

5.2 Versions

Version	Date	What changed	Where it is
v1	10-14	200 users, 3 clusters (100 m), 10% scattered, 2 classes (rescue 0.2 Mbps, normal 0.1 Mbps), 10 MHz, 8 altitudes	kept in `data_v1/`
v2	10-14	varied scenarios (100-1,000 users, 2-5 clusters), 5 classes, bandwidth per user, pairing option, 12 altitudes up to 1,000 m, then moving crowds and the trajectory table	overwritten by v3
v3 (current)	10-15	collapsed buildings, energy from the propulsion model, 5 mobility models (incl. surge and hotspot jump), obstacles table	`data/`

5.3 Files and splits

Every table exists per split (`*_train.csv`, `*_val.csv`, `*_test.csv`): train = seeds 0-99, val = 100-119, test = 120-139. The Excel workbook `data/uav_noma_dataset.xlsx` (145 MB) holds all five tables as sheets with an extra `split` column.

Table / sheet	Rows (v3)	One row is
`scenarios`	140	one scenario
`obstacles`	341	one collapsed building
`users`	74,900	one person at t = 0
`trajectory`	823,900	one person at one snapshot (every 30 s for 300 s)
`sweep`	813,120	one drone position, altitude and power in one scenario (140 x 121 positions x 12 altitudes x 4 powers)

5.4 Columns

scenarios: `seed`, `n_users`, `n_clusters`, `cluster_radius_m`, `scatter_fraction`, `area_m`, `bandwidth_hz`, `pairing`, `mobility`, `n_obstacles`.

obstacles: `seed`, `obstacle_id`, `x_min`, `y_min`, `x_max`, `y_max` (metres).

users: `seed`, `user_id`, `x`, `y` (metres), `cluster_id` (-1 = scattered), `class`, `min_rate_bps`.

trajectory: `seed`, `t` (s), `user_id`, `x`, `y`, `vx`, `vy` (m/s, the velocity of the step starting at t), `cluster_id`, `class`.

sweep: `seed`, `x`, `y`, `h` (drone position and altitude), `p_dbm` (total transmit power), `pairing`, `coverage` (share of users served with NOMA), `sum_rate_mbps`, `spectral_eff` (sum rate / bandwidth), `fairness` (Jain), `outage_rescue` (share of rescue users not served), `coverage_oma`, `sum_rate_oma_mbps`, `energy_j_per_step` (hovering energy at that power).

5.5 What the workbook contains (read back on 2026-10-15)

The v3 workbook was read cell by cell with openpyxl. No empty cells in any sheet.

- **scenarios:** users per scenario 100-1,000 (mean 535); clusters 2-5 (mean 3.72); radius 50.0-198.8 m (mean 127.2); scatter 5.1-9.9%; area 1,000 m in all; bandwidth 5-50 MHz; obstacles 0-5 (mean 2.4); pairing `extreme` in all 140; mobility: group drift 33, surge 31, hotspot jump 28, random waypoint 24, attraction 24.
- **users:** voice 22,639, normal 15,023, iot 14,950, video 14,899, rescue 7,389; minimum rate 10-500 kbps (mean about 156 kbps).
- **trajectory:** t from 0 to 300 s; velocity components from -3.9 to +4.1 m/s.
- **sweep:** altitude 50-1,000 m; power 0-30 dBm; coverage NOMA mean 0.507 (maximum 0.990), OMA mean 0.500 (maximum 1.000); sum rate NOMA mean 83.3 Mbps, OMA mean 75.2 Mbps; fairness mean 0.674; rescue outage mean 0.650; energy per step 173.49-175.99 J.

Points to know: only `extreme` pairing appears; the sweep uses the t = 0 positions only; NOMA coverage never reaches 1.0 while OMA does (not investigated); the high rescue outage comes from the assumed 200 kbps rescue minimum rate.

5.6 Key charts

- users_preview_v3.png: four training scenarios, crowds and scattered users, rescue users circled.

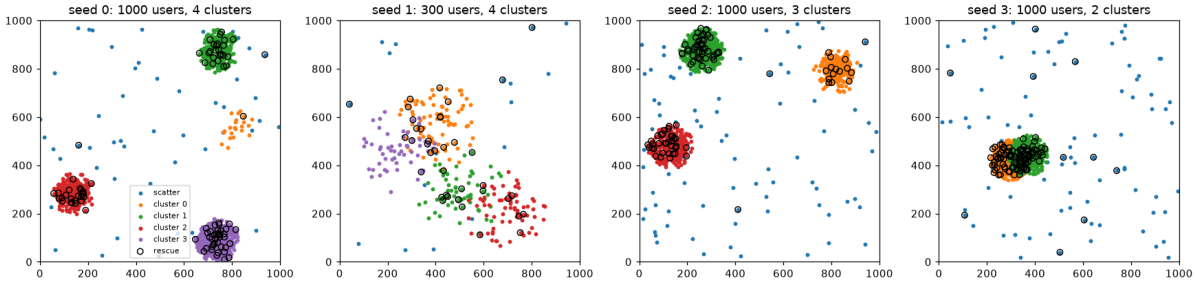


Figure: users_preview_v3.png

- altitude_check_v3.png: mean coverage against altitude, NOMA and OMA, per power level (all classes mixed).

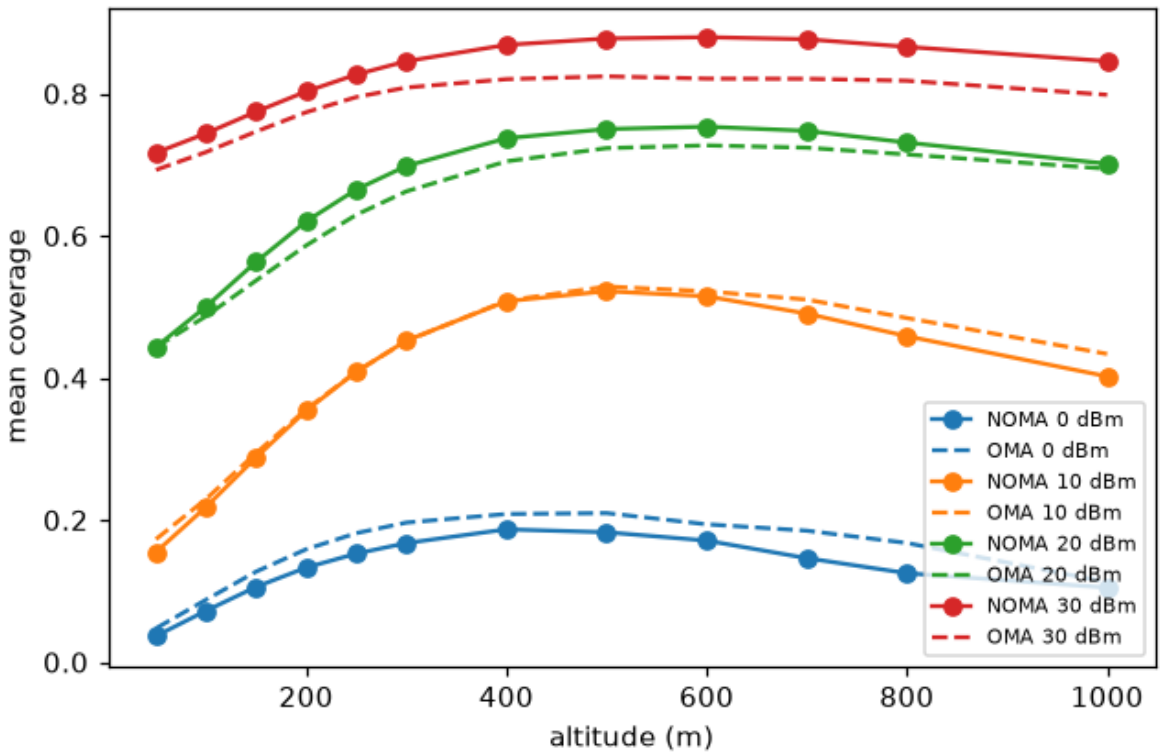


Figure: altitude_check_v3.png

- altitude_by_class_v2.png: the same per user class; every class has a best altitude.
- noma_vs_oma_v1.png: NOMA vs OMA sum rate and coverage per power level (dataset v1).

5.7 Is it enough data?

Count scenarios, not rows: 140. Enough for the physics checks, the NOMA-vs-OMA comparison and the baseline gate (100+ seeded scenarios). It is not training data for the AI. A "preview" workbook with a few scenarios was offered for the guide, because the full workbook is slow to open.

Chapter 6. The game and the AI

6.1 The game (Gymnasium environment)

Plain words. The "game" is one scenario played second by second. Each second the pilot (a simple rule or the AI) says how the drone should move and how strongly it should transmit; the people move; the game computes who is served and how much battery was used, and gives a score. Every pilot plays the same game with the same scenarios, which is what makes the comparison fair.

Technically. `UAVEnv` in `uav_dataset/uav_env.py` is a standard Gymnasium environment (`reset(seed)`, `step(action)` returning observation, reward, terminated, truncated, info). It passes Gymnasium's environment checker.

```
env = UAVEnv()                        # options: reward, fading, battery_wh, scenario_seeds, scenario_overrides, obs_
obs, info = env.reset(seed=3)         # seed = scenario seed: same seed, same crowd, buildings and movement
obs, reward, terminated, truncated, info = env.step(action)
```

State of the drone. Starts at the map centre at 150 m with a full battery. Position stays inside the map and between 50 and 300 m; horizontal speed is capped at 15 m/s, vertical at 5 m/s.

Action (4 numbers in [-1, 1]): `[vx, vy, vz, power]`. vx and vy are scaled by 15 m/s (the speed vector is capped at 15 m/s), vz by 5 m/s; power maps linearly to 0..P_max (never fully off, minimum 1 mW).

Step. Move the drone; advance the users one second along their scenario trajectory; compute the channel to every user (Chapter 4) and the NOMA rates; coverage = share of users at or above their minimum rate; subtract the energy of the step from the battery.

Reward. Each part is reported separately in `info` so reward hacking can be spotted:

- `reward_coverage` = coverage this second (version v1 uses only this);
- `reward_energy` = -0.05 x (energy this step / hovering energy) in version v2;
- `reward_terminal` = -1.0 x (share of users not covered) when the battery runs out.

End of the episode. `terminated` when the battery is empty; `truncated` after 300 s.

Observation (default 211 numbers, all scaled into [-1, 1]), built from named groups (D-22):

Group	Numbers	Content
`pos`	3	x / map size, y / map size, altitude scaled between 50 and 300 m
`vel`	3	own velocity / maximum speed
`battery`	1	battery left as a fraction
`time`	1	time as a fraction of 300 s
`prev`	3	last coverage, last throughput (sum rate / twice the sum of minimum rates, capped at 1), last fairness
`heat_all`	100	a 10 x 10 map of where users are (share of users per cell x 10, capped at 1)
`heat_rescue`	100	the same for rescue users only

`UAVEnv(obs_groups=...)` drops groups; `env.observation(groups)` lets a trained agent ask for the subset it was trained on.

Other tools in the environment.

- `evaluate(X, Y, H, power)`: coverage the drone *would* get at candidate spots, for the current users, without changing the game (used by the search baselines B3-B6).
- `scenario_seeds=[...]`: when training, `reset()` draws scenarios only from this pool (the training seeds).

- `scenario_overrides={...}`: replace scenario fields (users, clusters, mobility, obstacles, radius, scatter) to build your own.

6.2 How the AI controls the drone (wrappers)

`uav_dataset/rl_utils.py`.

Control modes (`ControlWrapper`).

Mode	The AI outputs	Held fixed
`xy` / `xyz` / `xyzp`	velocity (horizontal / + vertical / + power)	altitude and/or full power
`goto` / `gotoz` / `gotozp`	a spot on the map (/ + a target altitude / + power)	altitude and/or full power

Waypoint ("goto") controls were introduced because velocity-controlled agents drifted away from the crowd (D-19): with a spot as the action, staying put just means repeating the same spot. The drone flies to the spot at full speed and stops there.

Decision rhythm (`HoldWrapper`). The AI decides once every `hold` seconds (default 10, D-20); its choice is re-applied each second and the rewards of those seconds are added up. A game becomes 30 decisions instead of 300.

Adapters let the AI and random play through the same runner as the baselines: `RLPolicy(model, controls, hold, obs_groups)` and `RandomPolicy(controls)`. `model_settings(path)` reads the settings saved next to a model (D-23); old models without a hold setting default to deciding every second.

6.3 The AI's brain

Plain words. The AI is a small neural network. It looks at the observation and outputs an action. PPO (Proximal Policy Optimization) is the method that adjusts the network after many games so that actions that led to more coverage become more likely.

Technically. Stable-Baselines3 PPO with a Gaussian policy (actions are sampled around the network's output with a learned "exploration noise" during training; the network's output itself is used at test time). Two network choices (`--policy`):

- `mlp`: two hidden layers of 64 units for the policy and the value function;
- `cnn` (`uav_dataset/rl_net.py`, `CnnMapExtractor`): the two maps are read as a 2-channel picture plus two coordinate channels (CoordConv) by two 3 x 3 convolution layers (16 and 32 filters) and a 128-unit layer; the scalar inputs go through a 32-unit layer; the two are joined, followed by the 64-64 heads. It adds no information, only a better way to read the maps.

6.4 Training (`rl_train.py`)

All settings come from `config.yaml` (`rl:`); flags override them:

Setting	Value	Meaning
profile	`rl_static`	2 x 2 km, 20 dBm, 3-8 obstacles, still crowd, 16 Wh
controls	`goto`	the AI chooses where to fly
hold_steps	10	it decides every 10 s
reward	v1	coverage only (the energy cost is curriculum step 5)
total_timesteps	100,000 decisions	each decision is 10 simulated seconds
n_envs	4	four scenarios played in parallel
n_steps / batch_size / n_epochs	256 / 256 / 10	PPO update sizes
learning_rate	0.0003	
gamma / gae_lambda / clip_range	0.995 / 0.95 / 0.2	
ent_coef	0	
log_std_init	-1	starting exploration noise exp(-1) = 0.37
std_end	null	if set (e.g. 0.05), the noise shrinks linearly to it (D-27)

Setting	Value	Meaning
norm_reward	true	rewards are rescaled (episode returns are about 150)
policy / obs_groups	mlp / all	
val_eval_every_steps / val_eval_scenarios	20,000 / 8	progress check on validation scenarios during training
checkpoint_every_steps	10,000	

Training uses only training seeds 0-99 (D-18). During training it prints `VAL @ <steps>: coverage_full ...` on the first eight validation scenarios, logs every reward part separately (TensorBoard tags `reward/coverage`, `reward/energy`, `reward/terminal`, `val/coverage_full`), saves checkpoints and finally `results/rl/<name>/model.zip` with `config.json`.

```
pip install -r requirements-rl.txt
python rl_train.py --name my_run                              # defaults from config.yaml
python rl_train.py --policy cnn --std_end 0.05 --name e6      # flags override
```

Speed on the owner's laptop: about 1,000 simulated seconds per second plugged in, about 160 on battery; with the 10-second hold, about 155 decisions per second for one run alone.

6.5 Evaluation and pictures

- `rl_eval.py --model <zip> --split val` plays the model, random and the chosen baselines on the same scenarios of a split and prints mean +/- 95% CI and paired differences (Chapter 7 explains the statistics). Test seeds are for the end only.
- `view_run.py` plays one scenario with one pilot and saves a GIF and the last frame; `compare_run.py` puts several pilots on one scenario in one picture (coverage over time plus each flight path).

```
python view_run.py --seed 9 --users 600 --clusters 4 --mobility surge --obstacles 6 --policy B2
python compare_run.py --seed 105 --profile rl_static --policies random,B2,B3,ppo --model results/rl/<name>/model.zip
```

6.6 Curriculum (from the plan)

1. Move only, still crowd (current, Gate G4).
2. Moving crowds.
3. Add altitude.
4. Add power.
5. Battery-aware reward (v2).
6. NOMA grouping and power split as decisions. Each step must work before the next is switched on.

Chapter 7. Baselines and their results

7.1 The simple pilots

Plain words. Before trusting an AI pilot we need strong rivals that do not learn: if the AI only beats "a drone that does nothing", it proves nothing. Each rival plays exactly the same game (`env.step()`), on exactly the same scenarios.

ID	Name	What it does	Settings
B0	Fixed	sits at the map centre at 150 m and full power, forever	
B1	Follow the crowd	every second, flies toward the centre of the biggest true cluster	uses true cluster ids
B2	Weighted k-means	every 10 s, clusters the users (k = 3, rescue users count 3x) and flies to the heaviest cluster centre	uses true positions
B3	Grid search	every 10 s, scores a grid of candidate spots (10 x 10, or 14 x 14 on 2 km maps) plus the current spot at 150 m and flies to the best	uses `env.evaluate`
B4	Particle swarm (PSO)	every 10 s, searches position, altitude and power with 12 particles x 10 iterations; objective = the v2 reward	
B5	Energy-aware	like B3, but moves only if the coverage gain x 10 s exceeds the extra energy cost of moving; that cost is clamped at 0 because forward flight is cheaper than hovering, so B5 behaves like B3 (D-14)	
B6	Grid search + altitude	like B3, but tries 5 altitudes (50, 100, 150, 200, 300 m) at every grid spot	added 10-19 (D-16)
B7	Circling pilot	chooses spots like B6, but circles them (radius 50 m) at the speed that uses the least power (9.8 m/s) instead of hovering	added 10-21 (D-45, 7.10)
B3obs, B3obsU, B3obsUO	Information ladder	B3's search with only the AI's capped maps / uncapped maps / plus buildings; diagnostic only, not in the default runs	added 10-21 (D-37, Chapter 8.7)

The baselines see the true user positions (privileged information); the AI only sees maps. That makes them a strong test. Code: `uav_dataset/baselines.py`; settings in `config.yaml` (`baselines:`).

7.2 How results are measured

- **coverage_full**: the share of users served, averaged over the whole 300 s, counting 0 after the battery dies (the main metric).
- **coverage_alive**: the same, only over the seconds the drone was flying.
- **survived the horizon**: share of scenarios where the battery lasted 300 s.
- **served-user-seconds per Wh**, **rescue outage** (share of rescue users not served), **fairness** (Jain's index).
- Every value is **mean +/- 95% confidence interval** over scenarios (`evalstats.mean_ci`, 1.96 standard errors).
- **Paired differences** (`evalstats.paired_diff`): for every scenario, pilot A's score minus pilot B's; then mean and CI of those differences. Fairer than comparing two averages because it cancels "that scenario was easy for everyone". A `*` means the CI excludes 0, so the difference is not luck (a tiny `+0.000*` is not a real gap).

Reproduce: `cd uav_dataset && python run_baselines.py [--profile <name>] [--battery_wh <Wh>] [--seeds N]` (results go to `uav_dataset/results/`, not committed; the tables below are copies).

7.3 What the results say (summary)

1. **Default scenarios, 10 Wh:** no pilot survives 300 s (hovering needs 14.7 Wh), so the ranking mostly measures survival time. B4 has the best coverage while alive (0.913) but ties B0 overall because it dies first.

2. **Moving costs less than hovering** in this energy model (134 vs 176 J/s), so pilots that move last longer.
3. **Default scenarios, 16 Wh (everyone survives):** B4 0.914, B3 0.909, B5 0.908, B2 0.887, B1 0.884, B0 0.882. A parked drone already covers 88%: very little headroom. B1 and B2 do not significantly beat B0.
4. `hard` **profile:** the parked drone falls to 0.428 and the best pilots reach 0.740: a 31-point gap. **H1 holds here.** Neither low power alone nor a big map alone produces it; the combination does.
5. **B6 ties B4** (0.741 vs 0.740): B4's lead over B3 was choosing a better altitude, not better placement. The bar for the AI on `hard` is about 0.74.
6. **On the still-crowd test used for RL (`rl_static`, validation seeds):** Random 0.413, B0 0.441, B1 0.600, B2 0.610, B3 = B5 0.659; the best single spot reached instantly would give 0.686.
7. Caveats: baseline settings are untuned engineering choices; minimum rates are unsigned assumptions; one-step-greedy searches (B3, B4, B6) cannot anticipate where a crowd is going.

Bar charts: default 10 Wh, default 16 Wh, hard, 7 pilots. Example flights: hard scenario 5, surge custom scenario.

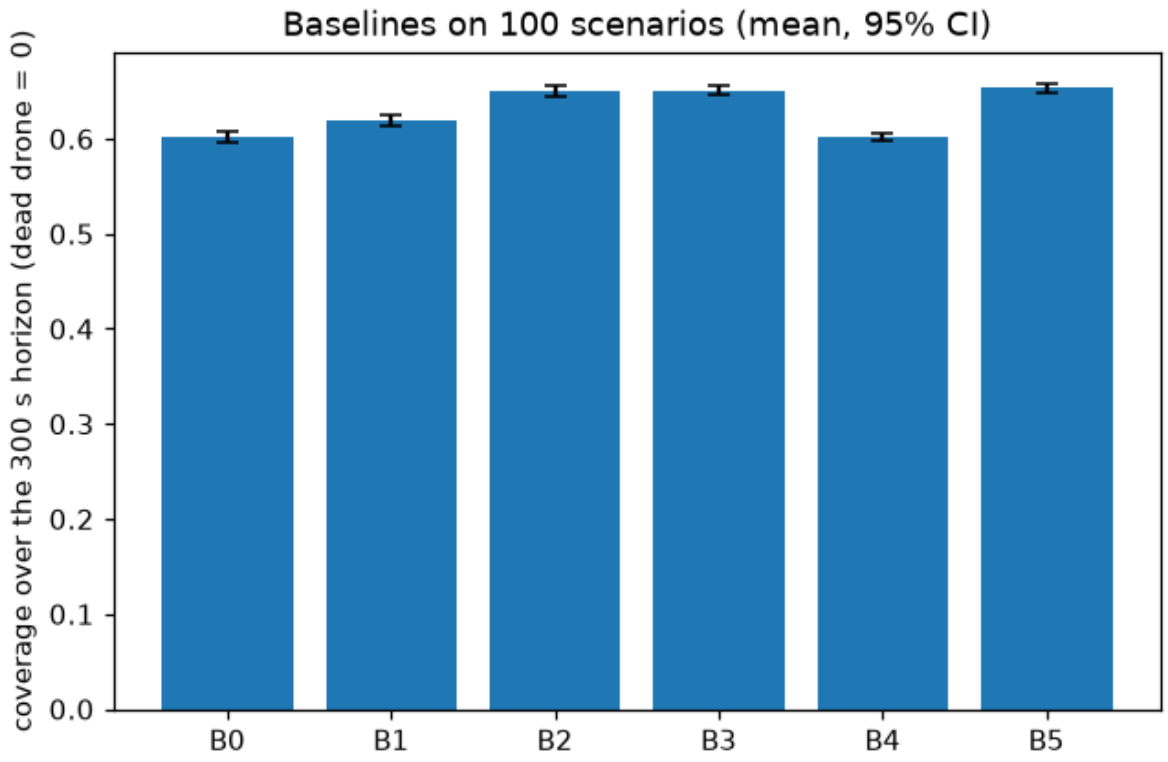


Figure: baselines_default_10wh.png

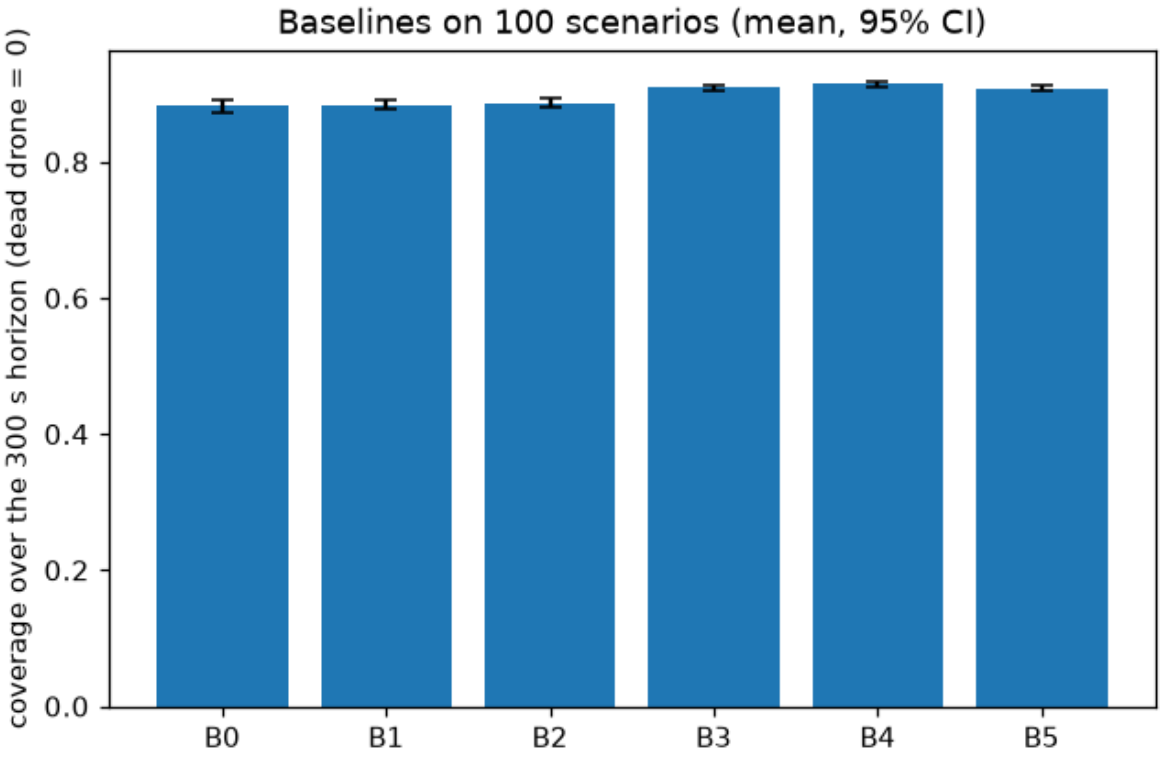


Figure: baselines_default_16wh.png

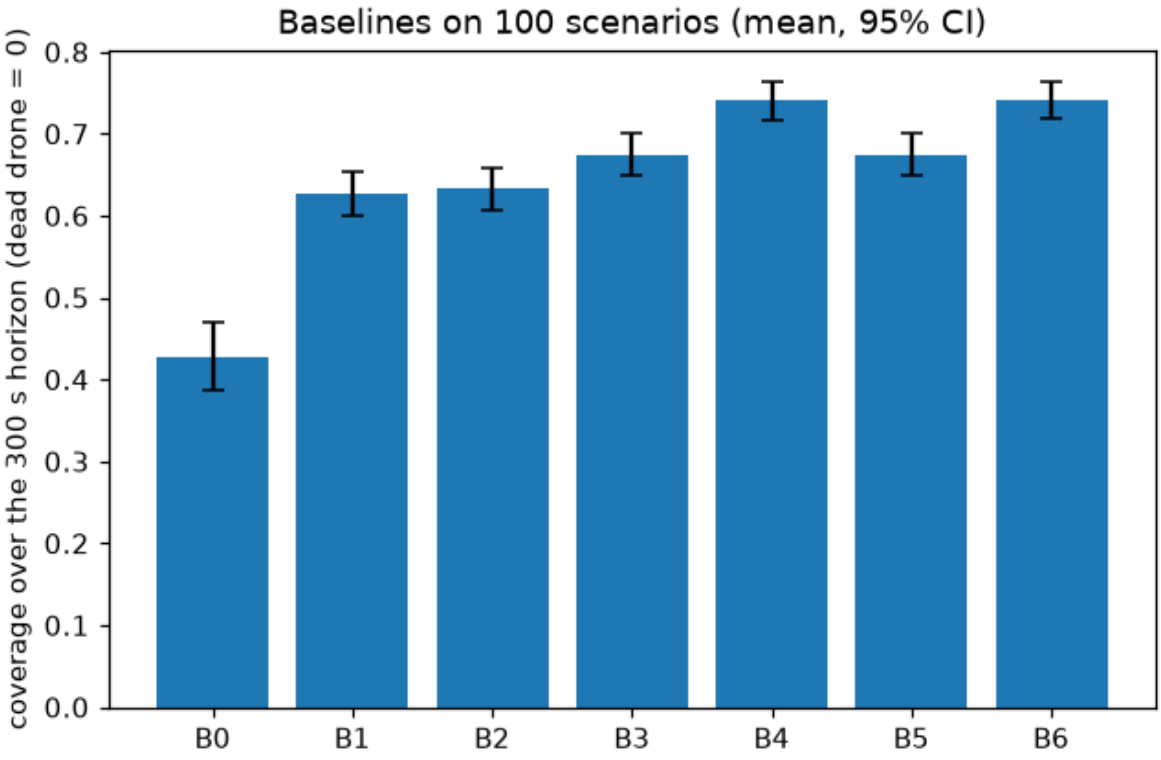


Figure: baselines_hard_7policies.png

7.4 Default scenarios, 10 Wh (100 scenarios, seeds 0-99)

Battery: 10 Wh, horizon 300 s.

Scenarios per policy: 100. Values are mean +/- 95% CI over scenarios.

Policy	coverage_full	coverage_alive	survived the horizon	served-user-s per Wh	rescue outage	fairness
B0	0.602 +/- 0.006	0.880 +/- 0.009	0%	10052.6 +/- 1208.2	0.073 +/- 0.024	0.729 +/- 0.008
B1	0.619 +/- 0.005	0.882 +/- 0.007	0%	10362.0 +/- 1249.4	0.101 +/- 0.025	0.689 +/- 0.006
B2	0.650 +/- 0.006	0.886 +/- 0.006	0%	10860.6 +/- 1310.1	0.089 +/- 0.021	0.694 +/- 0.005
B3	0.651 +/- 0.004	0.908 +/- 0.005	0%	10880.0 +/- 1302.9	0.040 +/- 0.013	0.726 +/- 0.005
B4	0.601 +/- 0.004	0.913 +/- 0.004	0%	10092.4 +/- 1213.9	0.021 +/- 0.005	0.778 +/- 0.005
B5	0.653 +/- 0.005	0.907 +/- 0.005	0%	10901.3 +/- 1302.3	0.039 +/- 0.013	0.727 +/- 0.005

Paired difference in coverage_full (row minus column; positive = row is better; * = the 95% CI excludes 0):

	B0	B1	B2	B3	B4	B5
B0	-	-0.017*	-0.048*	-0.049*	+0.000	-0.052*
B1	+0.017*	-	-0.031*	-0.032*	+0.017*	-0.035*
B2	+0.048*	+0.031*	-	-0.001	+0.049*	-0.003
B3	+0.049*	+0.032*	+0.001	-	+0.050*	-0.002*
B4	-0.000	-0.017*	-0.049*	-0.050*	-	-0.052*
B5	+0.052*	+0.035*	+0.003	+0.002*	+0.052*	-

Mean coverage_full by mobility model (scenario counts: group_drift 23, surge 22, hotspot_jump 21, random_waypoint 17, attraction 17):

mobility	B0	B1	B2	B3	B4	B5
attraction	0.605	0.623	0.656	0.656	0.609	0.664
group_drift	0.589	0.604	0.639	0.647	0.601	0.647
hotspot_jump	0.608	0.62	0.645	0.648	0.597	0.649
random_waypoint	0.606	0.616	0.657	0.657	0.598	0.658
surge	0.603	0.63	0.656	0.65	0.603	0.652

7.5 Default scenarios, 16 Wh (100 scenarios)

Battery: 16 Wh, horizon 300 s.

Scenarios per policy: 100. Values are mean +/- 95% CI over scenarios.

Policy	coverage_full	coverage_alive	survived the horizon	served-user-s per Wh	rescue outage	fairness
B0	0.882 +/- 0.008	0.882 +/- 0.008	100%	10068.6 +/- 1209.4	0.069 +/- 0.023	0.731 +/- 0.008
B1	0.884 +/- 0.007	0.884 +/- 0.007	100%	10343.3 +/- 1248.1	0.098 +/- 0.025	0.689 +/- 0.006
B2	0.887 +/- 0.006	0.887 +/- 0.006	100%	10850.0 +/- 1306.9	0.088 +/- 0.021	0.694 +/- 0.004
B3	0.909 +/- 0.005	0.909 +/- 0.005	100%	10907.3 +/- 1308.6	0.037 +/- 0.011	0.726 +/- 0.004
B4	0.914 +/- 0.004	0.914 +/- 0.004	100%	10308.0 +/- 1241.1	0.018 +/- 0.004	0.783 +/- 0.005
B5	0.908 +/- 0.005	0.908 +/- 0.005	100%	10924.7 +/- 1308.0	0.037 +/- 0.011	0.727 +/- 0.004

Paired difference in coverage_full (row minus column; positive = row is better; * = the 95% CI excludes 0):

	B0	B1	B2	B3	B4	B5
B0	-	-0.002	-0.005	-0.027*	-0.033*	-0.027*
B1	+0.002	-	-0.003*	-0.025*	-0.031*	-0.025*
B2	+0.005	+0.003*	-	-0.022*	-0.028*	-0.022*
B3	+0.027*	+0.025*	+0.022*	-	-0.006*	+0.000*
B4	+0.033*	+0.031*	+0.028*	+0.006*	-	+0.006*
B5	+0.027*	+0.025*	+0.022*	-0.000*	-0.006*	-

Mean coverage_full by mobility model (scenario counts: group_drift 23, surge 22, hotspot_jump 21, random_waypoint 17, attraction 17):

mobility	B0	B1	B2	B3	B4	B5
attraction	0.885	0.892	0.893	0.917	0.919	0.917
group_drift	0.866	0.866	0.874	0.9	0.911	0.9
hotspot_jump	0.889	0.88	0.884	0.905	0.911	0.905
random_waypoint	0.89	0.888	0.887	0.909	0.915	0.909
surge	0.882	0.895	0.897	0.914	0.918	0.914

7.6 Profile `lowpower` (50 scenarios, 10 Wh)

Profile: lowpower. Area 1000 m, max power 20 dBm, battery 10 Wh, horizon 300 s.

Scenarios per policy: 50. Values are mean +/- 95% CI over scenarios.

Policy	coverage_full	coverage_alive	survived the horizon	served-user-s per Wh	rescue outage	fairness
B0	0.552 +/- 0.023	0.796 +/- 0.032	0%	8699.8 +/- 1464.1	0.354 +/- 0.079	0.746 +/- 0.017
B1	0.581 +/- 0.017	0.817 +/- 0.023	0%	9403.9 +/- 1612.2	0.313 +/- 0.049	0.690 +/- 0.018
B2	0.617 +/- 0.018	0.826 +/- 0.022	0%	9948.8 +/- 1701.0	0.298 +/- 0.048	0.704 +/- 0.014
B3	0.616 +/- 0.014	0.859 +/- 0.019	0%	9986.7 +/- 1694.2	0.221 +/- 0.048	0.742 +/- 0.012
B4	0.582 +/- 0.010	0.880 +/- 0.015	0%	9548.8 +/- 1633.6	0.125 +/- 0.038	0.818 +/- 0.011
B5	0.617 +/- 0.014	0.859 +/- 0.019	0%	9990.6 +/- 1694.4	0.221 +/- 0.048	0.742 +/- 0.012

Paired difference in coverage_full (row minus column; positive = row is better; * = the 95% CI excludes 0):

	B0	B1	B2	B3	B4	B5
B0	-	-0.030*	-0.065*	-0.065*	-0.031*	-0.065*
B1	+0.030*	-	-0.036*	-0.035*	-0.001	-0.035*
B2	+0.065*	+0.036*	-	+0.000	+0.035*	+0.000
B3	+0.065*	+0.035*	-0.000	-	+0.034*	-0.000
B4	+0.031*	+0.001	-0.035*	-0.034*	-	-0.034*
B5	+0.065*	+0.035*	-0.000	+0.000	+0.034*	-

Mean coverage_full by mobility model (scenario counts: group_drift 14, hotspot_jump 11, random_waypoint 10, surge 8, attraction 7):

mobility	B0	B1	B2	B3	B4	B5
attraction	0.556	0.597	0.639	0.634	0.590	0.636
group_drift	0.531	0.532	0.582	0.585	0.563	0.585
hotspot_jump	0.571	0.607	0.626	0.629	0.592	0.629
random_waypoint	0.578	0.590	0.623	0.626	0.587	0.625
surge	0.525	0.607	0.638	0.626	0.589	0.626

7.7 Profile `bigarea` (50 scenarios, 10 Wh)

Profile: bigarea. Area 2000 m, max power 30 dBm, battery 10 Wh, horizon 300 s.

Scenarios per policy: 50. Values are mean +/- 95% CI over scenarios.

Policy	coverage_full	coverage_alive	survived the horizon	served-user-s per Wh	rescue outage	fairness
B0	0.522 +/- 0.018	0.765 +/- 0.027	0%	8298.9 +/- 1384.0	0.497 +/- 0.099	0.730 +/- 0.031
B1	0.582 +/- 0.016	0.812 +/- 0.022	0%	9420.7 +/- 1610.3	0.375 +/- 0.057	0.622 +/- 0.017
B2	0.627 +/- 0.017	0.821 +/- 0.021	0%	10135.2 +/- 1732.4	0.357 +/- 0.055	0.649 +/- 0.014
B3	0.603 +/- 0.014	0.845 +/- 0.019	0%	9786.4 +/- 1665.5	0.306 +/- 0.060	0.675 +/- 0.017

Policy	coverage_full	coverage_alive	survived the horizon	served-user-s per Wh	rescue outage	fairness
B4	0.589 +/- 0.010	0.876 +/- 0.013	0%	9639.5 +/- 1647.1	0.209 +/- 0.046	0.746 +/- 0.013
B5	0.604 +/- 0.014	0.844 +/- 0.019	0%	9788.0 +/- 1665.7	0.307 +/- 0.060	0.675 +/- 0.017

Paired difference in coverage_full (row minus column; positive = row is better; * = the 95% CI excludes 0):

	B0	B1	B2	B3	B4	B5
B0	-	-0.060*	-0.104*	-0.081*	-0.066*	-0.081*
B1	+0.060*	-	-0.044*	-0.021*	-0.006	-0.021*
B2	+0.104*	+0.044*	-	+0.023*	+0.038*	+0.023*
B3	+0.081*	+0.021*	-0.023*	-	+0.015*	-0.000
B4	+0.066*	+0.006	-0.038*	-0.015*	-	-0.015*
B5	+0.081*	+0.021*	-0.023*	+0.000	+0.015*	-

Mean coverage_full by mobility model (scenario counts: group_drift 14, hotspot_jump 11, random_waypoint 10, surge 8, attraction 7):

mobility	B0	B1	B2	B3	B4	B5
attraction	0.527	0.599	0.645	0.615	0.600	0.613
group_drift	0.493	0.541	0.598	0.567	0.568	0.567
hotspot_jump	0.529	0.610	0.641	0.625	0.598	0.625
random_waypoint	0.532	0.582	0.613	0.616	0.599	0.616
surge	0.549	0.602	0.658	0.612	0.589	0.615

7.8 Profile `hard`, all 7 pilots (100 scenarios, 16 Wh)

Profile: hard. Area 2000 m, max power 20 dBm, battery 16 Wh, horizon 300 s.

Scenarios per policy: 100. Values are mean +/- 95% CI over scenarios.

Policy	coverage_full	coverage_alive	survived the horizon	served-user-s per Wh	rescue outage	fairness
B0	0.428 +/- 0.042	0.428 +/- 0.042	100%	4207.4 +/- 664.6	0.846 +/- 0.034	0.504 +/- 0.041
B1	0.627 +/- 0.027	0.627 +/- 0.027	100%	7157.9 +/- 870.4	0.535 +/- 0.031	0.538 +/- 0.021
B2	0.633 +/- 0.027	0.633 +/- 0.027	100%	7575.5 +/- 901.2	0.524 +/- 0.031	0.562 +/- 0.019
B3	0.675 +/- 0.025	0.675 +/- 0.025	100%	7680.0 +/- 901.9	0.506 +/- 0.029	0.592 +/- 0.018
B4	0.740 +/- 0.023	0.740 +/- 0.023	100%	8079.1 +/- 936.5	0.417 +/- 0.036	0.709 +/- 0.017
B5	0.675 +/- 0.025	0.675 +/- 0.025	100%	7680.6 +/- 903.0	0.505 +/- 0.029	0.593 +/- 0.018
B6	0.741 +/- 0.023	0.741 +/- 0.023	100%	8004.4 +/- 926.0	0.419 +/- 0.038	0.715 +/- 0.017

Paired difference in coverage_full (row minus column; positive = row is better; * = the 95% CI excludes 0):

	B0	B1	B2	B3	B4	B5	B6
B0	-	-0.199*	-0.205*	-0.247*	-0.312*	-0.247*	-0.313*
B1	+0.199*	-	-0.006	-0.048*	-0.114*	-0.048*	-0.114*
B2	+0.205*	+0.006	-	-0.042*	-0.108*	-0.042*	-0.108*
B3	+0.247*	+0.048*	+0.042*	-	-0.065*	+0.000	-0.066*
B4	+0.312*	+0.114*	+0.108*	+0.065*	-	+0.066*	-0.000
B5	+0.247*	+0.048*	+0.042*	-0.000	-0.066*	-	-0.066*
B6	+0.313*	+0.114*	+0.108*	+0.066*	+0.000	+0.066*	-

Mean coverage_full by mobility model (scenario counts: attraction 27, surge 25, group_drift 25, hotspot_jump 23):

mobility	B0	B1	B2	B3	B4	B5	B6
attraction	0.482	0.665	0.664	0.700	0.770	0.700	0.769
group_drift	0.419	0.646	0.652	0.701	0.760	0.701	0.757
hotspot_jump	0.376	0.575	0.577	0.625	0.699	0.624	0.700
surge	0.427	0.615	0.630	0.667	0.727	0.667	0.730

7.9 Still crowd `rl_static`, validation seeds 100-119 (reference for the AI)

Split: val (20 scenarios, seeds 100-119). Profile: rl_static. Controls: xy. Area 2000 m, 20 dBm, battery 16 Wh.

Scenarios per policy: 20. Values are mean +/- 95% CI over scenarios.

Policy	coverage_full	coverage_alive	survived the horizon	served-user-s per Wh	rescue outage	fairness
B0	0.441 +/- 0.101	0.441 +/- 0.101	100%	2717.5 +/- 881.6	0.865 +/- 0.071	0.489 +/- 0.100
B1	0.600 +/- 0.076	0.600 +/- 0.076	100%	4190.8 +/- 1121.8	0.584 +/- 0.093	0.494 +/- 0.058
B2	0.610 +/- 0.068	0.610 +/- 0.068	100%	4484.4 +/- 1128.2	0.526 +/- 0.086	0.529 +/- 0.043
B3	0.659 +/- 0.069	0.659 +/- 0.069	100%	4576.6 +/- 1156.6	0.542 +/- 0.102	0.557 +/- 0.043
B5	0.659 +/- 0.069	0.659 +/- 0.069	100%	4576.6 +/- 1156.6	0.542 +/- 0.102	0.557 +/- 0.043
Random	0.413 +/- 0.091	0.413 +/- 0.091	100%	3193.6 +/- 932.4	0.870 +/- 0.056	0.459 +/- 0.095

Paired difference in coverage_full (row minus column; positive = row is better; * = the 95% CI excludes 0):

	B0	B1	B2	B3	B5	Random
B0	-	-0.158*	-0.169*	-0.218*	-0.218*	+0.028
B1	+0.158*	-	-0.011	-0.060*	-0.060*	+0.186*
B2	+0.169*	+0.011	-	-0.049*	-0.049*	+0.197*
B3	+0.218*	+0.060*	+0.049*	-	+0.000	+0.246*
B5	+0.218*	+0.060*	+0.049*	+0.000	-	+0.246*
Random	-0.028	-0.186*	-0.197*	-0.246*	-0.246*	-

Mean coverage_full by mobility model (scenario counts: static 20):

mobility	B0	B1	B2	B3	B5	Random
static	0.441	0.600	0.610	0.659	0.659	0.413

7.10 The circling pilot B7: does circling save battery? (plan step 5, 2026-10-21)

Plain words. In the energy model of Zeng, Xu and Zhang, a drone flying at about 10 m/s uses less power than one hovering (126 W against 168 W), as with helicopters, which get extra lift from forward flight. So instead of parking over the best spot, B7 flies a 50 m circle around it. It chooses its spots exactly like B6. All runs with the acceleration limit and the turning cost switched on, so the comparison cannot be won by the shaking trick.

Main test (D-45): fly until the battery is empty. Still crowds (`rl_static`), 10 Wh, up to 600 s, 50 scenarios:

Pilot	People-seconds served	Flight time	Coverage while flying	Mean power
B6 (hovers)	67,682	198 s	0.697	182.5 W
B7, circle 50 m	**85,033 (+25.6%*)**	**249 s**	0.696 (same)	144.9 W
B7, circle 100 m	83,725 (+23.7%*)	251 s	0.680	143.7 W

Circling serves **about 26% more people per battery** with the same coverage while flying. Simple arithmetic predicts +31% (power ratio 173.7 / 132.7 W); the measured gain is a little lower because both pilots pay the same fixed cost of flying to their spot. A 50 m circle beats a 100 m circle (the wider circle strays further from the best spot).

In the normal 5-minute mission (`hard`, moving crowds, 100 scenarios):

Battery	B6 (hovers)	B7 (circles)	Difference
12 Wh	0.596 (never survives)	**0.716**	+0.120*
13 Wh	0.652 (never survives)	**0.730** (survives 98%)	+0.079*
14 Wh	0.707 (survives 3%)	**0.731**	+0.024*
16 Wh	**0.739**	0.731	-0.009*

When the battery cannot cover hovering for the whole mission, circling wins clearly; when there is plenty of battery, hovering is slightly better (circling keeps the drone up to 50 m from the best spot).

Robustness (30 scenarios, one parameter changed at a time): the gain stays between +21% and +31%: drag d0 0.3 (the paper's value) +30.7%, induced power Pi +-20% +30.4% / +20.8%, acceleration limit 2 or 8 m/s^2 +24.9% / +26.2%, radius 100 m +24.4%. So the conclusion does not depend on the unverified d0 = 0.6.

Not verified. The turning-energy term is our own quasi-steady extension; whether Zeng's forward-flight formula holds in tight, slow circles is not verified (in this model even a 10 m circle is cheaper than hovering). Vertical acceleration is not charged. Runs used today's NOMA, no fading, full power.

Reproduce: `python accel_loiter_exp.py run b` (also `a`, `c`, `d`), then `python accel_loiter_exp.py report`.

7.11 How much could an AI that anticipates gain? The clairvoyant pilot (plan step 3, 2026-10-21)

Plain words. H2 says an AI should beat the search pilots on moving crowds because it can *anticipate*: B6 only reacts to where people are now. Before spending two weeks training, we measured the prize. A "clairvoyant" pilot that is **told the crowd's entire future** plans the best possible flight (dynamic programming over every reachable spot, every 10 s, on B6's own grid of 980 spots), and its plan is flown in the real game. No AI can do better than this pilot, so its lead over B6 is the most an anticipating AI could ever gain. Pass mark fixed beforehand (D-51): at least 0.02 for some crowd type.

Result (`hard`, 16 Wh, 120 scenarios, today's NOMA; paired with B6):

Crowd type	B6	Clairvoyant (dp)	Gain	Teleport bound (ignores flight time)
attraction (32)	0.776	0.778	+0.001	+0.015
group drift (29)	0.745	0.747	+0.002	+0.024
hotspot jump (26)	0.701	0.702	+0.001	+0.021
surge (33)	0.748	0.750	+0.003	+0.024
all (120)	0.745	0.746	+0.002*	+0.021

Knowing the whole future is worth +0.002 coverage: far below the 0.02 pass mark, for every crowd type. The same holds with adaptive NOMA (30 scenarios, +0.001) and, as expected, on still crowds (+0.000).

Why. B6 re-plans every 10 s and flies at 15 m/s, while crowds move at 0.5-3 m/s and the drone's coverage footprint is hundreds of metres wide, so a greedy pilot simply keeps up. Even during surges and jumps (60-180 s) a teleporting drone would add only +0.002. The only real gap is in the first minute (teleport bound +0.093): the unavoidable flight from the centre of the map to the crowd, which no amount of foresight can shorten (dp +0.004 there).

Side finding. A pilot that looks only 30 s ahead does *worse* than B6 (-0.011, significant): it will not start a trip whose payoff comes later. Half-way anticipation can lose to plain greedy re-planning.

What it means for H2 (decision D-51). In these scenarios there is no room for anticipation, so an AI cannot beat B6 by anticipating, however well it learns. This is a result in itself ("greedy re-planning is near-optimal when the drone is much faster than the crowd"). Not tested here: tight batteries (16 Wh never binds), where *when to move and how to spend the battery* matters, and where B7 already showed gains of +0.08 to +0.12. Before G5 the scenarios must be redesigned so that decisions now have consequences later, and the clairvoyant test re-run on them.

How it was checked. 22 tests (the coverage table equals the game's evaluation; the planner's first decision equals B6's; every planned move is flyable; the planner equals brute force on a small grid; the bound is never below the planner; replays are deterministic). I recomputed the gains from the raw episode file and re-ran B6 on one scenario: identical (0.7206), and B6's mean on seeds 0-99 equals the G3 table (0.741). **Not verified:** a finer grid (that would be placement, not anticipation); the planner is optimal for its model of the game, whose score is 0.005 above the replay.

Reproduce: `python oracle.py` (see its `--help`; coverage tables are cached in `results/oracle_cache/`).

Chapter 8. AI experiments

This chapter records every training run, what it showed and why. It supersedes the earlier summary in `docs/rl_g4.md`.

8.1 The task and the bar (Gate G4)

Task. Profile `rl_static`: a 2 x 2 km map, 20 dBm, 3-8 collapsed buildings, a crowd that stands still, 16 Wh. The AI only controls the drone's position (altitude 150 m, full power): curriculum step 1, the "hello world" of the project. **Gate G4:** the AI must beat random **and** match or beat B2 (not significantly worse in the paired comparison).

Level (validation seeds 100-119)	coverage_full
Random	0.413-0.430 (depends on the random controls used)
B0, parked in the middle	0.441
B2, k-means (the gate)	**0.610**
B3, grid search	0.659
Ceiling: best single spot, reached instantly	0.686

Rules (D-24, D-26): the AI gets raw measurements and the coverage score only; training on seeds 0-99; models compared on validation seeds 100-119; test seeds 120-139 untouched. Five planned experiments, then stop; one extra approved (D-27).

8.2 All runs

Run	Controls	Decides every	Network	Observation	Noise	Training	val8 during training	val20	vs B2 (paired)
`static_xy_v1_s0`	velocity	1 s	MLP	all	1.0 -> 0.93	300k steps	(training coverage 0.40 -> 0.49)	0.394	below random
`A_full`	velocity	1 s	MLP	all	0.37	1M steps	0.431 0.468 0.414 0.474 0.481 0.420 0.426 0.453 0.427 0.408	-	-
`goto_v1`	waypoint	1 s	MLP	all	0.37	500k steps	0.447 0.454 0.444 0.472 0.465	-	-
`goto_hold10`	waypoint	10 s	MLP	all	0.37 -> 0.32	60k decisions	0.477 0.486 0.481	0.525	-0.085 *
E2 `e2_mlp_lean`	waypoint	10 s	MLP	position + maps	0.37 -> 0.33	60k	0.481 0.466 0.492	0.543	-0.067 *
E3 `e3_cnn_full`	waypoint	10 s	**CNN**	all	0.37 -> 0.32	60k	0.496 0.502 0.516	**0.549**	-0.061 *
E3b `e3b_cnn_lean`	waypoint	10 s	CNN	position + maps	0.37 -> 0.34	60k	0.467 0.477 0.464	0.528	-0.082 *
E4 `e4_cnn_grid20`	waypoint	10 s	CNN	all, 20 x 20 maps	0.37 -> ~0.32	60k	0.487 0.520 0.509	0.545	-0.065 *
E5a `e5a_cnn_150k`	waypoint	10 s	CNN	all	0.37 -> 0.25	150k	0.496 0.502 0.516 0.512 0.532 0.514 0.516	**0.555**	**-0.055** *
E6 `e6_cnn_anneal`	waypoint	10 s	CNN	all	**0.37 -> 0.05 (annealed)**	60k	0.488 0.497 0.475	0.509	-0.102 *
E6b `e6b_mlp_anneal`	waypoint	10 s	MLP	all	**0.37 -> 0.05 (annealed)**	60k	0.476 0.498 0.490	0.536	-0.074 *

* = significant (the 95% CI of the paired difference excludes 0). Every run is one training seed; differences of about 0.02 between runs are within noise. `A_full` was a rerun after a session restart killed the first attempt at about 487k steps; it reproduced the first scores exactly (0.431, 0.468), which confirms that training is deterministic by seed.

8.3 What each step taught us

1. **Velocity controls: the drone finds the crowd and drifts away.** The first run scored below random. The picture 2_still_crowd_AI_vs_pilots.png (seed 105) shows the PPO drone reaching 0.8 coverage at about 30 s, like B3, then flying on and off the map edge (0.2). Hovering needs exactly zero velocity, and nothing rewards it. Training for 1M steps did not help (plateau 0.41-0.48).
2. **Waypoint controls stop the drift, but deciding every second makes it circle** (4_goto_checkpoint.png): coverage swings between 0.5 and 0.75.
3. **Deciding every 10 s stops the circling and learns about three times faster** (5_goto_hold10.png). First run that clearly beats random (+0.095, significant). Still below B2 (-0.085, significant).
4. **Experiment 1 (diagnosis):** the trained AI picks almost the same target from its very first decision and never changes it; it only passes through the good area on the way. Its target is 109 m, 348 m and 546 m from the best spot on seeds 110, 100 and

    105. Changing only its own velocity input moves the target 103 m; previous scores 89 m; the maps 266-445 m. It reads the maps but aims imprecisely, and is somewhat sensitive to inputs that should not matter for a still crowd.
5. **Experiments 2-4:** dropping inputs, a CNN, and a finer map all land within 0.03 of each other (0.528-0.549 on val20). **The brain and the inputs are not the bottleneck.**
6. **Experiment 5a:** 2.5 times more training creeps up slowly (peak 0.532 on val8).
7. **The exploration noise is the likely bottleneck.** In every run the noise ended at 0.32-0.34 (about +-330 m on a 2 km map), wider than a crowd (50-200 m). The score of each tried spot is dominated by that random wobble, so the AI cannot refine its aim. Experiment 6 shrank the noise from 0.37 to 0.05 over training (D-27). **It did not help** (val20 0.509 with the CNN, 0.536 with the MLP), so the noise was not the main bottleneck either. My diagnosis was wrong, and this is recorded as such.
8. **Best result: E5a, the CNN trained longest: 0.555 on val20, still significantly below B2 (-0.055).** Every variant lands between 0.51 and 0.56: network, inputs, map resolution, training length and exploration schedule each change the score by at most a few hundredths. The AI learns to head roughly for the crowd but does not aim precisely enough to match a pilot that evaluates candidate spots directly.

8.4 Status and what happens next

Gate G4 is **half met** (beats random, not B2). **The agreed stopping rule has been reached** (D-26): all five experiments and the extra noise experiment are done, and the best AI (0.555) is still significantly below B2 (0.610). Per the rule we stop tuning and report this result. The owner decides whether to relax G4 and move on to moving crowds, where one-step search pilots are always a step late and an AI that anticipates has a real chance. Other learners (SAC, a network with memory) remain options that give no hints.

Update 2026-10-21: step 1 of the agreed plan (Chapter 14.6) asked *why* the AI stalls. Answer in 8.7: about half of its gap to B3 is information it does not have, the other half is the AI not using what it does have.

8.7 Why the AI stalls: the information ladder and the train/validation check (plan step 1)

The question. The search pilots score spots with the true positions of every person, the buildings and each person's needs. The AI sees only two coarse 10 x 10 maps (everyone, rescue users), and those maps are **capped**: a cell holding more than 10% of the crowd simply reads "full". Is the AI stuck because it lacks information, or because it learns badly?

The method: an information ladder. Three new pilots run *exactly* B3's search (same candidate spots, altitude, power, rhythm, tie-break) but score the spots with less information. Each rung adds one piece:

Pilot	What it knows
B3obs	only the AI's own two capped maps; it rebuilds a stand-in crowd of 500 people at the cell centres (crowd size = engineering assumption, D-37), no buildings, needs from the population mix
B3obs U	the same maps **without the cap** (true share of people per cell)
B3obs U0	plus the true **buildings**
B3	plus exact positions inside each 200 m cell, the true crowd size and every person's needs

Results, validation (20 scenarios, seeds 100-119, profile `rl_static`), paired with the AI E5a:

Pilot	Coverage	vs the AI (paired)
B3 (everything)	0.659	+0.104*
B3obsUO (maps uncapped + buildings)	0.629	+0.074*
B3obsU (maps uncapped)	0.613	+0.059*
B3obs (exactly what the AI sees)	**0.605**	**+0.050***
B2 (k-means, true positions)	0.610	+0.055*
AI E5a	0.555	
Random	0.430	-0.125*

On 20 *training* scenarios (seeds 0-19, which are harder for every pilot): B3 0.603, B3obsU 0.586, B3obsUO 0.581, B3obs 0.561, B2 0.568, AI 0.538 (AI vs B3obs -0.023*). Steps along the ladder that are significant: cap removal +0.025* (train only), last rung (exact positions, crowd size, needs) +0.030* val and +0.022* train; the buildings step is not significant on either split.

Gap AI to B3	caused by missing information (B3obs to B3)	caused by the AI not using what it sees (AI to B3obs)
validation 0.104	0.054	0.050
training 0.065	0.042	0.023

Train vs validation (is it memorising?). The same models on 20 training scenarios and the 20 validation scenarios, as a share of B3's score: goto_hold10 0.78 / 0.80, e3 CNN 0.88 / 0.83, e5a CNN 0.89 / 0.84, e4 CNN 20 x 20 0.89 / 0.83. The AI is clearly behind B3 **even on scenarios it trained on** (e5a -0.065*), so the main problem is **underfitting** (it has not learned the task well), not memorising. The CNNs lose a little more on new scenarios (about 0.04 in the gap to B3); whether that difference is significant was not tested.

What the capped map hides. On the 10 x 10 map, 3.4 cells per scenario read "full" on average and they hold 59% of all people; the densest cell holds 25% on average (up to 54%). Parking over the best "full" cell gives 0.616 coverage, over the worst 0.435, and the AI cannot tell them apart. But the 20 x 20 map mostly removes the saturation (22% of people in full cells) and the AI scored the same (0.545), so the cap is not the main limit.

The key insight. B3 often parks **between** crowds, sometimes over an almost empty cell (its own cell is "full" in only 17 of 40 scenarios, but a neighbouring cell is in 35 of 40), so one footprint reaches several crowds. Flying to the truly densest cell gives only 0.565, which is where every AI version stalls (0.53-0.56). **The AI has learned "go to the crowd", not "find the spot that reaches several crowds".**

What it means.

1. The plateau is **half information, half learning.** A one-step search with only the AI's maps reaches 0.605, level with B2. So better learning with the *current* inputs could plausibly reach about 0.60, i.e. match B2 and pass G4; matching B3 needs more information (finer positions

and the crowd size matter most; buildings least).

2. G4 as defined (match B2) is therefore **not impossible by design**: an information-matched search already matches B2.
3. For step 4 of the plan: an uncapped map is cheap and correct to add; an obstacle map is the least promising extra input; a finer map or a count of people (which a base station knows from connected users) are more promising.

Sensitivity check (mine, after the agents' runs): B3obs with an assumed crowd of 200 / 500 / 1,000 people scores 0.588 / 0.605 / 0.606 on validation, always above the AI's 0.555, so the conclusion does not depend on that assumption.

Who did what. Two agents worked in parallel (owner's request): one built the three pilots, their tests and the ladder runs; the other ran the train/validation check and measured the map cap. I reviewed the code (the pilots read only what their rung allows), re-ran the 500-person result (identical), ran the sensitivity check and wrote this section.

Reproduce: `python rl_eval.py --policies Random,B0,B2,B3,B3obs,B3obsU,B3obsUO --split val --model results/rl/e5a_cnn_150k/model.zip` (and `--split train --seeds 20`). Tests: 19 new in `test_baselines.py` (the pilots ignore everything outside their rung).

8.5 Validation table: `goto_hold10`

Split: val (20 scenarios, seeds 100-119). Profile: rl_static. Controls: goto. Area 2000 m, 20 dBm, battery 16 Wh.

Scenarios per policy: 20. Values are mean +/- 95% CI over scenarios.

Policy	coverage_full	coverage_alive	survived the horizon	served-user-s per Wh	rescue outage	fairness
B0	0.441 +/- 0.101	0.441 +/- 0.101	100%	2717.5 +/- 881.6	0.865 +/- 0.071	0.489 +/- 0.100
B1	0.600 +/- 0.076	0.600 +/- 0.076	100%	4190.8 +/- 1121.8	0.584 +/- 0.093	0.494 +/- 0.058
B2	0.610 +/- 0.068	0.610 +/- 0.068	100%	4484.4 +/- 1128.2	0.526 +/- 0.086	0.529 +/- 0.043
B3	0.659 +/- 0.069	0.659 +/- 0.069	100%	4576.6 +/- 1156.6	0.542 +/- 0.102	0.557 +/- 0.043
B5	0.659 +/- 0.069	0.659 +/- 0.069	100%	4576.6 +/- 1156.6	0.542 +/- 0.102	0.557 +/- 0.043
PPO	0.525 +/- 0.084	0.525 +/- 0.084	100%	3972.2 +/- 1099.8	0.701 +/- 0.112	0.512 +/- 0.076
Random	0.430 +/- 0.093	0.430 +/- 0.093	100%	3241.9 +/- 978.9	0.872 +/- 0.060	0.477 +/- 0.099

Paired difference in coverage_full (row minus column; positive = row is better; * = the 95% CI excludes 0):

	B0	B1	B2	B3	B5	PPO	Random
B0	-	-0.158*	-0.169*	-0.218*	-0.218*	-0.084*	+0.011
B1	+0.158*	-	-0.011	-0.060*	-0.060*	+0.075*	+0.170*
B2	+0.169*	+0.011	-	-0.049*	-0.049*	+0.085*	+0.180*
B3	+0.218*	+0.060*	+0.049*	-	+0.000	+0.134*	+0.229*
B5	+0.218*	+0.060*	+0.049*	+0.000	-	+0.134*	+0.229*
PPO	+0.084*	-0.075*	-0.085*	-0.134*	-0.134*	-	+0.095*
Random	-0.011	-0.170*	-0.180*	-0.229*	-0.229*	-0.095*	-

Mean coverage_full by mobility model (scenario counts: static 20):

mobility	B0	B1	B2	B3	B5	PPO	Random
static	0.441	0.600	0.610	0.659	0.659	0.525	0.430

8.6 Validation table: E3 `e3_cnn_full` (best so far)

Split: val (20 scenarios, seeds 100-119). Profile: rl_static. Controls: None. Area 2000 m, 20 dBm, battery 16 Wh.

Scenarios per policy: 20. Values are mean +/- 95% CI over scenarios.

Policy	coverage_full	coverage_alive	survived the horizon	served-user-s per Wh	rescue outage	fairness
B2	0.610 +/- 0.068	0.610 +/- 0.068	100%	4484.4 +/- 1128.2	0.526 +/- 0.086	0.529 +/- 0.043
B3	0.659 +/- 0.069	0.659 +/- 0.069	100%	4576.6 +/- 1156.6	0.542 +/- 0.102	0.557 +/- 0.043
PPO	0.549 +/- 0.094	0.549 +/- 0.094	100%	3991.7 +/- 1220.6	0.658 +/- 0.111	0.507 +/- 0.079
Random	0.430 +/- 0.093	0.430 +/- 0.093	100%	3241.9 +/- 978.9	0.872 +/- 0.060	0.477 +/- 0.099

Paired difference in coverage_full (row minus column; positive = row is better; * = the 95% CI excludes 0):

	B2	B3	PPO	Random
B2	-	-0.049*	+0.061*	+0.180*
B3	+0.049*	-	+0.110*	+0.229*
PPO	-0.061*	-0.110*	-	+0.119*
Random	-0.180*	-0.229*	-0.119*	-

Mean coverage_full by mobility model (scenario counts: static 20):

mobility	B2	B3	PPO	Random
static	0.610	0.659	0.549	0.430

8.8 Track 2: a better learner, and the G4 verdict (2026-10-21)

What changed for the AI (D-40), all raw measurements a real drone has:

- an **uncapped** crowd map (square root of the share of people per cell, so no cell ever reads "full"), and the same for the rescue map;
- the **number of connected users** (log scale);
- its **own history**: where it was at its last 4 decisions and how many people it served there;
- **fresh training scenarios** (seeds 10,000-19,999, never validation or test) instead of the same 100 maps;
- a planning horizon (gamma) of 0.95, matched to 30 decisions per game. Three learners: **E0** PPO with these inputs; **E2** SAC (a learner that scores actions directly); **E3** SAC with a **training-only privileged critic** (owner's decision D-38): during training, the part that judges moves saw the true 20 x 20 crowd density, rescue users, buildings, people's needs and crowd size; the flying part never did. Three tests guard this: the actor's network has no input for that data, its output is bit-identical when only the privileged data changes (before and after training), and when flying the AI never asks the game for it.

The pass mark (D-43, set by the owner before E3's result): G4 passes if the 3-run average on the 20 validation scenarios is at least B2 - 0.03 = 0.580, and the AI clearly beats random. A version earns 2 extra runs only if its first run reaches 0.570.

Results on the 20 validation scenarios (bars: B2 0.610, B3obs 0.605, B3 0.659, Random 0.430):

Version	Run 1	Run 2	Run 3	3-run average	vs pass mark 0.580
E0 PPO, new inputs	0.573	0.574	0.581	**0.576 +/- 0.011**	short by 0.004
E2 SAC	0.567	(no extra runs: below 0.570)			
E3 SAC, privileged critic	0.571	0.572	0.551	**0.565 +/- 0.029**	short by 0.015

Version	Run 1	Run 2	Run 3	3-run average	vs pass mark 0.580
old best E5a (8.3)	0.555				

E0 (all 3 runs averaged per scenario) vs B2 -0.034 +/- 0.038, vs B3obs -0.029*, vs B3 -0.083*, vs Random +0.146*.

A second set of scenarios the AI never saw (seeds 0-19; bars: B2 0.568, B3obs 0.561, B3 0.603, Random 0.320): E0 0.524 / 0.515 / 0.517 (average 0.519, vs B2 -0.050*), E2 0.534 (vs B2 -0.034, not significant), E3 0.502 / 0.521 / 0.478 (average 0.501, vs B2 -0.068*). Here the AI is clearly behind B2, so the gap is real and not an artefact of the 20 validation scenarios.

Verdict: Gate G4 is NOT met under the owner's rule. The best version (E0) is 0.004 short on validation and about 0.05 behind B2 on the second set. By the pre-agreed stopping rule we stop tuning for the still crowd.

What we learned.

1. **Better inputs and fresh scenarios helped, steadily:** 0.555 to 0.576, with very little spread between runs (0.573-0.581).
2. **The privileged critic did not help** (E3 0.565, more spread, worst on the second set), although in theory it is the version where a privileged critic should matter most. SAC was not better than PPO and costs 3-4 times more computer time.
3. **The AI still does not stay at the best spot.** On seed 105 it reaches B3's spot at about 30 s (0.80 coverage), then flies on to the upper crowd behind a building and settles at 0.66, while B3 and B3obs stay and score 0.78 (10_e0_vs_b3_seed105.png). Over the 20 validation scenarios its final position is 264-291 m from B3's. It no longer simply heads for the densest cell (it ends there in 0-10% of games, B3 in 30%), but it does not find the spot between crowds either.

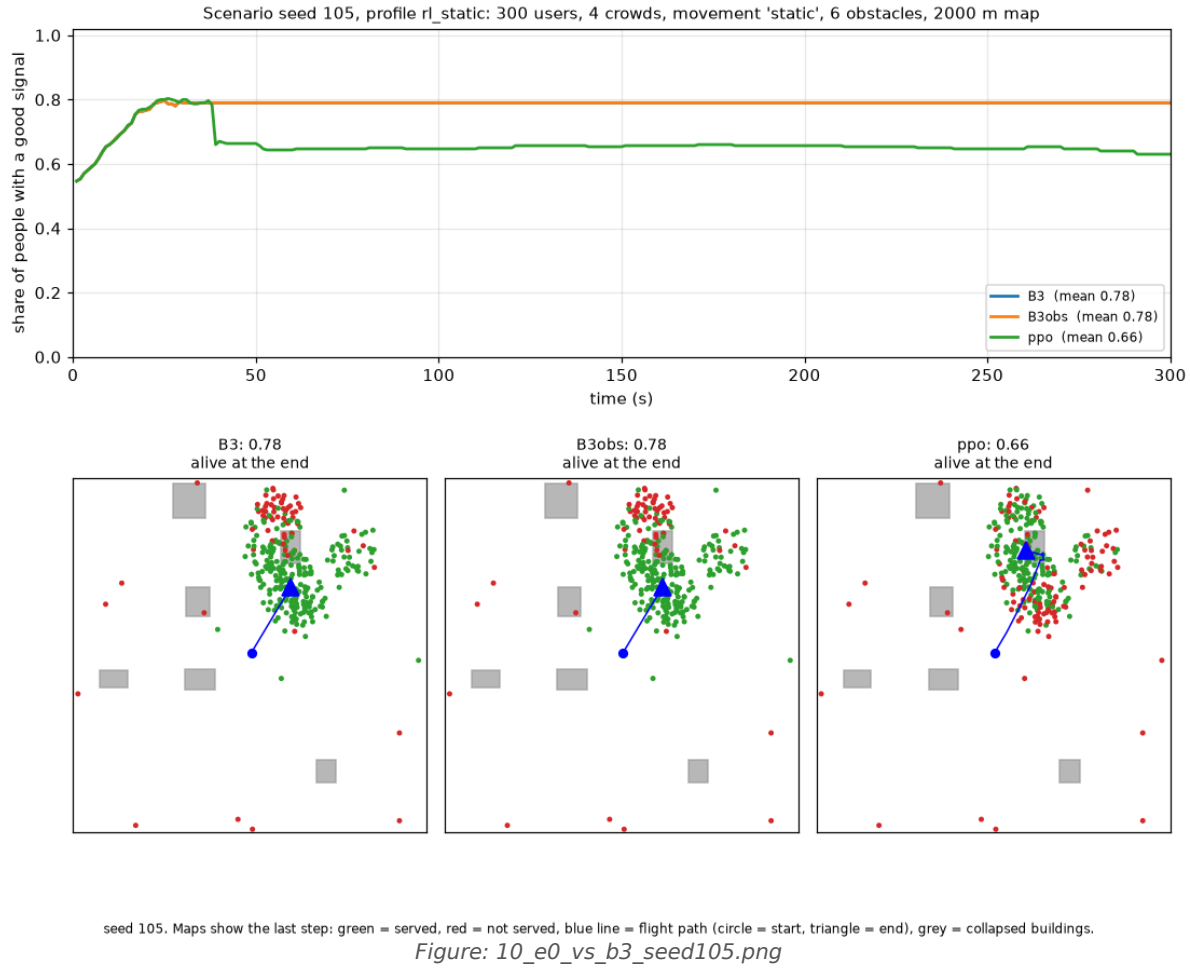


Figure: 10_e0_vs_b3_seed105.png

4. **The gap is learning, not information:** a plain search with exactly the same inputs (B3obsUN, uncapped maps plus the true crowd size) scores 0.606; with or without the crowd size or the cap, the search barely changes (0.605-0.613). The AI has the information; it does not turn it into the best position.

What comes next (owner's decision 4, D-38): move to moving crowds (G5), where a pilot that only reacts to the present is always a step late. Step 3's clairvoyant pilot measures first how much there is to win there. E0 is the starting agent.

Technical notes. PPO: 150k decisions per run, 41-79 min. SAC: 100k decisions, 80-111 min, replay buffer 50k (RAM limited), one gradient step per 4 decisions. CIs over scenarios use z = 1.96 (a t value would widen them by about 7%; no verdict changes). Reproduce: `python rl_train.py --algo PPO --obs_mode dict --gamma 0.95 --train_seeds 10000-19999 ...` (the exact settings are saved in each model's `config.json`); `python rl_eval.py --model <zip> --policies Random,B2,B3,B3obs,B3obsUN --split val`.

Chapter 9. Engineering

9.1 Repository

GitHub: `vathsa1703/Major_project` (public). Default branch `main`, protected. Collaborators: `vathsa1703` (owner); `Arani-Acharjee` invited with write access (pending acceptance on 2026-10-19). Sagar and Harshita not yet invited.

```
CLAUDE.md                project rules and current state (read at the start of every session)
CONTRIBUTING.md          how to work: branch, test, pull request, review
requirements.txt         simulator, baselines, tests (numpy pandas matplotlib pyyaml gymnasium pytest xlsxwriter ope
requirements-rl.txt      + stable-baselines3, torch, tensorboard (only for training and evaluating the AI)
requirements-docs.txt    + reportlab (only for rebuilding the book PDF with docs/book/build_pdf.py)
pyproject.toml           ruff (lint), pytest and coverage settings
.github/                 CI workflow, CODEOWNERS, pull-request template
docs/                    assumptions.md, g3_baselines.md, rl_g4.md, roadmap.md, book/ (this book)
uav_dataset/
  config.yaml            every setting and profile (change values here, never in the code)
  make_dataset.py        physics, scenarios, movement, obstacles, channel, NOMA, energy, dataset tables
  export_excel.py        CSVs -> one Excel workbook
  uav_env.py             the Gymnasium game
  baselines.py           pilots B0-B6
  run_baselines.py       runs pilots on many scenarios, writes the comparison table
  evalstats.py           mean +/- 95% CI, paired differences
  rl_utils.py            control modes, decision hold, AI and random adapters, model settings
  rl_net.py              CNN map reader (needs PyTorch)
  rl_train.py            PPO training
  rl_eval.py             AI vs random vs baselines on a split
  view_run.py            play one scenario with one pilot, save a GIF
  compare_run.py         several pilots on one scenario in one picture
  noma_vs_oma.py         H3: every pilot under NOMA and under OMA, paired comparison
  noma_picture.py        the NOMA pairs on the map and every user's speed under NOMA vs OMA
  access_benchmark.py    the four access schemes at each scheme's best spot (one snapshot)
  altitude_by_class.py   Gate G2 diagnostic (best altitude per user class)
  mobility_check.py      H1 check (fixed drone vs best position over time)
  test_*.py              315 tests
```

Not committed: `uav_dataset/data/`, `data_v1/`, `results/`, logs, Excel files, `docs/papers/` (local copies of papers).

9.2 How work flows (D-12)

1. Branch from `main`: `git checkout -b name/what-it-does`.
2. Before pushing: `python -m ruff check .` and `python -m pytest -q --cov`.
3. Open a pull request into `main` and fill in the template (what changed, which hypothesis or gate, checklist, how tested).
4. CI must be green; the code owner (`vathsa1703`) reviews; stale approvals are dismissed on new pushes; review conversations must be resolved. Merges so far were done by the owner's instruction with the admin bypass (the owner cannot approve his own PRs).

Branch protection on `main`: pull request required; 1 approving review from a code owner; dismiss stale reviews; status checks `test (3.11)` and `test (3.12)` required and the branch must be up to date; conversation resolution required; no force pushes, no deletion; admin enforcement off.

9.3 Continuous integration (`.github/workflows/ci.yml`)

Job	What it runs
`test (3.11)`, `test (3.12)`	install `requirements.txt`, `ruff check .`, `pytest -q --cov` (fails below 80% coverage on `make_dataset.py`, `uav_env.py`, `baselines.py`, `evalstats.py`), smoke run `make_dataset.py quick`
`rl smoke test (3.12)`	install `requirements-rl.txt` (CPU PyTorch), run `test_rl.py` and `test_rl_net.py` including a short PPO training

Lint rules: ruff E, F, W, I, B with line length 120; E741, E702 and E731 ignored (single-letter names, `a; b` and short lambdas are normal in the math code).

9.4 Tests (315 on 2026-10-21)

File	Tests	Covers
`test_noma.py`	9	gain vs distance, rate vs power, power split, SIC check (forced failure), pairing, scenario variety, class mix
`test_mobility.py`	19	bounds, snapshot count, reproducibility, speed limits, sudden events, attraction, group shape
`test_phase45.py`	13	obstacle geometry, blocked-link loss, fading on/off and mean power, energy model, battery
`test_env.py`	11	Gymnasium checker, determinism, limits, speed cap, battery end and penalty, truncation, reward versions, fading
`test_pipeline.py`	9	splits never overlap, config consistency, user ranges, sweep shape and values, NOMA vs OMA at high power, Jain index
`test_parameters.py`	7	values derived from Zeng's formulas, propulsion equation, FSPL reference, LoS probability, assumptions document coverage
`test_profiles.py`	9	profiles change only what they name, never leak, environment runs under each
`test_baselines.py`	62	statistics, helpers, every pilot valid and deterministic, B0-B6 behaviours, information-ladder pilots ignore what their rung may not see, runner accounting, report
`test_compare_run.py`	3	same scenario for every pilot, dead drone counts zero, determinism
`test_rl.py`	45	control modes, waypoint flight, decision hold, scenario pool, static crowd, reward parts, observation groups, model settings, viewer pilots, PPO smoke, Dict observation, uncapped maps, crowd size, history, fresh seed pool, privileged state only when asked, the AI never asks for it when flying
`test_rl_net.py`	11	CNN output size, reproducibility, position sensitivity, PPO with CNN, noise annealing, Dict extractor, the actor's output unchanged when only privileged data changes (before and after training), SAC save/load
`test_accel_loiter.py`	24	acceleration limit off = old results bit-exact, limit bound, shaking exploit closed, energy formula, braking without overshoot, B7 circle radius and speed, run-to-empty prediction
`test_oracle.py`	22	coverage table equals the game's evaluation, evaluate(t) leaves the game untouched, the planner's first move equals B6's, every planned move is flyable, planner equals brute force on a small grid, bound never below the planner, replay deterministic, never imported by the AI code
`test_dashboard_core.py`	16	dashboard logic without streamlit: same settings give the same disaster, trace equals the experiment code, maps render, comparisons, plan and battery advice
`test_access.py`	45	adaptive NOMA and optimised OMA: closed form, imperfect SIC, brute force on small crowds, power budget, monotone in power, odd crowds, rescue first, old schemes unchanged
`test_noma_access.py`	8	NOMA default, both schemes reported, OMA scoring, evaluate uses the tested scheme, NOMA total speed, H3 table and runner, NOMA picture

Bugs caught by tests or CI so far: the fading flag not reaching the channel (test), a missing `tabulate` dependency (CI), a policy adapter used before `reset()` (test).

9.5 Pull requests

PR	Merged	Title
#1	10-15	Add CI, lint config, code owners, PR template and pipeline tests
#2	10-19	Verify parameters against the papers; assumptions table and tests
#3	10-19	Baselines B0-B5, runner, stats and the Gate G3 comparison
#4	10-19	Config profiles (lowpower, bigarea, hard) and baselines on harder scenarios

PR	Merged	Title
#5	10-19	Add B6 (grid search + altitude): B4's lead was altitude choice
#6	10-19	RL pipeline (PPO), play-your-own-scenario viewer, RL CI job
#7	10-19	compare_run.py: compare several pilots on one scenario in one picture
#8	10-19	RL: waypoint controls + 10-second hold (PPO now beats random, not yet B2)
next	open	experiments 2-6, CNN, observation groups, roadmap, NOMA/OMA switch and H3, this book

The first commit (`4cd4cc4`, 2026-10-15) imported the generator, environment and 52 tests directly to `main` (the repository was empty); everything after went through pull requests.

9.6 Software versions (owner's laptop)

Windows 11, Python 3.14.7, numpy 2.5.3, pandas 3.0.6, matplotlib 3.11.2, pyyaml 6.0.3, gymnasium 1.4.0, pytest, xlsxwriter, openpyxl, stable-baselines3 2.9.0, torch 2.14.1 (CPU), tensorboard 2.21.0. CI uses Python 3.11 and 3.12 with the newest packages (dependencies are not pinned; a new release can break a build).

9.7 Commands

```
pip install -r requirements.txt                # simulator, baselines, tests
pip install -r requirements-rl.txt             # + the AI
cd uav_dataset
python make_dataset.py all && python export_excel.py
python -m pytest -q --cov                       # from the repository root
python run_baselines.py --profile hard
python rl_train.py --policy cnn --std_end 0.05 --name my_run
python rl_eval.py --model results/rl/my_run/model.zip --split val
python view_run.py --seed 5 --profile hard --policy B3
python compare_run.py --seed 105 --profile rl_static --policies random,B2,B3,ppo --model results/rl/my_run/model.zip
cd ..
pip install -r requirements-docs.txt && python docs/book/build_pdf.py   # rebuild the book PDF (about 1 minute)
```

9.8 Troubleshooting (things that actually happened)

Symptom	Cause	Fix
`ModuleNotFoundError: matplotlib`	packages not installed	`pip install -r requirements.txt`
`PermissionError` writing the Excel file	the file is open in Excel	close it, run `export_excel.py` again
CI fails with `Import tabulate failed`	a library installed locally but not listed	never rely on unlisted packages; fixed by a hand-written table
Training 6x slower than usual (about 160 steps/s)	laptop on battery, CPU throttled	plug in the charger (about 1,000 steps/s)
A training run disappeared	the session restarted and killed background jobs	re-run; training is deterministic by seed; checkpoints every 10k decisions
Two training runs side by side are slow	they share the CPU	set `OMP_NUM_THREADS=2`, or run one at a time
A run seems hung for hours, workers use almost no CPU	the laptop slept	keep it awake and plugged in; it resumes on wake
Commits show the wrong author	the machine's global git identity is another project's	`git config user.name/user.email` set inside this repository only
A model plays badly in the viewer	wrong controls or decision hold	models now carry their settings in `config.json` and the tools read them

Chapter 10. What is verified and what is not

Plain words. A simulator is only believable if its numbers come from somewhere. Every parameter is either checked against a published paper, derived from checked formulas, or openly labelled as our own assumption with a range to test. The full table is in `docs/assumptions.md`; this chapter summarises it and lists the other checks.

10.1 Status labels

- **VERIFIED**: read in the cited paper.
- **DERIVED**: follows from verified formulas for a drone we chose (the choice itself is an assumption).
- **CONSISTENT**: appears in secondary papers that cite the original; the original was not read.
- **STANDARD**: a physical constant or textbook convention.
- **ASSUMPTION**: our engineering choice, with a sensitivity range to test (experiment E13).

10.2 Summary

Area	Status	Notes
Energy formula (Zeng, Xu, Zhang, arXiv:1804.02238)	**VERIFIED**	propulsion power (Eq. 6), hover powers (Eq. 61), delta, k, rho, s
Our drone's P0, Pi, Utip, v0, A	**DERIVED**	a 20 N drone, R = 0.4 m, Omega = 300 rad/s; within 0.1% of the formulas. The paper's own drone is 100 N and hovers at about 1.37 kW
d0 = 0.6	unverified	the paper uses 0.3 for its own drone
Channel formulas (Al-Hourani et al., IEEE WCL 2014)	**CONSISTENT**	same form in arXiv:1802.01117 and arXiv:1705.03304; the original was not read
a = 9.61, b = 0.16, eta_LoS = 1 dB, eta_NLoS = 20 dB	**CONSISTENT**	appear together in two papers' tables at 2 GHz, without an environment label; "urban" not confirmed
Free-space path loss, thermal noise	**STANDARD**	FSPL at 1 km and 2 GHz = 98.46 dB (tested)
User-class minimum rates	**ASSUMPTION, not signed off**	video and rescue are often unreachable at low power
Obstacle loss 15 dB, fading K = 10 dB, PA efficiency 0.4, circuit power 5 W, battery 10/16 Wh, walking and surge speeds, reward energy weight 0.05	**ASSUMPTION**	ranges listed in `docs/assumptions.md`
NOMA power split 0.8 / 0.2, pairing strongest with weakest	**ASSUMPTION** (fixed design choice)	to become decisions in roadmap stage B

10.3 Other checks done

- **Altitude optimum (Gate G2):** reproduced on dataset v1 (peak at 0 dBm near 250 m) and per user class on v2 (Chapter 2, entry 7). The optimum moves lower as the rate target rises.
- **NOMA logic:** the power split sums to 1; more power to the weak user lowers the strong user's rate; a deliberately bad pair is flagged by the SIC check; rates rise with power; NOMA gives more total speed than OMA at full power (tests).
- **NOMA vs OMA in the game:** the same move gives the same NOMA and OMA numbers whichever scheme scores; the search pilots evaluate candidate spots with the scheme being tested (tests in `test_noma_access.py`).
- **Energy model:** hover = P0 + Pi; the power curve dips below hover at moderate speed; climbing costs extra; descending gives nothing back (our assumption); radio power is under 5% of flight power.

- **Game:** passes Gymnasium's checker; deterministic by seed; the drone never leaves its limits; the battery never goes negative.
- **Statistics:** paired comparisons on identical scenarios, 95% confidence intervals, fixed seed lists.
- **Training is reproducible:** a training run killed by a session restart was re-run with the same seed and printed the same validation scores.

10.4 Not done yet

1. **Read the Al-Hourani paper** (DOI 10.1109/LWC.2014.2342736) and confirm the table and which environment each row is. The owner is trying to get it (college library, IEEE Xplore, Google Scholar, or by asking the author). Save it as `docs/papers/Al-Hourani_2014_LAP_altitude.pdf` (git-ignored).
2. **MATLAB parity (Gate G1):** compare our gain, SINR and rate with reference numbers from the team's MATLAB code.
3. **Sign off the minimum rates** and the other assumptions, then run their sensitivity ranges (experiment E13).
4. **One real measurement:** a radio or phone on a mast or a cheap drone at several heights, compared with our altitude curve.
5. **A literature matrix** of 15-25 papers, every citation checked by a person (the plan's month-1 task).

10.5 What the results may and may not claim

May claim: comparisons between strategies inside this model, with the stated assumptions; qualitative lessons (an optimal altitude exists and depends on the rate target; forward flight can cost less than hovering; most of a pilot's advantage can come from choosing the altitude; NOMA's gain over OMA in total speed and coverage under stated conditions).

May not claim: absolute coverage numbers for a real disaster, real-world performance of the AI, or "digital twin". The data is synthetic and conclusions are conditional on the model (plan section 9).

Chapter 11. Outlook

11.1 Roadmap (agreed 2026-10-19, full version in `docs/roadmap.md`)

Stage	Content	Status
A	Close the still-crowd gate G4 (five experiments plus noise annealing), then stop tuning	experiments done; G4 half met
B	**NOMA becomes a decision**: an OMA switch for a clean H3 experiment in the game (built 10-19), then the AI chooses the power split and pairing; classical versions for the baselines; imperfect SIC and bit-error rate as ablations	in progress
C	Dynamic scenarios and events: moving crowds (curriculum steps 2-5), rising flood, evacuation, aftershock, survivors appearing, drone failure	planned
D	Realism: the AI does not know where people are; a real place (e.g. a Nepal flood) from public data; wind, GPS error, delays; backhaul; a safety layer; one real measurement	planned
E	Evaluation and write-up: E1-E14 with 5 seeds, generalisation, sensitivity, literature matrix, report, demo, viva	planned
F	**Last:** a simulated quadcopter with an autopilot model flying under our controller, no hardware	planned

11.2 Is it novel? (first literature check, 2026-10-19)

Two web searches, not a full review. Every paper must be read and checked by a person before it is cited.

- **Already published:** deep RL for UAV base stations with NOMA, including mobile users, 3D trajectories, power control, battery limits and disaster areas. Examples found:
  - NOMA in UAV-aided cellular offloading (Zhong et al., arXiv:2011.14776): k-means plus a deep Q-network for 3D trajectory and power, with roaming users;
  - AdaptSky (arXiv:2107.01004): joint NOMA power allocation and 3D placement;
  - a NOMA-DQN battery-aware path-planning paper (HAL hal-04762084);
  - a disaster-area UAV base-station study with NOMA pairing and power splitting;
  - multi-agent RL with action masking for UAV mobile communications (arXiv:2303.16737).
- **So the method is not new.** We cannot claim "first to apply RL to UAV-NOMA".
- **Possible contributions** (to be checked properly):
  1. an open, tested, reproducible simulator and benchmark (none found in the searches);
  2. a fair evaluation against strong non-learning pilots across regimes, with paired statistics (the papers seen compare against OMA, random or circular paths, according to the search snippets);
  3. specific findings: most of a pilot's lead can come from choosing the altitude; forward flight costs less energy than hovering; lessons on RL controls, decision rhythm and exploration noise;
  4. NOMA measured inside the game for every pilot, with confidence intervals (Chapter 13).
- **Defensible claim:** "an open, tested simulator and evaluation protocol for UAV-NOMA planning in disaster scenarios, a comparison against strong baselines across regimes, and an analysis of when RL helps". This matches the plan's own contribution statement.
- **How to strengthen it:** make NOMA a joint decision (stage B), test where RL should win (stage C), add a real place (stage D), and build the literature matrix.

11.3 The real-world gap

A real drone running this software would not perform as it does in the simulation. This is called the sim-to-real gap.

In the simulator	In reality
Pilots know where everyone is (the AI sees a density map)	Nobody knows where survivors are; it must be estimated from phone signals
The signal follows a statistical formula	Real buildings, terrain, trees, weather and interference
NOMA works perfectly	NOMA needs special base-station hardware and receivers; ordinary phones are believed not to support it (to be checked)
The drone moves exactly as told	Wind, GPS error, control delays, real battery behaviour
No rules	Aviation permission, a radio licence, a safety pilot
No backhaul	The drone needs its own link to the network (tether, microwave, satellite)

What carries over: the qualitative lessons and the method (compare strategies fairly before flying). What does not: the absolute numbers. The report must say so.

What already exists in practice (news sources from 2017-2018, may be out of date): AT&T's tethered "Flying COW" drone carried LTE radios to restore service in Puerto Rico after Hurricane Maria, and reportedly after Hurricane Ida in Louisiana. People place it; it uses no AI and no NOMA. NOMA was standardised for LTE downlink as "MUST" in Release 14, according to one survey; whether it is used in commercial networks was not found.

Why research often stays research: simple solutions already work; the sim-to-real gap; hardware is needed on both ends; regulation and safety; trust and liability; a small, slow market; and survivors' positions are unknown. These are also where the opportunity and the risk lie.

11.4 Hardware path (outside this project, in stages)

Our software is the "brain" that decides where the drone goes; a ready-made autopilot (ArduPilot or PX4) keeps it in the air. Our "fly to this spot" actions map directly onto autopilot waypoint commands (MAVLink).

Stage	What is done	Needs a drone
0	Software in the loop: our controller flies a simulated autopilot (roadmap stage F)	no
1	NOMA on the bench: two software-defined radio receivers and one transmitter; check pairing and SIC, measure bit errors	no
2	Measure the signal against height from a mast or building	no
3	Hover tests with a small drone carrying the radio	yes
4	Closed loop with a safety pilot or tether	yes
5	A pilot with a partner (an emergency team or telecom company)	yes

Needs: a mentor with radio or drone experience; India's DGCA drone rules; a radio licence or licence-free bands; and money (roughly a few hundred to a couple of thousand dollars per software-defined radio, from about a thousand for a capable drone; unchecked figures).

11.5 Business track

Novelty is not the test for a business; a paying customer with a painful problem is. For a student team the most realistic product is software only: a planning and rehearsal tool for emergency teams or telecom operators ("where to station drones, how many, how high, before flying"). It avoids hardware, flying and regulation, but whether anyone would pay is unknown, and drone cell sites already exist (above).

Next step, costing nothing: talk to 5-10 people who do this work (disaster agencies, telecom emergency teams, drone service companies) and ask what they do today, what goes wrong, and what it costs them. Then decide.

11.6 Open decisions for the owner

1. The minimum data rate for each user class (needed before any number goes into the report).
2. The battery size for the final experiments (10 Wh makes everyone die; 16 Wh makes everyone survive; a sweep is experiment E7).
3. Whether to relax Gate G4 and move to moving crowds (the agreed stopping rule has been reached; Chapter 8).
4. Sending Sagar's revised task and inviting Sagar and Harshita to the repository.
5. Getting the Al-Hourani paper and the MATLAB reference numbers.

Chapter 12. Glossary

Plain-word meanings first; the technical term in brackets where it helps.

Term	Meaning
UAV	Unmanned aerial vehicle: a drone.
UAV-BS, aerial base station	A drone carrying a mobile-network radio, acting as a temporary phone tower.
Downlink	Data going from the tower (the drone) to the phones.
NOMA	Non-orthogonal multiple access: two (or more) users share the same frequency slice at the same time; the transmitter mixes their signals with different power levels (superposition coding).
OMA	Orthogonal multiple access: every user gets a separate slice (here frequency slices, FDMA). The traditional way.
FDMA	Frequency-division multiple access: the bandwidth is cut into separate slices, one per user.
SIC	Successive interference cancellation: the strong user first decodes and removes the weak user's message, then decodes its own.
SIC feasibility	SIC only works if the strong user can decode the weak user's message at least as well as the weak user can.
Pairing	Choosing which two users share a slice in NOMA. Ours: strongest with weakest ("extreme") or random.
Power split	How the pair's power is divided: here 80% to the weak user, 20% to the strong user.
SINR	Signal-to-interference-plus-noise ratio: how strong the wanted signal is compared with everything else. Higher means faster data.
Rate, data speed	Bits per second a user gets: bandwidth x log2(1 + SINR) (Shannon).
Minimum rate	The speed a user needs to count as served; depends on their class (rescue, video, voice, iot, normal).
Coverage	The share of users who get at least their minimum rate. The main score in this project.
coverage_full	Coverage averaged over the whole 300 s, counting zero after the battery runs out.
Sum rate, total speed	All users' rates added together.
Spectral efficiency	Sum rate divided by bandwidth (bit/s per Hz).
Fairness (Jain's index)	1 if everyone gets the same speed, lower if speeds are uneven.
Outage	A user not reaching their minimum rate. "Rescue outage" is the share of rescue users in outage.
LoS / NLoS	Line of sight (a clear straight path between drone and phone) / non-line of sight (blocked).
Path loss	How much the signal weakens on the way, in decibels.
FSPL	Free-space path loss: the weakening with distance alone.
Air-to-ground channel (Al-Hourani model)	A formula for the probability of LoS from the elevation angle, plus extra loss for LoS and NLoS links. It creates a best altitude.
Elevation angle	How high the drone appears above the horizon from the user's position.
Fading (Rician / Rayleigh)	Quick random ups and downs of the signal. Rician for LoS links, Rayleigh for NLoS.
dBm, dB	Logarithmic units: 30 dBm = 1 W, 20 dBm = 0.1 W; +3 dB is about double.
Bandwidth	The width of the frequency range used, in Hz. More bandwidth means more speed.
Noise figure	How much noise the receiver adds, in dB.
Rotary-wing propulsion power (Zeng model)	The formula for the power a multicopter needs at a given speed. Hovering is not the cheapest; moderate forward flight costs less.
Hover	Staying in one place in the air.
Backhaul	The drone's own link back to the main network.
Scenario	One invented disaster situation (crowd, buildings, movement), identified by its seed.
Seed	The number that makes the random generator produce the same scenario every time.
Train / validation / test split	Scenarios for learning (seeds 0-99), for choosing between models (100-119), and for the final honest score (120-139, used once).
Profile	A named set of setting changes (e.g. `hard`, `rl_static`) applied on top of `config.yaml`.
Domain randomisation	Drawing different scenario settings for every episode, so the AI cannot memorise one map.
Synthetic data	Data produced by our simulator, not measured in the real world.

Term	Meaning
Gymnasium environment, the game	The standard Python interface (`reset`, `step`) that every pilot plays through.
Episode	One full game of one scenario (300 s or until the battery is empty).
Observation, state	What the pilot sees each step (here 211 numbers: own position, battery, previous scores, two crowd maps).
Action	What the pilot decides each step (velocity or a target spot, altitude, power).
Reward	The score signal the AI learns from (here the coverage, optionally minus an energy cost).
Policy	The pilot's strategy: a rule or a neural network that turns observations into actions.
Baseline	A simple non-learning pilot used for comparison (B0-B6).
Heuristic	A simple rule of thumb, such as "fly to the biggest crowd".
k-means	A method that groups points into k clusters around centres (used by B2).
Grid search	Trying every spot on a grid and keeping the best (B3, B6).
PSO	Particle swarm optimisation: many candidate solutions move towards the best found so far (B4).
Privileged information	Facts the baselines may use but the AI may not, such as exact user positions.
Reinforcement learning (RL)	Learning by trial and error from a score, without being told the right answer.
PPO	Proximal Policy Optimization: the RL method used here (from Stable-Baselines3).
SAC	Soft Actor-Critic: another RL method, the plan's second choice.
Neural network, MLP, CNN	The AI's "brain". MLP: layers of numbers. CNN: reads a grid as a picture (convolutions).
Exploration noise	Random wobble added to the AI's actions during training so it tries new things. Too much hides the effect of its own choices.
Annealing	Shrinking something (here the exploration noise) step by step during training.
Curriculum	Training in stages from easy to hard (still crowd, moving crowd, altitude, power, battery, NOMA).
Waypoint control	The AI says "fly to this spot" instead of "fly this fast".
Decision hold	The AI decides once every 10 s and its choice is repeated in between.
Checkpoint	A saved copy of the AI during training.
Gate (G1-G7)	A check that must pass before the next layer of the project starts.
Hypothesis (H1-H4)	A claim the project tests.
Confidence interval (95% CI)	The range in which the true average very probably lies.
Paired comparison	Comparing two pilots scenario by scenario on the same scenarios, then averaging the differences. Fairer than comparing two averages.
Significant (*)	The confidence interval of the difference does not include zero, so the difference is not luck.
Ablation	Removing or changing one part to see how much it matters.
Sensitivity analysis	Varying an assumption over its range to see whether conclusions change.
Parity test (MATLAB parity)	Checking that our Python numbers match an independent MATLAB implementation.
Sim-to-real gap	The difference between how something performs in simulation and in reality.
Digital twin	A simulation kept in sync with a real system using live data. This project is not one; it is a simulation-based planning tool.
Software in the loop (SITL)	Running real control software against a simulated vehicle.
MAVLink	The standard message protocol for talking to drone autopilots.
SDR	Software-defined radio: a programmable radio board, used to build NOMA test setups.
CI	Continuous integration: GitHub automatically runs lint and tests on every change.
Pull request (PR)	A proposed change that is reviewed and tested before it joins `main`.
Branch protection	GitHub rules that stop unreviewed or untested code from reaching `main`.
Lint (ruff)	Automatic style and bug checks on the code.
Coverage gate (test coverage)	CI fails if the tests exercise less than 80% of the core code. (Not the same "coverage" as user coverage.)

Chapter 13. NOMA in this project

NOMA is the core of the guide's idea ("a moving base station using NOMA"). This chapter collects everything about it: what it is, exactly where it sits in the code, what we measured, where it wins, where it loses and why, and what comes next.

13.1 What NOMA is

Plain words. A tower has a fixed amount of radio bandwidth and power to share between everyone it serves. The traditional way (**OMA**) cuts the bandwidth into separate slices, one per person. **NOMA** puts *two people on the same slice at the same time*: one with a strong signal (usually near the drone) and one with a weak signal (far away or behind a building). The tower sends both messages mixed together and gives most of the power to the weak person. The weak person treats the strong person's part as noise and reads their own. The strong person first decodes the weak person's message, subtracts it, and then reads their own clean message (successive interference cancellation, **SIC**). Each NOMA slice is twice as wide as an OMA slice, so the pair gets more total speed.

Technically. Downlink power-domain NOMA with two-user pairs, superposition coding at the transmitter and SIC at the strong user. The OMA reference is FDMA with the same total bandwidth and the same total power.

13.2 Where NOMA sits in the code

Every coverage number in this project is a NOMA number unless OMA is chosen.

1. `make_dataset.py: pair_users()` sorts users by channel gain and pairs the strongest with the weakest (`extreme`), or pairs at random (`random`); the weak and strong roles are set by gain.
2. `make_dataset.py: noma_rates()` gives each pair bandwidth B / (N/2) and power P / (N/2), splits the pair's power 0.8 to the weak user and 0.2 to the strong user, and computes:

```
SINR_weak   = 0.8 p g_w / (0.2 p g_w + N0)
SINR_strong = 0.2 p g_s / N0                (after SIC)
R = (B / (N/2)) log2(1 + SINR)
```

It checks SIC feasibility (`sic_feasible`) and also returns the OMA rates: R = (B/N) log2(1 + (P/N) g / (N0 density x B/N)).

3. `uav_env.py`: every step calls `noma_rates` for the drone's position. `access="noma"` (the default) scores with the NOMA rates, `access="oma"` with the OMA rates; `info` always reports both (`coverage_noma`, `coverage_oma`, `sum_rate_noma_mbps`, `sum_rate_oma_mbps`).
4. The dataset's `sweep` table stores NOMA and OMA coverage and total speed for every drone position, altitude and power.

Settings in `config.yaml`: `noma_alpha_weak: 0.8`, `pairing: extreme`, `env.access: noma`. **Nobody chooses the NOMA settings yet:** the pairing and the 80/20 split are fixed. Making them decisions is roadmap stage B.

13.3 NOMA in the dataset (all drone positions, from the sweep)

Averaged over every drone position in the sweep, good and bad alike:

Power	NOMA / OMA total speed, v1	NOMA / OMA total speed, v2	Coverage NOMA / OMA, v2
0 dBm	0.894	0.800	0.146 / 0.167
10 dBm	1.097	1.008	0.409 / 0.410
20 dBm	1.170	1.144	0.682 / 0.654
30 dBm	1.152	1.166	0.847 / 0.816

NOMA's advantage in total speed holds at every crowd size (ratio 1.13-1.15 from 100 to 1,000 users). At extremely low power (0 dBm) OMA is better: noise dominates and the weak user is held back by the strong user's interference. Figure: noma_vs_oma_v1.png.

13.4 NOMA measured inside the game (hypothesis H3)

`python noma_vs_oma.py --profile <name>`: every pilot plays the same 100 scenarios twice, once with NOMA and once with OMA, with a 16 Wh battery so every pilot survives the 300 s. The search pilots evaluate candidate spots with the scheme being tested, so each scheme gets its own best decisions. Values: NOMA / OMA / paired NOMA minus OMA (95% CI); * = significant.

At 30 dBm (default scenarios: 1 x 1 km, 0-5 buildings)

NOMA vs OMA (H3). Profile default, 100 scenarios, 30 dBm, battery 16 Wh. Values: NOMA / OMA / paired NOMA - OMA (95% CI); * = the CI excludes 0.

Pilot	coverage: NOMA / OMA / NOMA-OMA	total speed (Mbps): NOMA / OMA / NOMA-OMA	fairness: NOMA / OMA / NOMA-OMA	rescue outage: NOMA / OMA / NOMA-OMA
B0	0.882 / 0.870 / +0.012* (+/-0.008)	275.1 / 228.8 / +46.3* (+/-6.1)	0.731 / 0.922 / -0.192* (+/-0.012)	0.069 / 0.069 / +0.000 (+/-0.000)
B1	0.884 / 0.900 / -0.017* (+/-0.007)	328.7 / 262.8 / +66.0* (+/-9.2)	0.689 / 0.888 / -0.199* (+/-0.011)	0.098 / 0.098 / +0.000 (+/-0.000)
B2	0.887 / 0.905 / -0.018* (+/-0.007)	328.2 / 263.8 / +64.3* (+/-8.8)	0.694 / 0.896 / -0.201* (+/-0.011)	0.088 / 0.088 / +0.000 (+/-0.000)
B3	0.909 / 0.929 / -0.020* (+/-0.006)	297.7 / 270.9 / +26.8* (+/-6.6)	0.726 / 0.925 / -0.199* (+/-0.011)	0.037 / 0.055 / -0.018* (+/-0.009)

At 20 dBm (profile `lowpower`)

NOMA vs OMA (H3). Profile lowpower, 100 scenarios, 20 dBm, battery 16 Wh. Values: NOMA / OMA / paired NOMA - OMA (95% CI); * = the CI excludes 0.

Pilot	coverage: NOMA / OMA / NOMA-OMA	total speed (Mbps): NOMA / OMA / NOMA-OMA	fairness: NOMA / OMA / NOMA-OMA	rescue outage: NOMA / OMA / NOMA-OMA
B0	0.797 / 0.745 / +0.052* (+/-0.006)	171.8 / 141.2 / +30.6* (+/-4.1)	0.753 / 0.849 / -0.096* (+/-0.011)	0.323 / 0.323 / +0.000 (+/-0.000)
B1	0.830 / 0.764 / +0.065* (+/-0.011)	222.0 / 175.4 / +46.6* (+/-5.6)	0.706 / 0.815 / -0.109* (+/-0.011)	0.271 / 0.271 / +0.000 (+/-0.000)
B2	0.835 / 0.769 / +0.066* (+/-0.011)	221.7 / 176.0 / +45.7* (+/-5.4)	0.714 / 0.824 / -0.110* (+/-0.011)	0.262 / 0.262 / +0.000 (+/-0.000)
B3	0.865 / 0.808 / +0.058* (+/-0.009)	218.4 / 172.1 / +46.4* (+/-6.9)	0.751 / 0.884 / -0.134* (+/-0.008)	0.190 / 0.203 / -0.013* (+/-0.012)

Hard scenarios (profile `hard`: 2 x 2 km, 20 dBm, 3-8 buildings, sudden or drifting crowds)

NOMA vs OMA (H3). Profile hard, 100 scenarios, 20 dBm, battery 16 Wh. Values: NOMA / OMA / paired NOMA - OMA (95% CI); * = the CI excludes 0.

Pilot	coverage: NOMA / OMA / NOMA-OMA	total speed (Mbps): NOMA / OMA / NOMA-OMA	fairness: NOMA / OMA / NOMA-OMA	rescue outage: NOMA / OMA / NOMA-OMA
B0	0.428 / 0.420 / +0.008 (+/-0.009)	57.0 / 49.7 / +7.3* (+/-3.0)	0.504 / 0.566 / -0.062* (+/-0.010)	0.846 / 0.846 / +0.000 (+/-0.000)
B2	0.633 / 0.568 / +0.065* (+/-0.008)	159.5 / 119.2 / +40.3* (+/-4.7)	0.562 / 0.609 / -0.048* (+/-0.006)	0.524 / 0.524 / +0.000 (+/-0.000)
B3	0.675 / 0.616 / +0.059* (+/-0.005)	155.8 / 104.9 / +50.8* (+/-7.4)	0.592 / 0.670 / -0.078* (+/-0.009)	0.506 / 0.537 / -0.031* (+/-0.016)

Charts: 30 dBm, 20 dBm, hard.

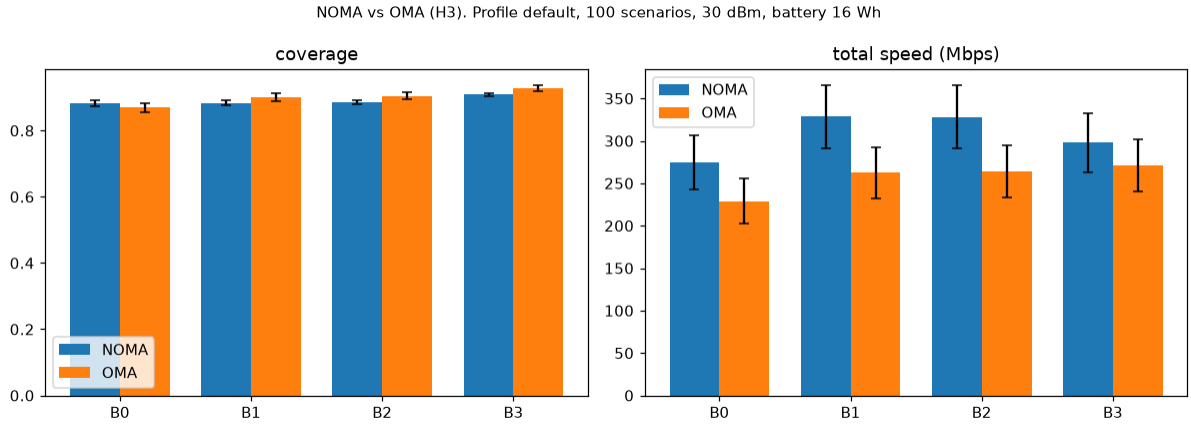


Figure: noma_vs_oma_in_game_default_30dBm.png

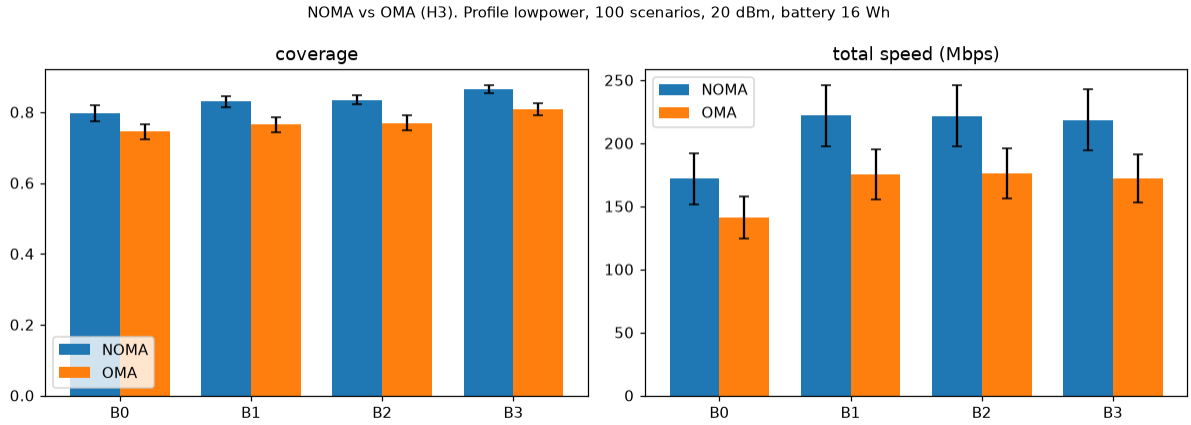


Figure: noma_vs_oma_in_game_lowpower_20dBm.png

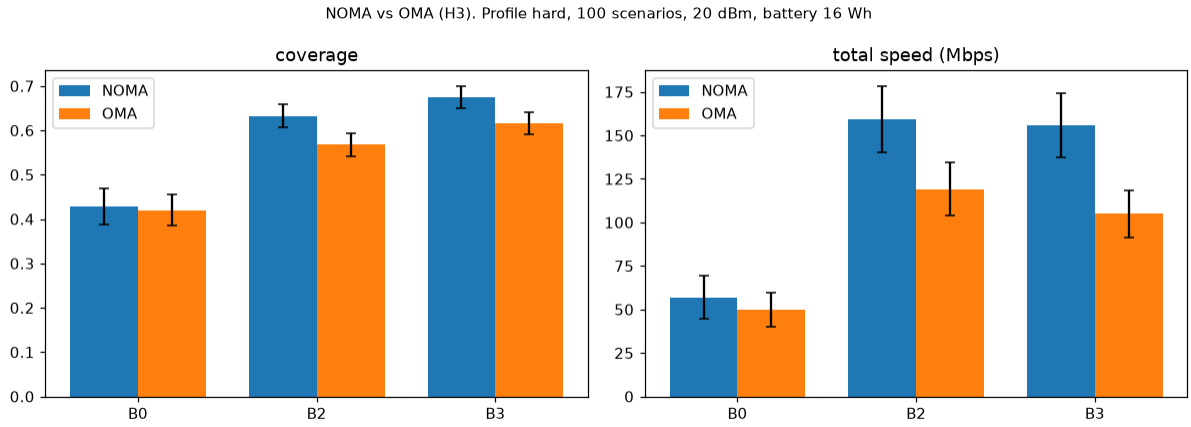


Figure: noma_vs_oma_in_game_hard_20dBm.png

13.5 What the measurements say

1. **NOMA always gives more total speed:** about 20-27% more for every pilot at 30 and 20 dBm, all significant.
2. **NOMA serves more people when power is scarce:** at 20 dBm it covers 5-7 more people per 100 for every pilot, all significant.
3. **At full power (30 dBm) OMA serves slightly more people** with pilots that place the drone well (B1-B3: 1.7-2.0 fewer people per 100 with NOMA). Only the parked drone B0 still gains coverage with NOMA.
4. **NOMA is less fair** (Jain's index 0.10-0.20 lower): strong users get much more than weak users.

5. Rescue outage is the same or slightly better with NOMA (B3: 3.1 fewer rescue users unserved per 100 in `hard`).
6. **The harder the scenario, the more NOMA helps:** in `hard` the moving pilots serve 6-7 more people per 100 with NOMA (B2 +0.065, B3 +0.059, significant) and get up to 48% more total speed (B3: 155.8 vs 104.9 Mbps). For the parked drone B0 the coverage gain (+0.008) is not significant: NOMA helps most when the drone is also placed well.

13.6 Why NOMA loses coverage at full power: the 232 kbps ceiling

With a fixed split, the weak user always receives the strong user's signal as interference:

```
SINR_weak = 0.8 p g_w / (0.2 p g_w + N0)  <  0.8 / 0.2 = 4      however large p g_w becomes
R_weak    <  (2 x 50 kHz) x log2(1 + 4)  =  232 kbps
```

Checked numerically on scenario 1: the fastest weak user gets exactly 232 kbps at 30, 40 and even 60 dBm, while the same users under OMA get 507, 673 and 1,005 kbps. **Video users need 500 kbps, so a video user placed in the weak half of a pair can never be served with NOMA**, however close the drone flies. With more power OMA catches up and overtakes on coverage, while NOMA keeps its lead in total speed. The NOMA picture shows the ceiling: the weak half of the users sits on a flat line just above 200 kbps (7_noma_pairs_lowpower.png, right panel).

This is the classic limitation of fixed power allocation in NOMA. It is a property of the model, not a bug, and it points to the fix.

13.7 Pictures

- 6_noma_pairs_default.png: scenario 1 (300 users, 30 dBm). Coverage NOMA 0.927 vs OMA 0.923; total speed 182.5 vs 146.0 Mbps.

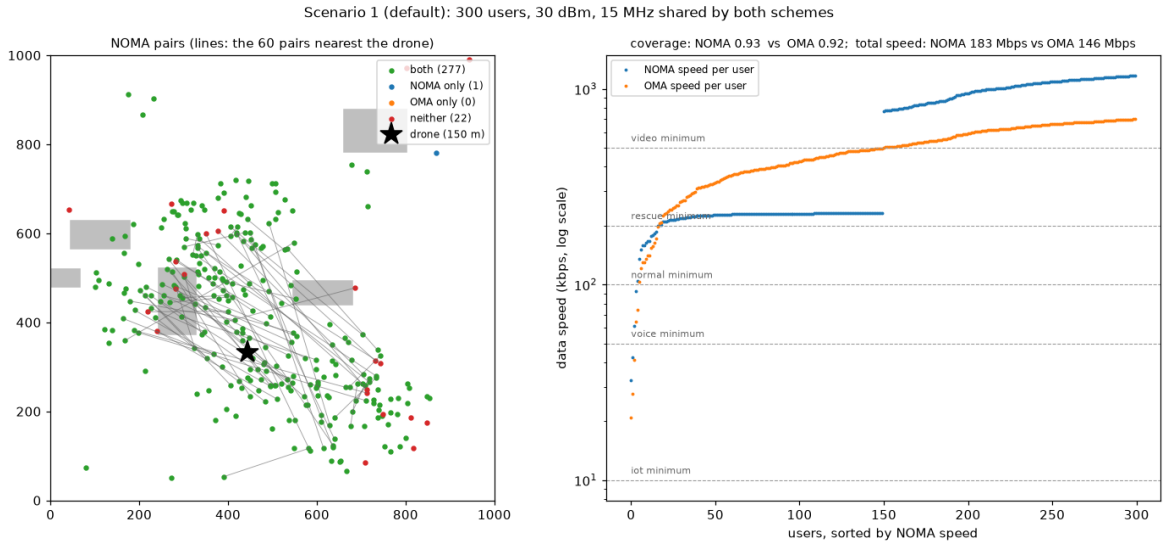


Figure: 6_noma_pairs_default.png

- 7_noma_pairs_lowpower.png: scenario 7 (1,000 users, 20 dBm). Coverage NOMA 0.81 vs OMA 0.76; total speed 337 vs 263 Mbps; 58 users served only thanks to NOMA, none only by OMA. Lines join the 60 pairs nearest the drone.
- Make your own: `python noma_picture.py --seed <n> [--profile <name>] [--x X --y Y --h H].`

13.8 Next steps for NOMA (roadmap stage B)

(Update 2026-10-21: items 1 and 4 (imperfect SIC) are done, see 13.9.)

1. **Choose the power split per pair** instead of a fixed 80/20: for example a closed-form split that gives each weak user just its minimum rate and the rest to the strong user, and later the AI choosing it. Expected to remove the 232 kbps ceiling and the coverage loss at full power.
2. **Better pairing:** k-means on channel gain, and need-aware pairing (avoid putting a high-rate user in the weak role).
3. **The AI decides the NOMA settings** together with position, altitude and power (curriculum step 6): the joint problem studied in the literature.
4. **Imperfect SIC** (a residual factor) and **bit-error rate** as ablations. Only then do real SIC failures appear.
5. Report H3 with all of the above, with 5 seeds for any AI result.

13.9 Adaptive NOMA vs equally optimised OMA (Track 1, 2026-10-21)

Plain words. Until now every pair of users got the same power and the same fixed 80/20 split, whatever they needed. Now the drone works out, for every person, the *least* power that still gives them the speed they need, serves the cheapest people first until the power runs out, and only then pairs them up for NOMA. To be fair, OMA gets exactly the same treatment. The question becomes: when both are run as well as possible, does NOMA still serve more people?

How it works (`access_rates()` in `make_dataset.py`, D-39). Two new access schemes; the old `noma` and `oma` are unchanged (byte-identical code, and the old numbers reproduce exactly).

- `oma_opt`: every user has its own slice b = B/N. Minimum power for user i: p_i = (2^(R_i/b) - 1) x noise x b / g_i. Users are admitted cheapest-first while the total stays within the drone's power P (this is provably the best way to maximise the number served). Leftover power is shared by scaling every admitted user's power by the same factor.
- `noma_opt`: the same cheapest-first order, but the admitted users are paired strongest-with-weakest on a double slice 2b, and each pair gets its **minimum NOMA power** in closed form, with an imperfect-SIC residual eps (the share of the removed signal that stays behind):

```
p_s = gamma_s N (eps gamma_w / g_w + 1 / g_s) / (1 - eps gamma_s gamma_w)      (strong user)
p_w = gamma_w (p_s + N / g_w)                                                 (weak user)
gamma = 2^(R / 2b) - 1,  N = noise power on 2b
```

If NOMA would cost more than serving the two users separately (possible only when eps > 0), the pair uses OMA inside its slot. The largest number of users that fits is found by binary search. Unserved users' slots are wasted in **both** schemes.

- Options: `sic_eps` (0 or 0.05, engineering assumption) and `rescue_first` (admit rescue users before everyone else).
- Checks: 45 new tests, including brute force on small crowds (`oma_opt` exact; `noma_opt` within one user), every rate recomputed from the returned powers, SIC decoding always possible, more power never serves fewer people. I also re-derived the closed form independently: rates come out exact to 1e-15 over 2,000 random pairs.

Results in the game (100 scenarios, 16 Wh, every pilot survives; * = significant after Holm correction over all 60 comparisons of the table):

Profile, pilot	OMA	NOMA (fixed)	OMA optimised	NOMA adaptive	adaptive NOMA minus optimised OMA
lowpower (20 dBm), B0	0.745	0.797	0.893	**0.944**	+0.051*
lowpower, B2	0.769	0.835	0.924	**0.962**	+0.039*
lowpower, B3	0.808	0.865	0.945	**0.983**	+0.038*
hard (2 x 2 km, 20 dBm), B0	0.420	0.428	0.658	**0.690**	+0.032*

Profile, pilot	OMA	NOMA (fixed)	OMA optimised	NOMA adaptive	adaptive NOMA minus optimised OMA
hard, B2	0.568	0.633	0.777	**0.811**	+0.034*
hard, B3	0.616	0.675	0.808	**0.849**	+0.041*

Same runs, other measures (B3 shown; B0 and B2 behave the same way):

B3	total speed (Mbps)	fairness (Jain)	rescue users not served
lowpower: NOMA fixed / OMA opt / **NOMA adaptive**	218.4 / 71.3 / **93.1**	0.751 / 0.409 / **0.472**	0.190 / 0.030 / **0.004**
hard: NOMA fixed / OMA opt / **NOMA adaptive**	155.8 / 45.9 / **54.0**	0.592 / 0.340 / **0.356**	0.506 / 0.287 / **0.203**

Against optimised OMA, adaptive NOMA is better on **every** measure in every case (coverage +0.03 to +0.05, total speed +5 to +23 Mbps, fairness and rescue outage better, all significant). One snapshot at 30 dBm (default scenarios, 20 validation maps): adaptive NOMA 1.000 vs optimised OMA 0.989, while fixed NOMA still trails OMA (0.913 vs 0.933). **The 232 kbps ceiling of 13.6 is gone.**

Charts: 8_access_schemes_hard.png, 9_access_schemes_lowpower.png.

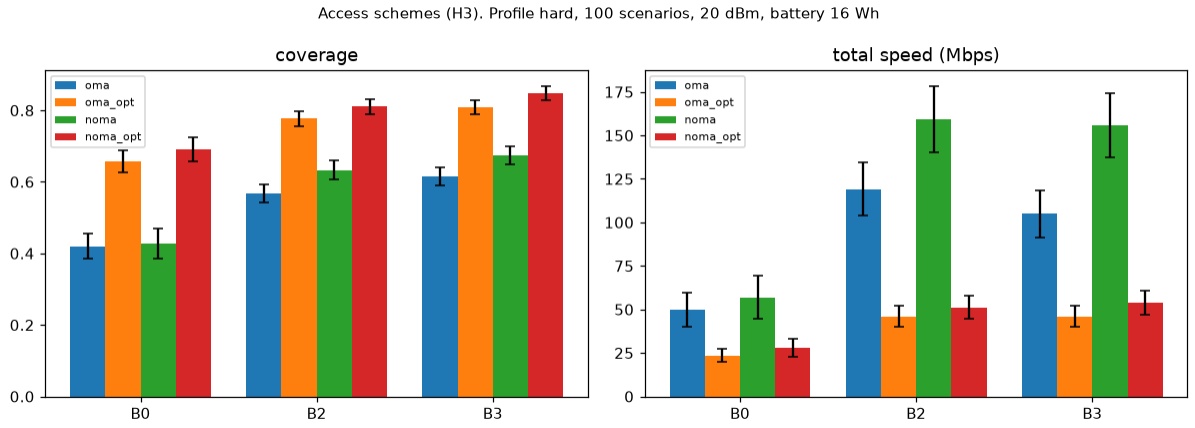


Figure: 8_access_schemes_hard.png

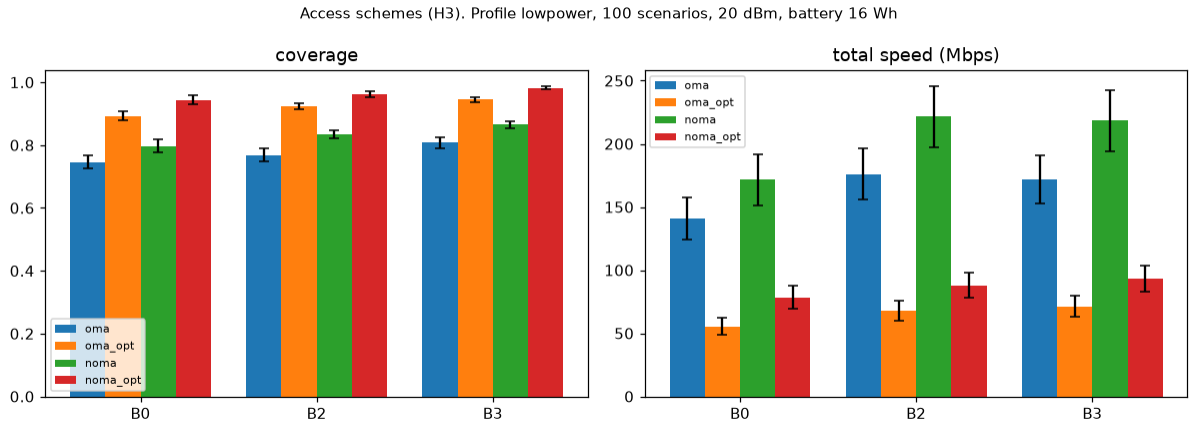


Figure: 9_access_schemes_lowpower.png

Imperfect SIC (eps = 0.05, hard). Adaptive NOMA loses 0.014-0.024 coverage and 16-36% of pairs switch to OMA inside the pair, but it still beats optimised OMA by +0.016 to +0.018*. **Rescue first (hard).** With adaptive NOMA, rescue outage drops sharply (B3 0.203 to 0.117, B2 0.258 to 0.169) for a small coverage loss (0.011-0.029); with optimised OMA the same rule costs 0.11-0.20 coverage, because a far rescue user is very expensive alone, while in NOMA the pairing absorbs that cost.

The honest trade-off. The optimised schemes give everyone just their minimum speed and spend the rest of the power reaching far, expensive users. So compared with the old fixed

schemes they **serve far more people (+0.12 to +0.26) but deliver about 55-65% less total speed and lower fairness**. This is a property of the goal "serve as many people as possible", not of NOMA. In a disaster that goal is the right one (a survivor with a working connection matters more than a fast connection for someone already served), but the report must say "more people served, less total speed", not "better on every metric".

What is not verified. The binary search can miss a few users at single candidate spots (up to 4% of N measured, mean about 0.1%, and 0 at every best spot); it can only under-count adaptive NOMA, so its advantage is if anything understated. "Within one user of the optimum" is shown on small crowds, not proven. About 10-17% of NOMA pairs give the weak user less than half the power (fine in theory; practical receivers sometimes assume otherwise; no clamp tested). sic_eps and the minimum rates are engineering assumptions. The game runs use seeds 0-99 like the earlier H3 runs.

Reproduce: `python noma_vs_oma.py --profile hard --schemes oma,oma_opt,noma,noma_opt` (also `--profile lowpower`, `--eps 0.05`, `--rescue_first`), and `python access_benchmark.py`.

13.10 How to explain it to an examiner

"NOMA lets two users share one frequency slice by splitting power; the strong user removes the weak user's signal with SIC. In our simulator NOMA always increases total throughput by about 20-27%, and it serves more users when transmit power is limited. With a fixed 80/20 split the weak user's SINR is capped at 4, so its rate cannot exceed 232 kbps, which is why OMA can cover slightly more users at full power. That is why the next step is adaptive power allocation per pair."

Since Track 1: "When both schemes allocate power optimally to serve as many users as possible, NOMA still serves 3-5 more users per 100 than OMA, with more throughput and fewer unserved rescue users; imperfect SIC halves but does not remove that gain. The price of the coverage objective, for both schemes, is lower total throughput."

Chapter 14. How a senior communications and AI engineer would solve this

Status: **draft proposal (14.1-14.4), independently reviewed (14.5), agreed plan (14.6)**, written 2026-10-21. Nothing in this chapter has been built yet. The owner's decisions in 14.7 are pending.

How to read it: 14.1-14.4 are my first draft, kept as written so the review makes sense. Where the review corrected the draft, the correction is in 14.5 and the plan to follow is 14.6.

14.1 Reading the evidence first

1. **The still-crowd task is close to solved by search.** The best single spot reached instantly gives 0.686; grid search (B3) already gets 0.659. The maximum any controller can add over B3 is about 0.03 (less in practice: see the review, 14.5). Gate G4 tests *learnability*, not a chance to win.
2. **The AI plateaus at 0.51-0.56 whatever we change** (network, inputs, map resolution, training length, exploration schedule). The probe showed why: it maps the heatmap to a target point imprecisely (100-550 m off) and commits to it. We are asking a policy-gradient method to learn an *argmax over a 2D coverage field* from a coarse 10 x 10 density picture. That is an optimisation problem dressed up as a sequential decision problem; RL is the wrong tool for that layer.
3. **NOMA, as configured, has a hard ceiling.** With a fixed 80/20 split the weak user's SINR is capped at 4, so its rate is capped at 232 kbps whatever the power. That is why OMA covers slightly more users at 30 dBm.
4. **Where RL could genuinely win is untested and unmeasured.** H2 says RL should win only under dynamics and battery limits. We have never measured how much a perfect *anticipating* controller could gain over the best myopic one (B6) in moving scenarios. If that headroom is small, no RL can win there either.

14.2 The principle: separate the problem by time scale

A wireless-systems engineer splits the control problem by how fast each decision must change and how much is known:

Layer	Time scale	Decision	Right tool
Radio resource allocation	every slot (1 s)	NOMA pairing, power split, NOMA or OMA per pair	closed-form / optimisation (known channels)
Placement	every 10 s	the best 3D spot for the users as they are now	search / optimisation (B3, B6, PSO)
Mission strategy	tens of seconds to minutes	where the crowd will be, when to move, how to spend the battery, which group to prioritise	**learning (RL)**, because it needs anticipation and long-term trade-offs

"Do not learn what you can compute." Learning is spent where computation cannot reach: the future and the long-term budget.

14.3 The plan, in priority order

P1. Fix the NOMA layer with adaptive hybrid NOMA/OMA (days, no learning)

For each pair (strong user s, weak user w) with the pair's resources b = 2B/N and p = 2P/N:

- **QoS-aware power split:** give the weak user just enough power to reach its minimum rate R_w: gamma_w = 2^(R_w / b) - 1, alpha* = gamma_w (p g_w + N0) / ((1 + gamma_w) p g_w),

valid if alpha* < 1 (N0 here is the noise *power* over the pair's bandwidth b, not the density; corrected in 14.5). The strong user gets the rest: rate b log2(1 + (1 - alpha*) p g_s / N0). SIC stays feasible because g_s >= g_w. This is the "cognitive-radio-inspired" power allocation discussed in the NOMA literature (to verify the citation).

- **Per-pair mode choice:** compute the users served (then the total rate) with NOMA at alpha* and with OMA (each user b/2, p/2, exactly our current OMA), and use the better one. Because OMA is one of the options for every pair, **coverage can never be worse than OMA**, and NOMA is used wherever it helps.
- **Need-aware pairing:** sort by gain, but avoid pairing two high-rate users; compare extreme pairing with need-aware pairing.
- Then re-run H3 with three schemes: OMA, NOMA fixed 80/20, hybrid adaptive NOMA. Expected: the 232 kbps ceiling disappears and NOMA's coverage gain at 30 dBm turns positive. This strengthens the NOMA story the guide cares about, without any AI.

P2. Measure the headroom for RL before training more (days)

Build an **anticipating oracle**: it knows the crowd's future positions (we simulate them) and plans the drone's waypoints with model-predictive control or dynamic programming over a 30-60 s horizon, with the drone's speed limit and battery. Compare oracle vs B6 (best myopic) on the dynamic profiles (surge, hotspot jump, attraction) and with binding batteries.

- If oracle - B6 is large (say over 0.05), there is a real prize for an AI that learns to anticipate: H2 is worth testing.
- If it is small, H2 should be reported as "no room for RL in these regimes", or the scenarios redesigned (e.g. sudden events, multiple crowds competing for one drone, tight batteries) until there is headroom. This is what a senior would do before spending weeks on training.

P3. Re-architect the AI (owner's decision on the rule D-24)

- **Option A, hierarchical (recommended for performance):** the AI decides the *mission-level* choices every 30 s (which area or crowd to serve next, altitude band, how much battery to spend now), and the computed layers (P1 radio allocation, a local placement search) execute them. The AI still learns by itself; it is never told the answer to its own decision. But it relies on computed sub-layers, which is a different reading of "the AI learns by itself" and must be the owner's call.
- **Option B, end-to-end (respecting the rule literally):** keep the AI flying directly, but give it realistic measurements a base station really has (per-user signal-quality reports, CQI) and train with an **asymmetric critic** (the critic sees true positions during training only; the flying policy never does), SAC or a recurrent network, and many more samples. Accept that it may stay below the search pilots on still crowds.

P4. Exploit the energy physics (days)

Forward flight at about 10 m/s costs less than hovering. Add **B7, a loitering pilot** that circles the best spot at the minimum-power speed. In battery-limited scenarios this may serve more user-seconds per Wh than hovering pilots. It is a good baseline and a good question for RL: can it discover loitering by itself?

P5. Realism of the radio layer (later)

Time-domain scheduling (serve users over time with proportional-fair scheduling instead of all users every second), an antenna pattern, imperfect SIC, and the "drone does not know where people are" observation. These matter for credibility and for the real-world gap (Chapter 11).

P6. Then dynamic RL (Gate G5) with 5 seeds, on the regimes P2 found to have headroom.

14.4 What this changes in the project story

"NOMA with adaptive power allocation never does worse than OMA and does better where it matters; search solves placement; the AI's job is anticipation and battery strategy, and we measured how much room that job has before training it." That is a defensible, senior-level contribution, honest about where learning helps.

14.5 The independent review

A second agent, briefed as a senior wireless and RL engineer, read the code, the results and this draft. It was asked to find errors and better ideas, not to agree. I checked each of its claims against the code before accepting it.

Confirmed as correct.

- The QoS-aware split alpha* is right, **with N0 meaning the noise power over the pair's bandwidth** (the code computes noise as density x bandwidth). The case alpha* >= 1 (the weak user needs all the power, the strong user gets nothing) must be handled explicitly: then the pair falls back to OMA or the weak user is not served.
- A split with alpha < 0.5 is valid in theory (the decoding order comes from the channel gains, not from the power), but practical receivers usually assume alpha >= 0.5. Report how often alpha* < 0.5 happens and test a clamp as a sensitivity check.
- SIC is always feasible when roles follow gain, so `sic_feasible` **can never fail in the current code**. A real failure needs imperfect SIC or errors in the gain estimates. Already noted in Chapter 4; now listed as a known limitation.

Errors and gaps the review found in the draft.

1. **"Hybrid NOMA never does worse than OMA" is a tautology**, and the comparison is unfair if OMA stays unoptimised. A fair H3 optimises OMA as well (a QoS-aware bandwidth and power split inside each pair, and a "serve only the strong user" option). The finding to report is the *size* of the gap between optimised NOMA and optimised OMA. Also: the per-pair choice must check the strong user's own minimum rate, and the number of users must be even (an unpaired user gets nothing under NOMA).
2. **The 0.686 ceiling ignores flight time.** It assumes the drone is instantly at the best spot, but flying there from the centre takes up to about 94 s at 15 m/s. The real headroom over B3 on still crowds is below 0.03.
3. **The AI's information was never checked.** Verified in the code: the observation (`OBS_GROUPS` in `uav_env.py`) has **no obstacle map, no demand information** (only the rescue map) and no motion information (one heatmap per frame). The search pilots B3-B6 evaluate spots with the true positions *and* the obstacles. So the plateau may be an **information ceiling**, not an RL failure. This is the most important point of the review.
4. **A bigger NOMA lever than the split:** every pair gets the same power p = 2P/N. Greedy admission (give each pair the minimum power it needs, admit the cheapest pairs first until the power runs out) serves more users, for NOMA and OMA alike.
5. `gamma: 0.995` in `config.yaml` is commented "300 steps", but a game is now 30 decisions (verified). The still-crowd task is close to a one-shot choice.
6. Training uses only 100 maps. Draw training scenarios from fresh seeds (1,000+), kept disjoint from validation and test.
7. Credibility: a regime grid (battery x mobility x power) with paired CIs, a correction for many comparisons (Holm), the test set read once, and H3 with four schemes (OMA equal split, OMA

optimised, NOMA fixed, NOMA hybrid).

8. P5 (time scheduling, antenna patterns) should mostly be cut: it changes the definition of coverage and would invalidate every earlier result. Only imperfect SIC is worth its half day.

Where I disagree with the reviewer.

- It recommends a **spatial action head** (the network outputs a probability over map cells plus an offset inside the cell). That is technically sound and common, but in effect it is the **"choose one of 100 cells" design the owner rejected** (D-24). It is not adopted without the owner's explicit approval.
- It says that with D-24 as written G4 "cannot be won by design", because the baselines use privileged information. Plausible, but not shown yet. The information-matched baseline (step 1 below) will tell.

14.6 The agreed plan (draft and review combined), in order

#	Step	Why	Effort
1	**Information-matched baseline "B3-obs"**: grid search that uses only what the AI sees (density map cells, no obstacles, no classes); also compare the AI's training and validation scores	decides whether the plateau comes from missing information or from RL itself	1 day. **Done 10-21** (Chapter 8.7): half information, half learning
2	**Hybrid adaptive NOMA**: QoS-aware split, per-pair NOMA/OMA choice checking both users' minimum rates, an *equally optimised* OMA, greedy power admission across pairs; added as new `access` options so old results stay reproducible; tests; re-run H3 with four schemes and the baseline tables	removes the 232 kbps ceiling and makes H3 fair and strong	3-4 days
3	**Clairvoyant oracle on moving crowds** at a battery that binds: a receding-horizon B6 that knows the next 30-60 s of positions; later report how much of the oracle's lead over B6 the AI closes	measures whether H2 has anything to win before more training	2-3 days
4	**Give the AI the raw measurements it lacks (within D-24):** an obstacle map, a demand map, the previous frame; fresh training seeds; fix gamma	the cheapest fixes that stay inside the owner's rule	3-4 days
5	**B7 loitering pilot** (circles at the cheapest speed) with an energy-parameter sensitivity sweep; metric: served user-seconds at a binding battery	tests a real physics insight; cheap	1-2 days
6	Owner's decisions (14.7), then **dynamic RL (Gate G5)** on the regimes with headroom, 5 seeds, regime grid	the actual research question	1.5-2 weeks
7	Imperfect SIC as an ablation	makes the SIC check meaningful	0.5 day

About 4-5 weeks in total, realistic for a final-year project if time scheduling, antenna patterns and multi-UAV stay out. Steps 1 and 2 need no decision from the owner beyond approving the plan; step 4's new observations are raw measurements and fit D-24; steps 3 and 5 add pilots, not hints.

14.7 Decisions needed from the owner

1. **Asymmetric critic:** may the critic (the part of PPO that only judges during training and is thrown away afterwards) see true user positions during training? The flying policy never would. Both reviewers think this keeps the spirit of D-24 if disclosed.
2. **Spatial action head:** allowed or not (it resembles the rejected 100-cell menu)?
3. **Hierarchical design (Option A)** for the dynamic stage, where the AI chooses mission-level goals and computed layers carry them out: allowed or not?
4. **Relax G4** and move on to moving crowds after steps 1 and 4, whatever the still-crowd score?

14.8 Progress

- **Step 1 done (2026-10-21), Chapter 8.7.** About half of the AI's gap to B3 is missing information and half is the AI not using what it sees (it underfits even on its training scenarios).

A search with only the AI's maps reaches 0.605, level with B2, so G4 is reachable in principle. Consequences for step 4: remove the map cap and give the AI a finer map or a count of people; an obstacle map is the least promising input. The learning half points at the action design (the AI aims at the densest cell instead of a spot between crowds), which is exactly where owner decisions 1 and 2 in 14.7 matter.

- **Owner's decisions (2026-10-21), D-38:** asymmetric critic yes; cell action head no; hierarchical later; move to moving crowds after tracks 1 and 2.
- **Second review before tracks 1 and 2 (2026-10-21).** Before writing code I sent the concrete designs for adaptive NOMA (step 2) and the better learner (step 4) to a new reviewer agent, then questioned its answers. It ran the real code on 5 scenarios and found that my NOMA design was flawed (per-pair choice decided in advance, wrong admission order, a split formula that assumed equal power per pair) and that a privileged critic does little in PPO but a lot in SAC. One snapshot of its measurements (best spot per scheme, 5 validation scenarios, 20 dBm): today's NOMA 0.56-0.80, today's OMA 0.46-0.74, optimised OMA 0.68-0.91, adaptive NOMA 0.75-0.96 (0.71-0.90 with imperfect SIC). These are a reviewer's quick check, not results; the real runs are in Track 1. The agreed designs are D-39, D-40 and D-41; both tracks now run in parallel.
- **Step 5 done (Chapter 7.10):** circling instead of hovering serves 26% more people per battery.
- **Step 3 done (Chapter 7.11, D-51): there is no room for anticipation in today's moving-crowd scenarios** (a pilot that knows the future gains +0.002 over B6). Options for the redesign before G5, to be chosen by the owner:
  1. **Tight battery** (12-13 Wh): the decisions become *when to move, when to circle, when to save energy*, which have long-term consequences (B7 already gains +0.08 to +0.12 there). The clairvoyant test then needs the battery in its plan.
  2. **Competing crowds:** crowds that drift apart so one drone cannot serve them all and must choose and time its moves.
  3. **Sudden, fast events** (larger, faster surges; a crowd appearing far away) or a slower drone.
  4. **Narrower footprints** (lower power or more buildings), so position matters more.

Appendix. Figure gallery

Figures and animations from docs/book/figures/ that are not shown inside a chapter.

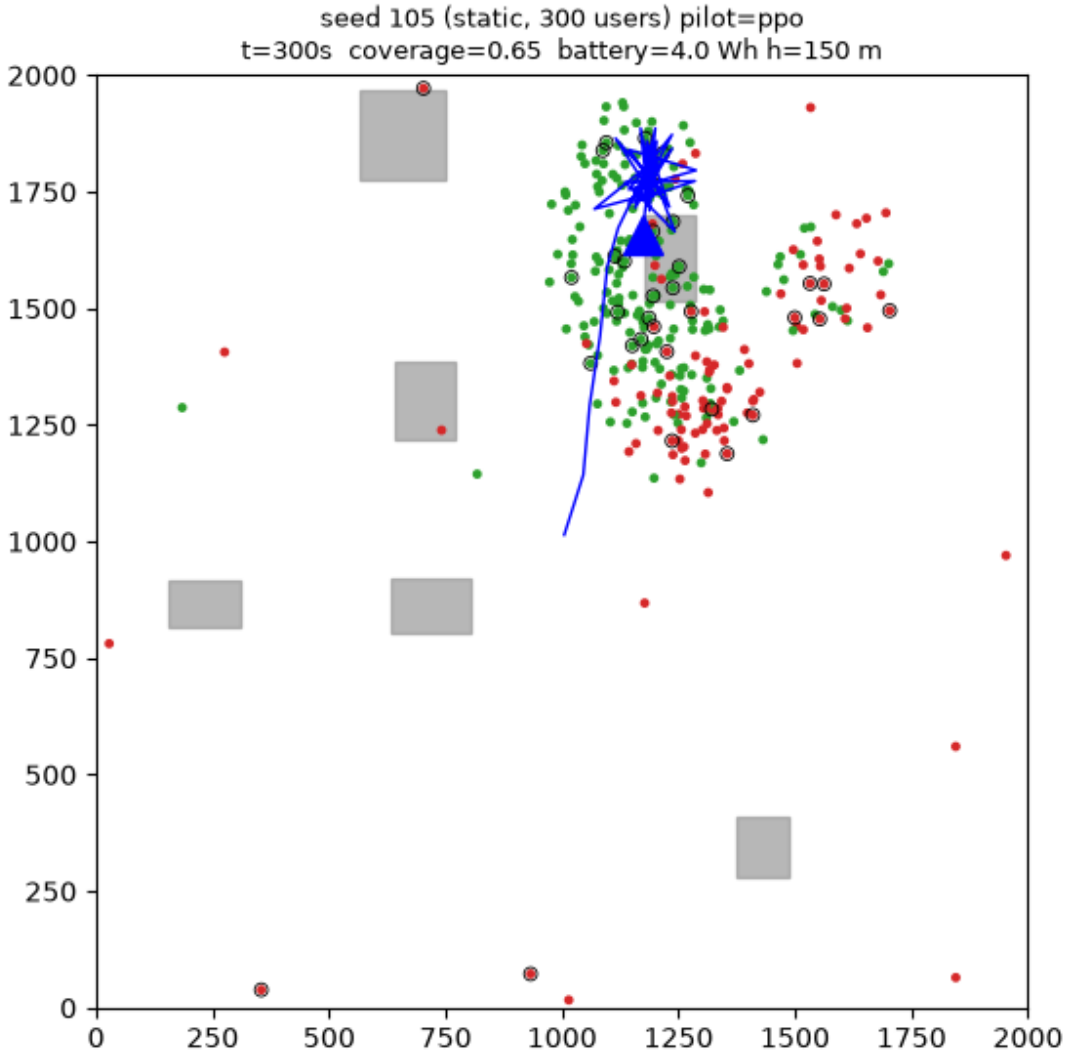


Figure: best_ai_e5a_seed105_last.png

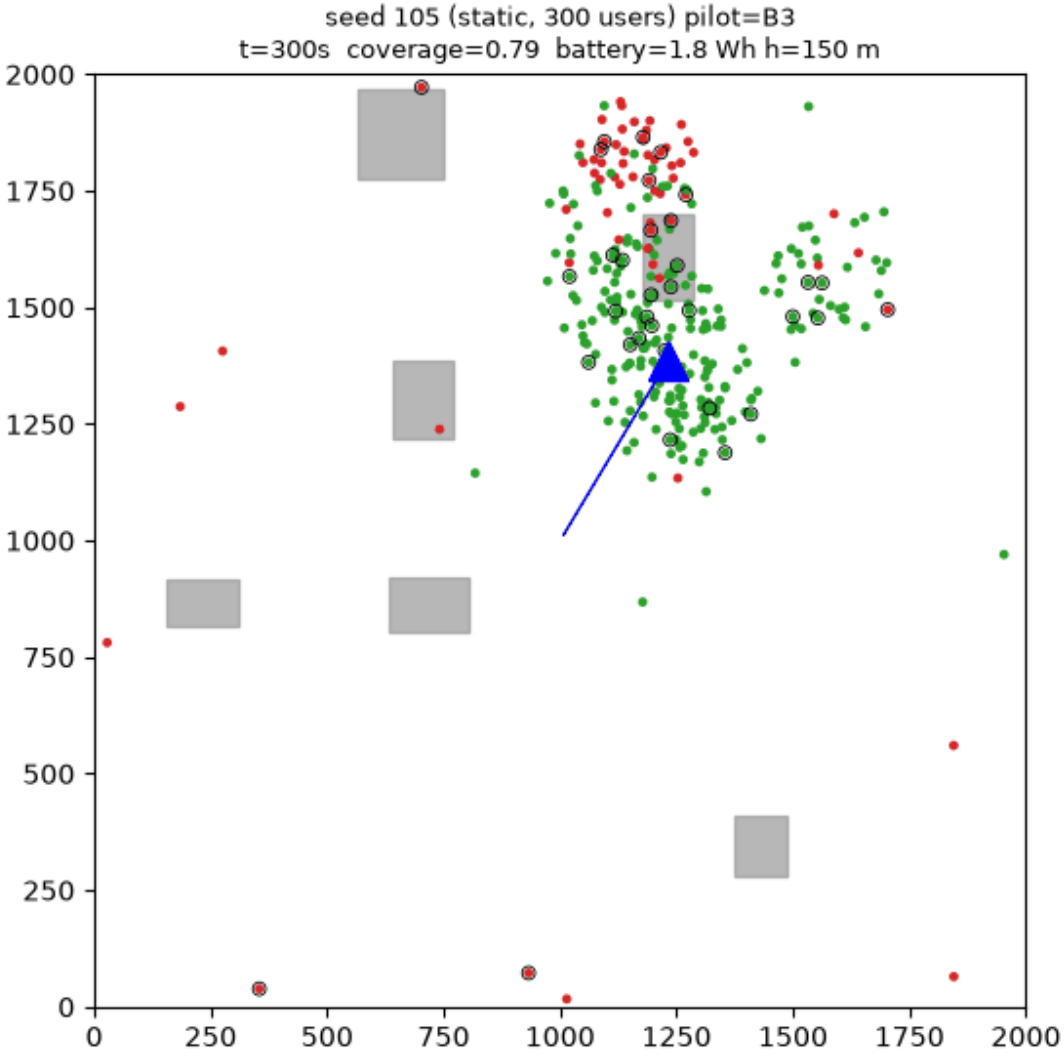


Figure: B3_still_seed105.gif (last frame of the animation; the GIF is in docs/book/figures/gifs/)

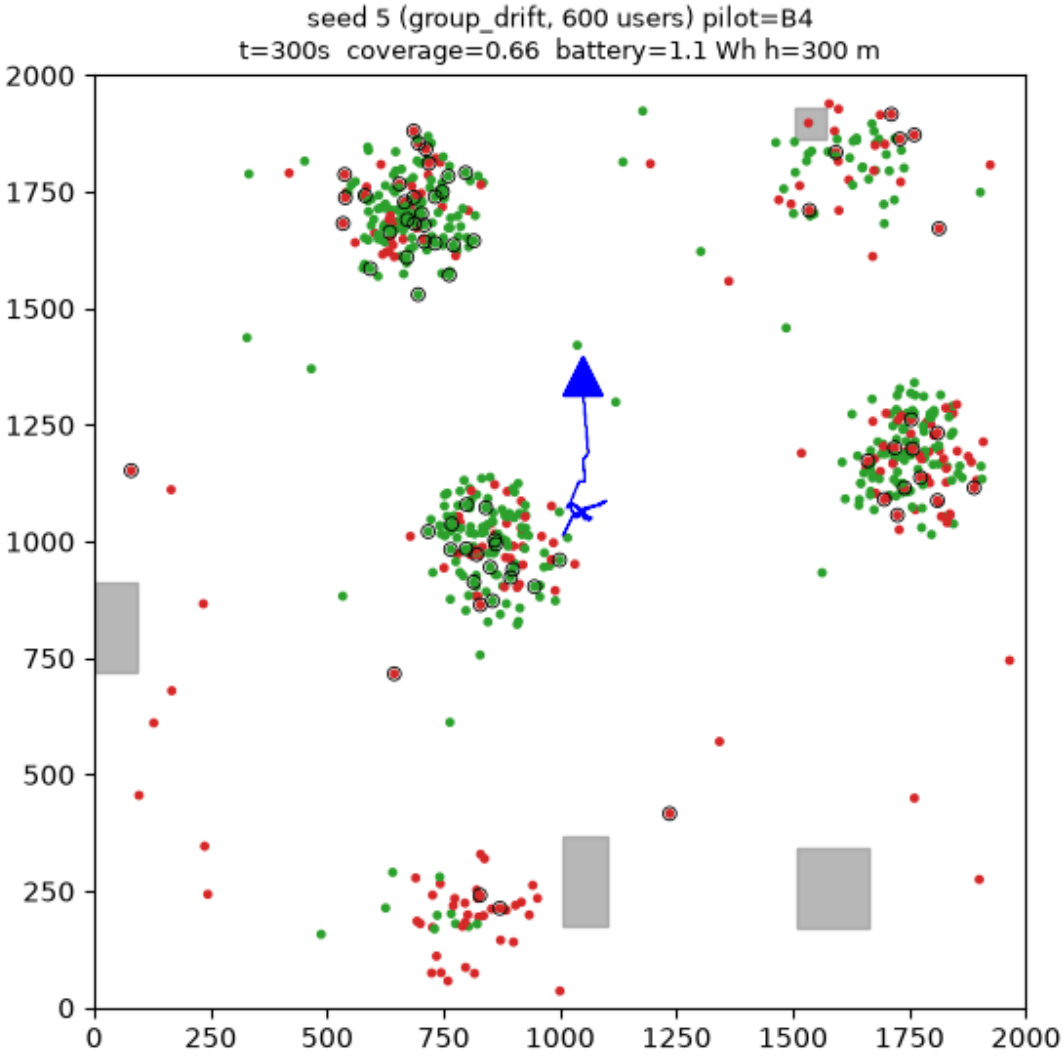


Figure: B4_hard_seed5.gif (last frame of the animation; the GIF is in docs/book/figures/gifs/)

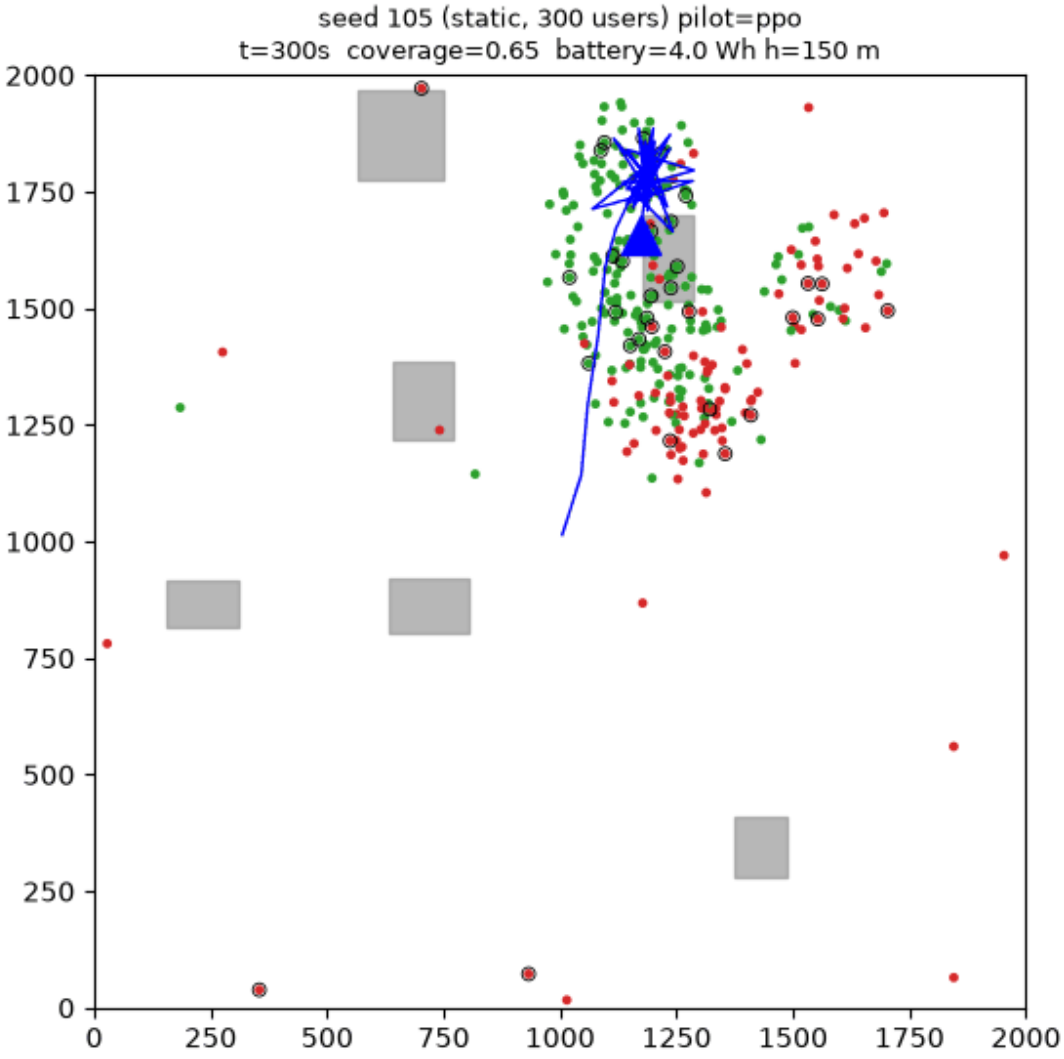


Figure: best_ai_e5a_seed105.gif (last frame of the animation; the GIF is in docs/book/figures/gifs/)

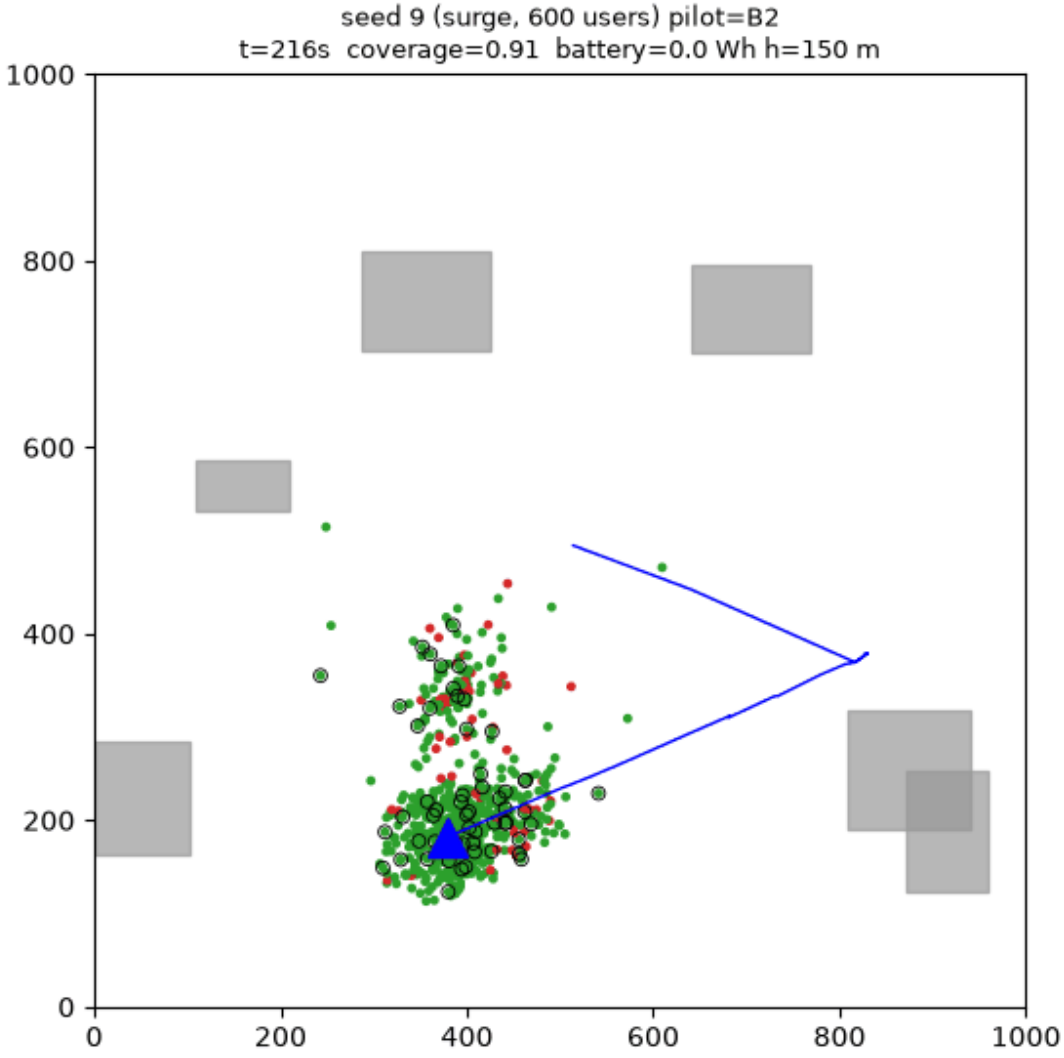


Figure: custom_surge_600users_B2.gif (last frame of the animation; the GIF is in docs/book/figures/gifs/)

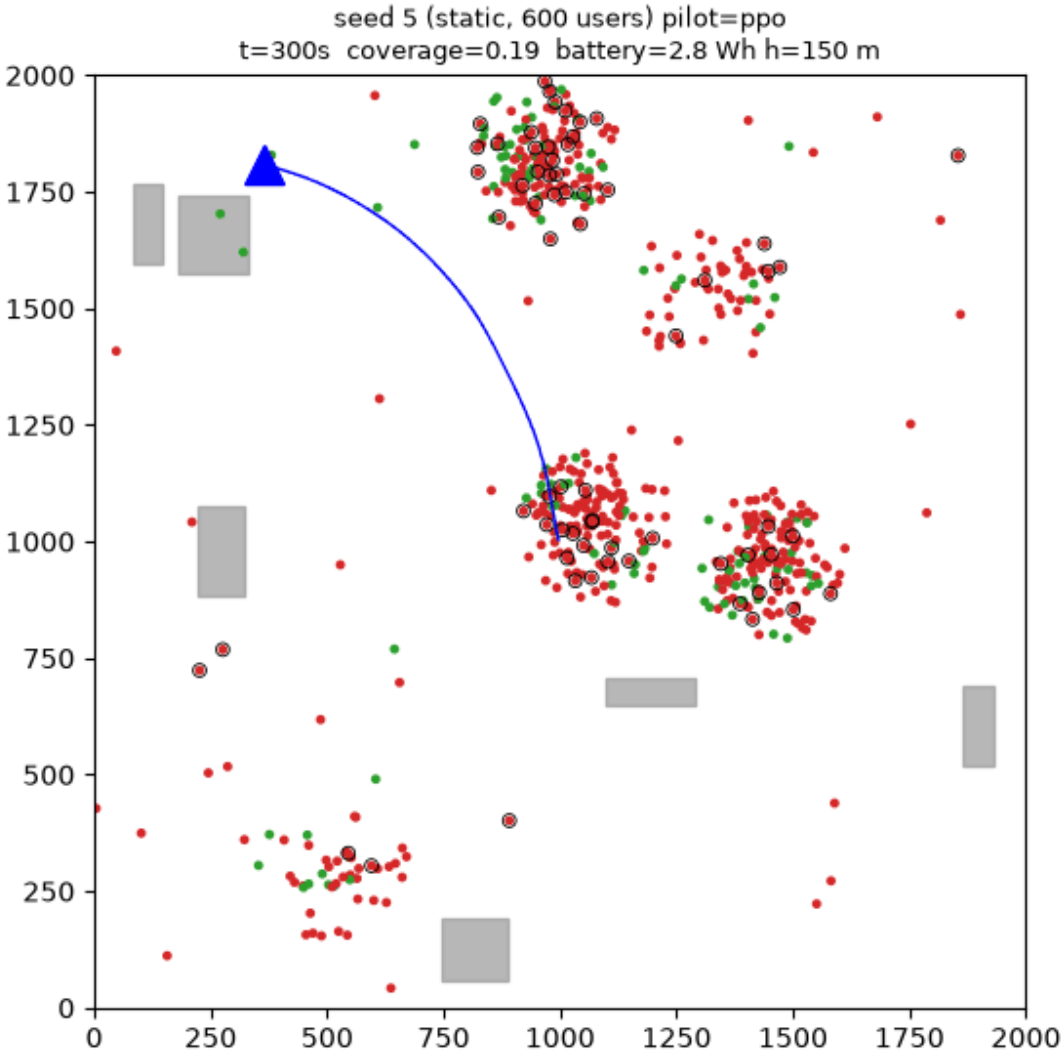


Figure: first_ppo_model_seed5.gif (last frame of the animation; the GIF is in docs/book/figures/gifs/)

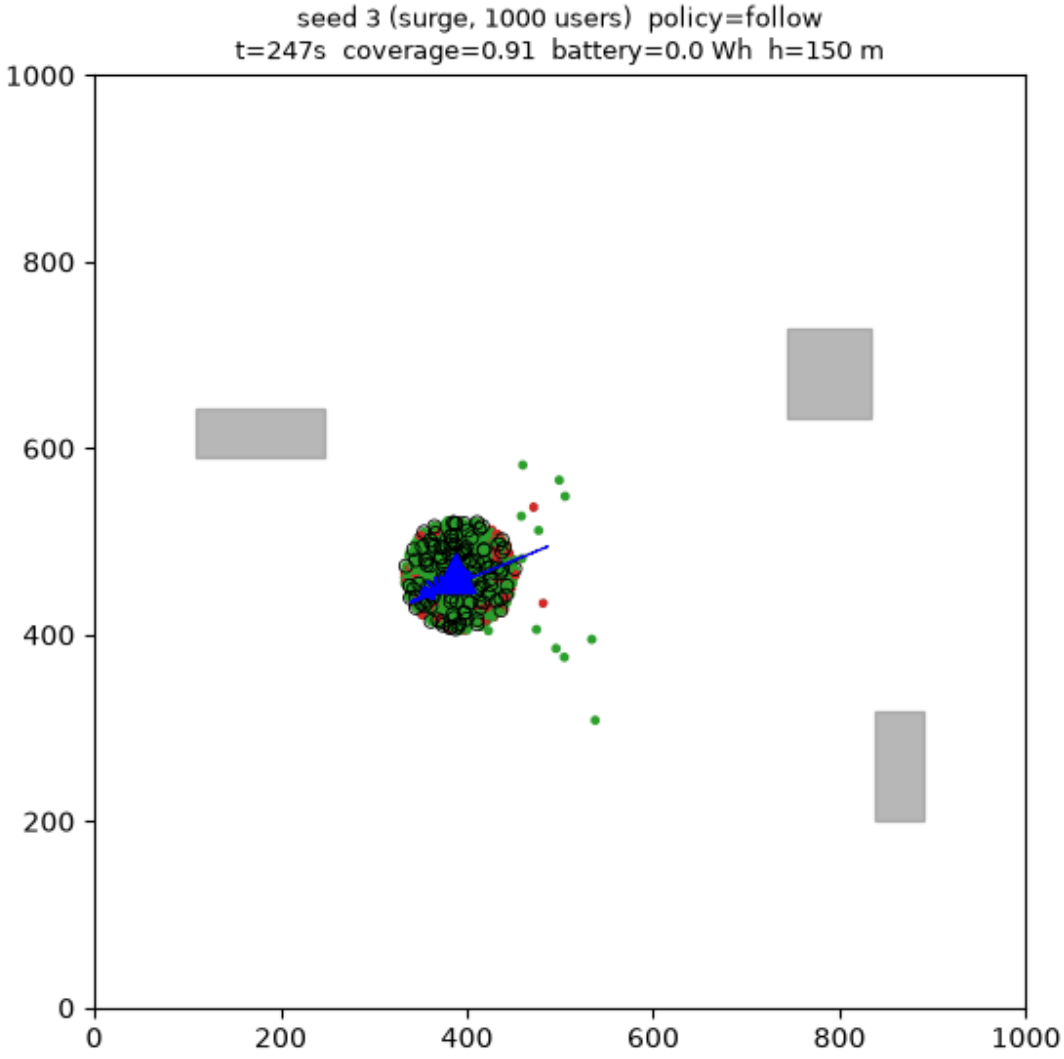


Figure: viewer_first_run_surge_follow.gif (last frame of the animation; the GIF is in docs/book/figures/gifs/)

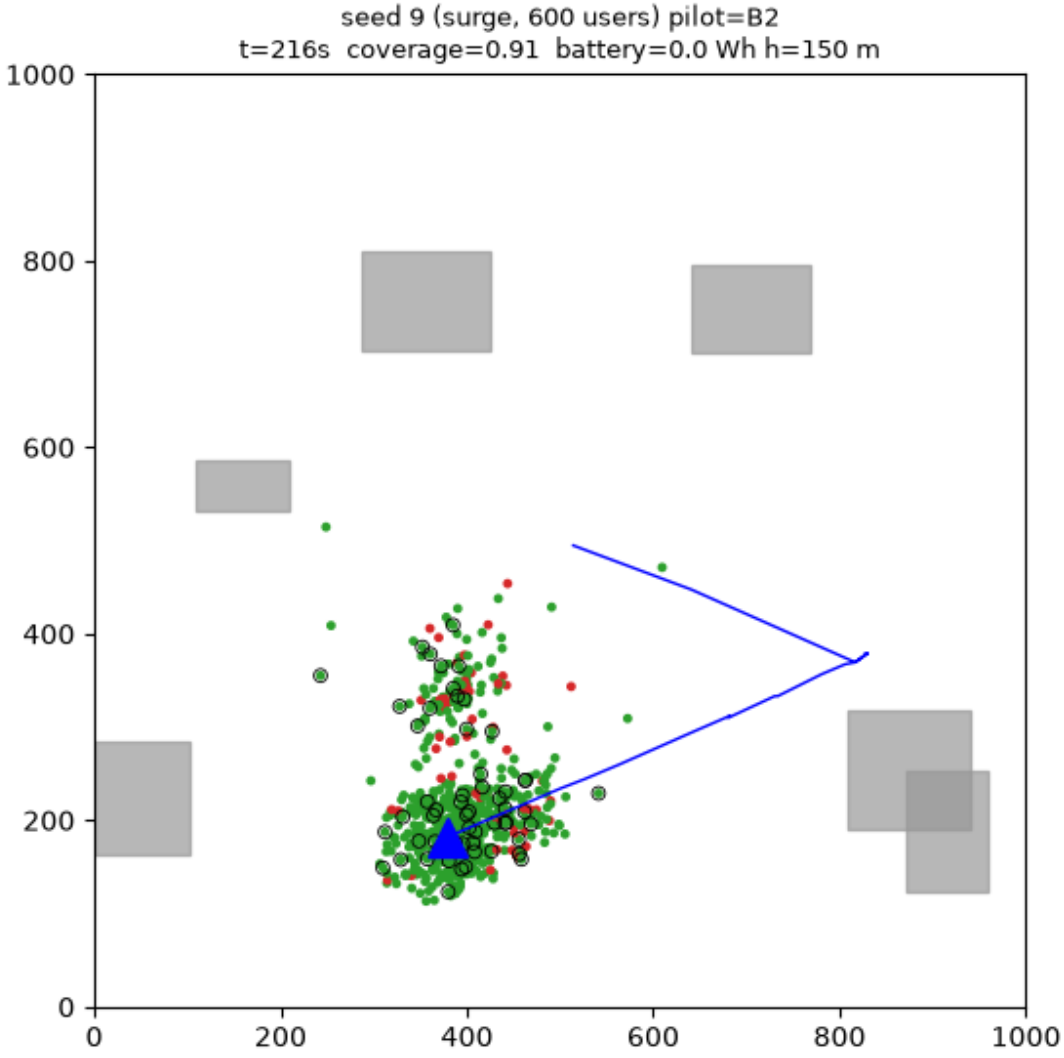


Figure: viewer_custom_scenario.png